UNSW



(ㄴ°□°)ㄴ ┻┻

Miles Conway, Alex Wang, Yiheng You

2026-10-12

1 Contest 1
2 Mathematics 2
3 Data structures 4
4 Numerical 13
5 Number theory 18
6 Combinatorial 21
7 Graph 23
8 Geometry 36
9 Strings 42
10 Heuristics 45
11 Various 47

Contest (1)

template.cpp

28 lines

```cpp
#include <bits/stdc++.h>

using namespace std;
using ll = long long;
using db = double;

#define FOR(i, a, b) for (int i = a; i < (b); i++)
#define F0R(i, b) FOR(i, 0, b)
#define all(x) begin(x), end(x)
#define vt vector
#define size(x) ((int) (x).size())
#define ROF(i, a, b) for (int i = (b) - 1; i >= (a); i--)
#define pb push_back
#define f first
#define s second
using vi = vt<int>;
using vl = vt<ll>;
using pi = pair<int, int>;
using pl = pair<ll, ll>;
const int inf = 1e9;
const ll INF = 1e18;
#define dbg(x) cerr << #x << " = " << (x) << endl

int main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);

}
```

.vimrc

18 lines

```
set cin aw ai is et ts=4 sw=4 sts=4 ttm=50 nu noeb ru cul bs=2 mouse=a
      noswf
sy on | ino <A-[> <Esc>
for k in split('h j k l o') | exe 'ino <A-'.k.'> <C-o>'.k | exe 'nno <
     A-'.k.'> '.k | endfor
" y/d/p use the system clipboard. "E519: Option not supported" on
" startup = vim lacks +clipboard: delete this line
set cb=unnamedplus

" F5: save, compile with sanitizers, run (paste the input, then Ctrl-D
     ):
nno <F5> :w<CR>:!g++ -D_GLIBCXX_DEBUG -Wall -Wextra -Wfatal-errors -
     fsanitize=address,undefined -g -Og % -o %< && ./%<<CR>
nno <F6> :w<CR>:!g++ -D_GLIBCXX_DEBUG -Wall -Wextra -Wfatal-errors -
     fsanitize=address,undefined -g -Og % -o %< && ./%< < in<CR>
" F7: same, then type the input file name and Enter
nno <F7> :w<CR>:!g++ -D_GLIBCXX_DEBUG -Wall -Wextra -Wfatal-errors -
     fsanitize=address,undefined -g -Og % -o %< && ./%< <<Space>

" Select region and then type :Hash to hash your selection.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \| md5sum \|
      cut -c-6

" OPTIONAL:
" set bg=dark  (if the terminal background is dark)
```

hash.sh

7 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
# eg:
# cat sus.cpp | cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum
     |cut -c-6
# a printed "// abc123" in a template is the hash of everything
# up to and including that line (comments don't affect hashes)
```

troubleshoot.txt

70 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.
How sure are you its a precision error and not a logic error?

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Stack overflow? ulimit -s unlimited (check the judge's limit)
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output?
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Stupid bugs:
Did you abs() and type your epsilons correctly?
Is your maximum precomputed value really the maximum?
Is your sorting comparator a strictly weak ordering?
Are your binary search bounds big enough?
Are you reading into a char when it should be a string?
Are your return types big enough?
Are you missing arguments and accidentally defaulting them?
Are you shadowing variables?
Is your Pi correct (atan2 is y, x)

Last ditch attempts:
#define int ll
#define int lll
#define sort stable_sort
```

stress.sh

13 lines

```
#!/usr/bin/env bash
# A and B are executables you want to compare, gen takes int
# as command line arg. Usage: './stress.sh' (bash, not sh)
for((i = 1; ; ++i)); do
    echo $i
    ./gen $i > in
    # Remember to compile!!!
    ./A < in > out1
    ./B < in > out2
    diff -w out1 out2 || break
    # diff -w <(./A < in) <(./B < in) || break
    # ./A < in > out || break
done
```

interactor.sh

18 lines

```
#!/usr/bin/env bash
# Runs JUDGE and SOL, each one's stdout wired to the other's stdin.
# Usage: ./interactor.sh JUDGE SOL [input_file]
#   ./interactor.sh ./judge ./sol in
#   ./interactor.sh "python3 judge.py" ./sol   (quote multi-word cmds)
```

```
# input_file is passed to JUDGE as argv[1]: ifstream fin(argv[1]);
# cin/cout stay the interaction. Both sides must flush after every
# write. A side that hangs is killed after T seconds (rc 124); the
# script exits nonzero if either side does, so in a stress loop:
#   ./gen $i > in; ./interactor.sh J S in || break
# JUDGE's stdout is SOL's stdin: report WA by exit code (+ cerr),
# not by printing. T=5 ./interactor.sh ... raises the time limit.
T=${T:-1}
rm -f pipe; mkfifo pipe
timeout $T $1 $3 < pipe | timeout $T $2 > pipe
r=(${PIPESTATUS[@]}); rm -f pipe
echo "judge rc ${r[0]}, sol rc ${r[1]}"
((r[0] == 0 && r[1] == 0))
```

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b ($\det A \neq 0$).

2.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

2.3 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \\ \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan \frac{v - w}{2} = (V - W) \tan \frac{v + w}{2}$$

where V, W are lengths of sides opposite angles v, w.

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c, opposite angles α, β, γ

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Stable ($a \geq b \geq c$): $A = \frac{1}{4}\sqrt{(a + (b + c))(c - (a - b))(c + (a - b))(a + (b - c))}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Incenter: $\frac{aP_a + bP_b + cP_c}{a+b+c}$, centroid: $\frac{P_a + P_b + P_c}{3}$ (P_a opposite a)

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b + c}\right)^2\right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

Pick (lattice polygon): $A = I + B/2 - 1$ (I interior, B boundary points, $B = \sum \gcd(|dx|, |dy|)$ over edges)

Circle, angle θ: sector $\frac{r^2 \theta}{2}$, segment $\frac{r^2(\theta - \sin\theta)}{2}$, chord $2r \sin \frac{\theta}{2}$

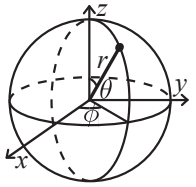
2.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F \tan\theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$, $p = \frac{a+b+c+d}{2}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x &= r \sin\theta \cos\phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin\theta \sin\phi & \theta &= \operatorname{acos}(z/\sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos\theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1 - x^2}} \qquad \frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1 - x^2}}$$

$$\frac{d}{dx} \tan x = 1 + \tan^2 x \qquad \frac{d}{dx} \arctan x = \frac{1}{1 + x^2}$$

Constants omitted; for $f(ax)$ divide the result by a.

$$\int \sin x = -\cos x \qquad \int \cos x = \sin x$$

$$\int \tan x = -\ln|\cos x| \qquad \int \cot x = \ln|\sin x|$$

$$\int \sec x = \ln|\sec x + \tan x| \qquad \int \csc x = \ln|\tan \tfrac{x}{2}|$$

$$\int \sin^2 x = \tfrac{x}{2} - \tfrac{\sin 2x}{4} \qquad \int \cos^2 x = \tfrac{x}{2} + \tfrac{\sin 2x}{4}$$

$$\int \tan^2 x = \tan x - x \qquad \int \cot^2 x = -\cot x - x$$

$$\int \sec^2 x = \tan x \qquad \int \csc^2 x = -\cot x$$

$$\int \sin^3 x = \tfrac{\cos^3 x}{3} - \cos x \qquad \int \cos^3 x = \sin x - \tfrac{\sin^3 x}{3}$$

$$\int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2} \qquad \int x \cos ax = \frac{\cos ax + ax \sin ax}{a^2}$$

$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) \qquad \int x e^{ax} dx = \frac{e^{ax}}{a^2}(ax - 1)$$

Reduction formulas ($n \geq 2$):

$$\int \sin^n x = -\frac{\sin^{n-1} x \cos x}{n} + \frac{n-1}{n} \int \sin^{n-2} x$$

$$\int \cos^n x = \frac{\cos^{n-1} x \sin x}{n} + \frac{n-1}{n} \int \cos^{n-2} x$$

$$\int \tan^n x = \frac{\tan^{n-1} x}{n-1} - \int \tan^{n-2} x$$

$$\int \sec^n x = \frac{\sec^{n-2} x \tan x}{n-1} + \frac{n-2}{n-1} \int \sec^{n-2} x$$

$\int_0^{\pi/2} \sin^n x = \int_0^{\pi/2} \cos^n x = \frac{(n-1)!!}{n!!}$, times $\frac{\pi}{2}$ if n is even.
$\int \sin^m x \cos^n x$: m odd: $u = \cos x$; n odd: $u = \sin x$; both even: use $\sin^2 x = \frac{1-\cos 2x}{2}$, $\cos^2 x = \frac{1+\cos 2x}{2}$. Rational in $\sin, \cos$: $t = \tan \frac{x}{2}$, $\sin x = \frac{2t}{1+t^2}$, $\cos x = \frac{1-t^2}{1+t^2}$, $dx = \frac{2\,dt}{1+t^2}$.

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Vector calculus

$\mathbf{F} = (P, Q[, R])$ is a vector field and $\nabla = (\partial_x, \partial_y[, \partial_z])$, so $\nabla \cdot \mathbf{F}$ is the divergence (net outflow per unit area) and $\nabla \times \mathbf{F}$ the curl ($= Q_x - P_y$ in 2D). Along a curve $d\mathbf{r} = (dx, dy)$, so $\oint \mathbf{F} \cdot d\mathbf{r}$ is the circulation; $\mathbf{n}$ is the outward unit normal, $\mathbf{n}\,ds = (dy, -dx)$ on a CCW curve, so $\oint \mathbf{F} \cdot \mathbf{n}\,ds$ is the outward flux. Both theorems say "what crosses the boundary equals what is generated inside"; pick $\mathbf{F}$ so that one side is trivial.

2.6.1 Change of variables

For $\mathbf{x} = \mathbf{g}(\mathbf{u})$ with Jacobian $J = (\partial x_i / \partial u_j)_{ij}$:

$$\int_{g(U)} f(\mathbf{x})\,d\mathbf{x} = \int_U f(\mathbf{g}(\mathbf{u}))\,|\det J|\,d\mathbf{u}$$

Polar: $|\det J| = r$. Cylindrical: r. Spherical, θ from the z-axis: $r^2 \sin\theta$.

2.6.2 Green's theorem

C a simple closed CCW curve bounding D:

$$\oint_C \mathbf{F} \cdot d\mathbf{r} = \iint_D (\nabla \times \mathbf{F})\,dA$$

Area: $A = \oint_C x\,dy = \frac{1}{2}\oint_C (x\,dy - y\,dx)$, the shoelace formula on a polygon. Stokes, in 3D: $\oint_{\partial S} \mathbf{F} \cdot d\mathbf{r} = \iint_S (\nabla \times \mathbf{F}) \cdot \mathbf{n}\,dS$.

2.6.3 Divergence theorem

$$\oint_C \mathbf{F} \cdot \mathbf{n}\,ds = \iint_D \nabla \cdot \mathbf{F}\,dA$$

$$\iint_{\partial V} \mathbf{F} \cdot \mathbf{n}\,dS = \iiint_V \nabla \cdot \mathbf{F}\,dV$$

Polyhedron volume from its triangles, CCW seen from outside ($\mathbf{F} = \mathbf{x}/3$): $V = \frac{1}{6}\sum \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$.

2.7 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.8 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.9 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

2.9.1 Useful expectations

n i.i.d. U(0, 1): the k-th smallest has mean $\frac{k}{n+1}$. Random permutation: expected cycles = left-to-right maxima = H_n. Coupon collector, n types: nH_n. Probabilities p_i: $\mathbb{E}\max_i T_i = \sum_{S \neq \emptyset} (-1)^{|S|+1} \mathbb{E}\min_{i \in S} T_i = \sum_S (-1)^{|S|+1} / \sum_{i \in S} p_i$. Birthday: k draws from N values collide w.p. $\approx 1 - e^{-k^2/2N}$.

2.9.2 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is Bin(n, p), $n = 1, 2, \ldots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

Bin(n, p) is approximately Po(np) for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is Fs(p), $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is Po(λ), $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

2.9.3 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is U(a, b), $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is Exp(λ), $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ are independent, then

$$aX_1 + bX_2 + c \sim \mathcal{N}(a\mu_1 + b\mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.10 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \ldots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

Stationary distribution

π is a stationary distribution if $\pi = \mathbf{P}\pi$ (note $p_{ij} = \Pr(i \mid j)$: columns sum to 1). If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i. π_j/π_i is the expected number of visits in state j between two visits in state i.

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i's degree.

Ergodicity

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty} \mathbf{P}^k = \pi \mathbf{1}^T$.

Absorption

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j, is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i, is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Random walks

Gambler's ruin on $0..N$, start i, step $+1$ w.p. p, $q = 1 - p$, $\rho = q/p \neq 1$: $\Pr[\text{reach } N] = \frac{1-\rho^i}{1-\rho^N}$, expected duration $\frac{i}{q-p} - \frac{N}{q-p} \cdot \frac{1-\rho^i}{1-\rho^N}$. If $p = \frac{1}{2}$: $\frac{i}{N}$ and $i(N-i)$. Symmetric walk from 0 to $\pm a$: a^2 steps. Simple walk on a graph: hitting time $h_v = 1 + \frac{1}{\deg v} \sum_{u \sim v} h_u$ ($h_t = 0$). Commute time $h_{uv} + h_{vu} = 2mR_{uv}$, with effective resistance (unit resistors) $R_{uv} = \tau(G/uv)/\tau(G) = (e_u - e_v)^T L^+ (e_u - e_v)$, G/uv merging u, v.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change `null_type`. Multiset: store pairs (x, id), never `less_equal`. `find_by_order(k)` is `end()` for $k \geq$ size; `order_of_key(x)` counts elements $< x$.
Time: $\mathcal{O}(\log N)$ 782797, 17 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

// MEGA WARNING: use A.swap(B) instead of swap(A, B)!
template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;

void example() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

Rope.h

Description: Persistent sequence: insert, erase and cut out a block at any index in $O(\log n)$ amortized, 1.5–4μs each (an implicit treap is 3–5x faster, a vector insert is as fast below $n \approx 2 \cdot 10^5$). A copy is $O(1)$, `substr` and + are $O(\log n)$, all sharing nodes, so old versions stay valid: a persistent array is copy, then write, at 2–5KB per version (fine for 10^5, not 10^6). `r[i]` is a value, $O(\log n)$, 0.2–0.6μs at 10^6; a range-for is 2–17ns per step, but any mutation frees `begin()` iterators (`mutable_begin()` ones survive). Landmines: T must not be a std type (pair, string), wrap it in a struct; a literal 0 is ambiguous with the pointer overloads unless T is int, so `Rope<ll>` needs `0LL`; never the one-argument `erase(i)`, it erases $i+1$ elements up to gcc 11.2 and does not compile after.
Time: $\mathcal{O}(\log n)$ amortized 269d3d, 18 lines

```
#include <ext/rope>

template<class T>
using Rope = __gnu_cxx::rope<T>;

void example() {
  Rope<int> r;                    // Rope<int>(a, n) is faster
  F0R (i, 5) r.push_back(i);      // 0 1 2 3 4
  r.insert(1, 9);                 // 0 9 1 2 3 4
  r.erase(2, 3);                  // 0 9 4 (3 elements from 2)
  Rope<int> t = r.substr(1, 2);   // 9 4 (2 elements from 1)
  r = t + r; r.insert(1, t);      // 9 9 4 4 0 9 4
  r.mutable_reference_at(0) = 7;  // assignment: r[0] = 7; t is
      unchanged
  r.replace(0, 7);                // same as above but copies the
      path
  assert(r[0] == 7 && t[0] == 9 && size(r) == 7);
  for (int x : r) cout << x;      // O(1) amortized per step
  t.replace(0, 1, r.substr(3, 2)); // t = 4 0 4
}
```

MonotonicMap.h

Description: Prefix/suffix min/max over keys with insertions, a replacement for sparse segtrees.
Usage: `RangeQuery<less<>, less_equal<>> m; // suffix min`
`m.ins(1e9, INF); // sentinel past all keys`
`m.ins(k, v); m.query(k); // min v over keys >= k`
`prefix: greater<>, sentinel ins(-1e9, INF); max: greater_equal<>, sentinel -INF.`
Time: $\mathcal{O}(\log N)$ 1db6eb, 12 lines

```
template<class dir, class cmp>
struct RangeQuery {
  map<int, ll, dir> data;
  void ins(int k, ll v) {
    if (auto it = data.lower_bound(k); it != data.end() && cmp{}(it->s
        , v)) return;
    auto it = data.insert_or_assign(k, v).f;
    while (it != data.begin() && cmp{}(v, prev(it)->s)) data.erase(
        prev(it));
  }
  ll query(int k) { // inclusive
    return data.lower_bound(k)->s;
  }
};
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided). d77092, 8 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x * C); }
};
// for a set, use __gnu_pbds::null_type as the mapped (2nd) type
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {1 << 16})
    ;
```

ExponentialHash.h

Description: Multiset hash $H(S) = \sum_{x \in S} B^x \bmod p$, $p = 2^{61} - 1$, $B = 37^K$, a primitive root iff $\gcd(K, p-1) = 1$. $\pm$ adds/removes elements, $H(S)H(T)$ hashes all pairwise sums, H·`point(t)` translates by t. `range(a,d,n)`: $\{a, a+d, \ldots, a+(n-1)d\}$. Can do prefix hashes. For vectors, dot against fixed vector of random coprimes: $\prod_i$`point`(x_i, c_i) with c_i coprime to $p-1$, e.g. $10^{11} + \{3, 7, 13, 19, 21, 27\}$. Exponents wrap mod $p-1$. For uniformly random B, c_i: collision prob. $\leq D/\varphi(p-1)$, $\varphi(p-1) \approx 4 \cdot 10^{17}$, D = sum of coordinate spans. Double hash: another K with different c_i.
Usage: `using H = ExpHash<>;`
`H h = H::point(2) + H::point(5); // multiset {2,5}`
`h = h * H::point(-3); // translate to {-1,2}`
`H xy = H::point(x) * H::point(y, ll(1e11)+13);`
Time: $\mathcal{O}(1)$ arithmetic/point; $\mathcal{O}(\log p + \log(n+1))$ range.
Memory: $\mathcal{O}(1)$ per hash; shared compile-time power table.
"../number-theory/ModMulLL.h", "../number-theory/FixedModPow.h" 158507, 38 lines

```
template<ll K = ll(1e11)+3>
struct ExpHash {
  static constexpr ll P = (1LL << 61) - 1;
  static_assert(0 < K && K < P-1 && gcd(K, P-1) == 1);
```

```
  static constexpr ll B = FixedModPow<P,1>(37)(K);
  static constexpr FixedModPow<P> pw{B};
  ll x;
  ExpHash(ll v = 0) : x(v % P) { if (x < 0) x += P; }
  static ExpHash point(ll z, ll c = 1) {
    constexpr ll m = P-1;
    z %= m; if (z < 0) z += m;
    c %= m; if (c < 0) c += m;
    ExpHash r;
    r.x = pw((ull)((__uint128_t)z * c % m)); return r;
  } // 3c3df3
  // faster member operation impls
  ExpHash operator+(ExpHash o) const {
    o.x += x; if (o.x >= P) o.x -= P; return o;
  }
  ExpHash operator-(ExpHash o) const {
    o.x = x-o.x; o.x += o.x >> 63 & P; return o;
  }
  ExpHash operator*(ExpHash o) const {
    o.x = mul(x, o.x); return o;
  }
  bool operator==(ExpHash o) const { return x == o.x; } // f831ee
  static ExpHash range(ll a, ll d, ll n) {
    ExpHash q = point(d);
    return point(a) * (q.x == 1 ? ExpHash(n) :
      ExpHash(mpow(q.x, n, P)-1) *
      ExpHash(mpow(q.x-1, P-2, P)));
  }
  static ll mul(ll a, ll b) { // 0 <= a, b < P; faster
    auto z = (__uint128_t)a * b;
    ll r = ll(z & P) + ll(z >> 61);
    return r >= P ? r - P : r;
  }
};
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.

Time: Both operations are $\mathcal{O}(\log N)$. f2640e, 22 lines

```
struct FT {
  vl s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < size(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos - 1];
    return res;
  }
  int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= size(s) && s[pos + pw - 1] < sum)
        pos += pw, sum -= s[pos - 1];
    }
    return pos;
  }
};
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fake_update() before init()).

Usage: `FT2 f(n); // 0 <= x < n`
`f.fake_update(x, y); // once per point you will update`
`f.init(); f.update(x, y, d); // a[x][y] += d`
`f.query(I, J); // sum of a[x][y] over x < I, y < J`

Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h" ea1296, 26 lines

```
struct FT2 {
  vt<vi> ys; vt<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fake_update(int x, int y) {
    for (; x < size(ys); x |= x + 1) ys[x].pb(y);
  }
  void init() {
    for (vi &v : ys) sort(all(v)), ft.emplace_back(size(v));
  }
  int ind(int x, int y) {
    return lower_bound(all(ys[x]), y) - ys[x].begin();
  }
  void update(int x, int y, ll dif) {
    for (; x < size(ys); x |= x + 1) {
      int i = ind(x, y); // (x, y) must be fake_updated
      assert(i < size(ys[x]) && ys[x][i] == y);
      ft[x].update(i, dif);
    }
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x - 1].query(ind(x - 1, y));
    return sum;
  }
};
```

SegmentTree.h

Description: Generic segment tree for associative operations. `id` is for the identity object.

Time: $\mathcal{O}(\log N)$ 812619, 20 lines

```
struct Segtree {
  int n;
  vt<T> seg;
  void init(int _n) {
    for (n = 1; n < _n; n *= 2);
    seg.resize(2 * n, id);
  }
  void upd(int i, T v) {
    seg[i += n] = v;
    while (i /= 2) seg[i] = seg[2 * i] + seg[2 * i + 1];
  }
  T query(int l, int r) {
    T lhs = id, rhs = id;
    for (l += n, r += n; l < r; l /= 2, r /= 2) {
      if (l & 1) lhs = lhs + seg[l++];
      if (r & 1) rhs = seg[--r] + rhs;
    }
    return lhs + rhs;
  }
};
```

LazySegtree.h

Description: Generic lazy segment tree

Usage: `Implement +, upd and id for Node object; += and id for Lazy object.`

Time: $\mathcal{O}(\log N)$. c33fda, 66 lines

```
struct Lazy {
  ll v;
  bool inc;
  void operator+=(const Lazy &b) {
    if (b.inc) v += b.v;
    else v = b.v, inc = false;
  }
};

struct Node {
  ll mx, sum;
  Node operator+(const Node &b) {
    return {max(mx, b.mx), sum + b.sum};
  }
  void upd(const Lazy &u, int l, int r) {
    if (!u.inc) mx = sum = 0;
    mx += u.v, sum += u.v * (r - l);
  }
};

const Lazy lid = {0, true};
const Node nid = {-INF, 0};

const int sz = 1 << 18;
struct Lazyseg {
  vt<Node> seg;
  vt<Lazy> lazy;
  // careful: max is initialised to -1e18; perform range set or write
       build function
  Lazyseg() : seg(2 * sz, nid), lazy(2 * sz, lid) {}
  void push(int i, int l, int r) {
    seg[i].upd(lazy[i], l, r);
    if (r - l > 1) F0R (j, 2) lazy[2 * i + j] += lazy[i];
    lazy[i] = lid;
  }
  void upd(int lo, int hi, Lazy val, int i = 1, int l = 0, int r = sz)
       {
    push(i, l, r);
    if (r <= lo || l >= hi) return;
    if (lo <= l && r <= hi) {
      lazy[i] += val;
      push(i, l, r);
      return;
    }
    int m = (l + r) / 2;
    upd(lo, hi, val, 2 * i, l, m);
    upd(lo, hi, val, 2 * i + 1, m, r);
    seg[i] = seg[2 * i] + seg[2 * i + 1];
  }
  Node query(int lo, int hi, int i = 1, int l = 0, int r = sz) {
    push(i, l, r);
    if (r <= lo || l >= hi) return nid;
    if (lo <= l && r <= hi) return seg[i];
    int m = (l + r) / 2;
    return query(lo, hi, 2 * i, l, m)
      + query(lo, hi, 2 * i + 1, m, r);
  }
  // valid before any upd (then build()); after, query(i, i + 1)
  Node& operator[] (int i) {
    return seg[i + sz];
  }
  // void pull(int i) {
  //   seg[i] = seg[2 * i] + seg[2 * i + 1];
```

```
  // }
  // void build() {
  //   for (int i = sz - 1; i > 0; i--) pull(i);
  // }
};
```

RectangleUnion.h

Description: Area of the union of rectangles $[x_1, x_2) \times [y_1, y_2)$ with $y \in [0, \texttt{MY})$ (compress first).
Usage: `vt<array<int, 4>> r = {{x1, y1, x2, y2}, ...}; ll A = rect_union(r);`
Time: $\mathcal{O}(N \log N)$ 224598, 25 lines

```
const int MY = 1 << 20; // y range, power of two
int cnt[2 * MY], len[2 * MY];
void upd(int a, int b, int v, int i = 1,
    int l = 0, int r = MY) {
  if (b <= l || r <= a) return;
  if (a <= l && r <= b) cnt[i] += v;
  else {
    int m = (l + r) / 2;
    upd(a, b, v, 2 * i, l, m), upd(a, b, v, 2 * i + 1, m, r);
  }
  if (cnt[i]) len[i] = r - l;
  else len[i] = r - l == 1 ? 0 : len[2 * i] + len[2 * i + 1];
}
ll rect_union(vt<array<int, 4>> R) {
  vt<array<int, 4>> ev; // x, +-1, y1, y2
  for (auto [x1, y1, x2, y2] : R) if (x1 < x2 && y1 < y2)
    ev.pb({x1, 1, y1, y2}), ev.pb({x2, -1, y1, y2});
  sort(all(ev));
  ll res = 0;
  F0R (i, size(ev)) {
    if (i) res += (ll) len[1] * (ev[i][0] - ev[i - 1][0]);
    upd(ev[i][2], ev[i][3], ev[i][1]);
  }
  return res;
}
```

SparseSegtree.h

Description: Generic-ish sparse segment tree (point update, range query).
Usage: `ptr t = new Node{INF}; t->upd(i, v); t->query(l, r); upd assigns, query is [l, r), 0 <= i < sz; the root value is ID.`
Time: $\mathcal{O}(\log N)$.
Memory: $\log_2 sz$ nodes per upd. 297a92, 28 lines

```
using ptr = struct Node*;
const int sz = 1 << 30;
struct Node {
  #define func(a, b) min(a, b)
  #define ID INF
  ll val;
  ptr lc, rc;

  ptr get(ptr& p) { return p ? p : p = new Node {ID}; }

  ll query(int lo, int hi, int l = 0, int r = sz) {
    if (lo >= r || hi <= l) return ID;
    if (lo <= l && r <= hi) return val;
    int m = (l + r) / 2;
    return func(lc ? lc->query(lo, hi, l, m) : ID,
      rc ? rc->query(lo, hi, m, r) : ID);
  }

  ll upd(int i, ll nval, int l = 0, int r = sz) {
    if (r - l == 1) return val = nval;
    int m = (l + r) / 2;
    if (i < m) get(lc)->upd(i, nval, l, m);
    else get(rc)->upd(i, nval, m, r);
    return val = func(lc ? lc->val : ID, rc ? rc->val : ID);
  }
  #undef ID
  #undef func
};
```

PersistentSegtree.h

Description: Generic-ish persistent segment tree (point update, range query).
Usage: `edit func/ID. ptr r = new Node{inf}; r->lc = r->rc = r; // all-ID r = r->upd(i, v); r->query(l, r) on [l, r).`
Time: $\mathcal{O}(\log N)$.
Memory: $\log_2 sz + 1$ nodes per upd, never freed. cfa1a3, 30 lines

```
using ptr = struct Node*;
const int sz = 1 << 18;

struct Node {
  #define func(a, b) min(a, b)
  #define ID inf
  int v;
  ptr lc, rc;

  ptr pull(ptr lc, ptr rc) {
    return new Node {func(lc->v, rc->v), lc, rc};
  }

  ptr upd(int i, int nv, int l = 0, int r = sz) {
    if (r - l == 1) return new Node {nv};
    int m = (l + r) / 2;
    if (i < m) return pull(lc->upd(i, nv, l, m), rc);
    else return pull(lc, rc->upd(i, nv, m, r));
  }

  int query(int lo, int hi, int l = 0, int r = sz) {
    if (lo >= r || hi <= l) return ID;
    if (lo <= l && r <= hi) return v;
    int m = (l + r) / 2;
    return func(lc->query(lo, hi, l, m),
      rc->query(lo, hi, m, r));
  }
  #undef ID
  #undef func
};
```

PersistentLazySegtree.h

Description: Persistent lazy segment tree. Lazy/Node as in LazySegtree; Node::upd may only use r - l. Query allocates nothing.
Usage: `ptr t = new Seg{{0, 0}, lid}; t->lc = t->rc = t; // all-0 t = t->upd(l, r, {x, true}); t->query(l, r) on [l, r).`
Time: $\mathcal{O}(\log N)$.
Memory: $< 4 \log_2 sz$ nodes per upd, never freed. 8c23bd, 65 lines

```
struct Lazy {
  ll v;
  bool inc;
  void operator+=(const Lazy &b) {
    if (b.inc) v += b.v;
    else v = b.v, inc = false;
  }
};

struct Node {
  ll mx, sum;
  Node operator+(const Node &b) {
    return {max(mx, b.mx), sum + b.sum};
  }
  void upd(const Lazy &u, int l, int r) {
    if (!u.inc) mx = sum = 0;
    mx += u.v, sum += u.v * (r - l);
  }
};

const Lazy lid = {0, true};
const Node nid = {-INF, 0};

using ptr = struct Seg*;
const int sz = 1 << 18;
struct Seg {
  Node v;
  Lazy lz;
  ptr lc, rc;

  ptr apply(const Lazy &t, int l, int r) { // copy, then tag
    ptr p = new Seg(*this);
    p->v.upd(t, l, r), p->lz += t;
    return p;
  }

  // t: tag pushed from the parent, older than u
  ptr upd(int lo, int hi, const Lazy &u, Lazy t = lid,
      int l = 0, int r = sz) {
    if (r <= lo || l >= hi) return apply(t, l, r);
    if (lo <= l && r <= hi) return t += u, apply(t, l, r);
    int m = (l + r) / 2;
    Lazy p = lz; p += t;
    ptr a = lc->upd(lo, hi, u, p, l, m);
    ptr b = rc->upd(lo, hi, u, p, m, r);
    return new Seg{a->v + b->v, lid, a, b};
  }

  Node query(int lo, int hi, int l = 0, int r = sz) {
    if (r <= lo || l >= hi) return nid;
    if (lo <= l && r <= hi) return v;
    int m = (l + r) / 2;
    Node res = lc->query(lo, hi, l, m)
      + rc->query(lo, hi, m, r);
    res.upd(lz, max(l, lo), min(r, hi)); // lz is newest
    return res;
  }
};
// leaves a[0..sz), 2 sz nodes, no self-loop root needed
// ptr build(vt<Node> &a, int l = 0, int r = sz) {
//   if (r - l == 1) return new Seg{a[l], lid};
//   int m = (l + r) / 2;
//   ptr x = build(a, l, m), y = build(a, m, r);
//   return new Seg{x->v + y->v, lid, x, y};
// }
```

LiChaoTree.h

Description: LiChao tree: min of lines at integer $x \in [0, sz)$; empty tree gives INF.
Usage: `ptr root = 0; add(root, {m, c}); query(root, x).` Max: add {-m, -c}, negate.
Time: $\mathcal{O}(\log N)$. c7893a, 31 lines

```
struct Line {
  ll m, c;
  ll operator()(ll x) {
    return m * x + c;
  }
};

const ll sz = 1ll << 30;

using ptr = struct Node*;
```

```
struct Node {
  Line line;
  ptr lc, rc;
};

// min tree (flip signs for max)
void add(ptr& n, Line loser, ll l = 0, ll r = sz) {
  if (n ? 0 : n = new Node{loser}) return;
  ll m = (l + r) / 2;
  if (loser(m) < n->line(m)) swap(loser, n->line);
  if (r - l == 1) return;
  if (loser(l) < n->line(l)) add(n->lc, loser, l, m);
  else add(n->rc, loser, m, r);
}

ll query(ptr n, ll x, ll l = 0, ll r = sz) {
  if (!n) return INF;
  ll m = (l + r) / 2;
  if (x < m) return min(n->line(x), query(n->lc, x, l, m));
  else return min(n->line(x), query(n->rc, x, m, r));
}
```

SparseTable.h

Description: Generic sparse table for idempotent operations.
Usage: `RMQ<int> t; t.init(v); t.query(l, r) on [l, r), r > l; edit the func define (min).`
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(1)$ query.
Memory: $\mathcal{O}(N \log N)$, 160MB for ll at N = 1e6. def0a7, 19 lines

```
template<class T> struct RMQ {
  #define func min
  vt<vt<T>> dp;
  void init(const vt<T>& v) {
    assert(size(v));
    dp.resize(__lg(size(v)) + 1, vt<T>(size(v)));
    copy(all(v), begin(dp[0]));
    for (int j = 1; 1 << j <= size(v); ++j) {
      for (int i = 0; i <= size(v) - (1 << j); i++)
        dp[j][i] = func(dp[j - 1][i],
          dp[j - 1][i + (1 << (j - 1))]);
    }
  }
  T query(int l, int r) { // [l, r), r > l
    int d = __lg(r - l);
    return func(dp[d][l], dp[d][r - (1 << d)]);
  }
  #undef func
};
```

FunnySparseTable.h

Description: Generic sparse table for idempotent operations.
Usage: `init(v); query(l, r) on [l, r), r > l.`
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(1)$ query. 685050, 15 lines

```
template<class T> struct RMQ {
  vt<vt<T>> dp;
  T query(int l, int r) { // [l, r)
    int d = __lg(r - l - 1 | 1);
    return min(dp[d][l], dp[d][r - (1 << d)]);
  }
  void init(const vt<T>& v) {
    dp = {v};
    for (int w = 2; w <= size(v); w += w) {
      dp.emplace_back(size(v) - w + 1);
      for (int i = 0; i + w <= size(v); i++)
        dp.back()[i] = query(i, i + w);
    }
  }
};
```

StaticRangeQuery.h

Description: Generic static range query for associative operations. Queries are half-open: query(l, r) folds over $[l, r)$.
Usage: `edit comb and id (the identity); build(all(v)); query(l, r) needs r > l.`
Time: $\mathcal{O}(N \log N)$ build, $\mathcal{O}(1)$ query.
Memory: $\mathcal{O}(N \log N)$, about 220MB for $N = 2^{20}$ with 8-byte T. 10e422, 34 lines

```
template<class T> struct RangeQuery {
  #define comb(a, b) (a) + (b)
  #define id 0
  int lg, n;
  vt<vt<T>> stor;
  vt<T> a;
  void fill(int l, int r, int ind) {
    if (ind < 0) return;
    int m = (l + r) / 2;
    T prod = id;
    FOR (i, m, r) stor[i][ind] = prod = comb(prod, a[i]);
    prod = id;
    ROF (i, l, m) stor[i][ind] = prod = comb(a[i], prod);
    fill(l, m, ind - 1);
    fill(m, r, ind - 1);
  }
  template <typename It>
  void build(It l, It r) {
    lg = 1;
    while ((1 << lg) < r - l) lg++;
    n = 1 << lg;
    a.resize(n, id);
    for (It i = l; i != r; i++) a[i - l] = *i;
    stor.resize(n, vt<T>(lg + 1));
    fill(0, n, lg - 1);
  }
  T query(int l, int r) { // [l, r)
    if (r-- - l == 1) return a[l];
    int t = __lg(l ^ r);
    return comb(stor[l][t], stor[r][t]);
  }
  #undef id
  #undef comb
};
```

UnionFind.h

Description: Disjoint-set data structure.
Time: $\mathcal{O}(\alpha(N))$ 9211be, 14 lines

```
struct UF {
  vi e;
  UF(int n) : e(n, -1) {}
  bool sameSet(int a, int b) { return find(a) == find(b); }
  int sz(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

Matrix.h

Description: Basic operations on square matrices.
Usage: `Matrix<int, 3> A;`
`A.d = {{{1,2,3}}, {{4,5,6}}, {{7,8,9}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;` 70f60f, 26 lines

```
template<class T, int N> struct Matrix {
  using M = Matrix;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    F0R (i, N) F0R (j, N)
      F0R (k, N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    F0R (i, N) F0R (j, N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    F0R (i, N) a.d[i][i] = 1;
    while (p) {
      if (p & 1) a = a * b;
      b = b * b;
      p >>= 1;
    }
    return a;
  }
};
```

LineContainer.h

Description: Container where you can add lines of the form mx+c, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").
Usage: `add(m, c); query(x) = max m*x+c (x integer). Min: add(-m, -c), negate.`
Time: $\mathcal{O}(\log N)$ 191376, 30 lines

```
struct Line {
  mutable ll m, c, p;
  bool operator<(const Line& o) const { return m < o.m; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->m == y->m) x->p = x->c > y->c ? inf : -inf;
    else x->p = div(y->c - x->c, x->m - y->m);
    return x->p >= y->p;
  } // ce5124
  void add(ll m, ll c) {
    auto z = insert({m, c, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.m * x + l.c;
```

```
  }
};
```

KineticTournament.h

Description: seg[1] is the point minimising $p \cdot t$ while the direction t sweeps CCW from t0 to tn, less than a half turn (asserted). heaten advances t (never back), upd moves a point, fine mid-sweep. mt[i] is the first direction at which node i's winner can change; a subtree is rebuilt only once t reaches it. For argmax flip the $<$ in combine and use (p - q).perp() for e. Needs Point's radial $<$ and a sweep inside angles $[0, 180)$ (rotate points and t by perp() to fit). Lines $ax + b$ at increasing x: point $(-a, b)$, t0 $(B, 1)$, heaten $(-x, 1)$, tn $(-B, 1)$. Safe for $|coord|, |t| \leq 10^9$ (a cross of two certificates reaches $8 \cdot 10^{18}$); Point<__int128> beyond.
Time: $\mathcal{O}(\log n)$ per upd. A node's winner changes at most (subtree size) times per sweep, so $\mathcal{O}(n \log n)$ rebuilds, each walking a root path: $\mathcal{O}(n \log^2 n)$ per sweep, i.e. $\mathcal{O}(\log^2 n)$ amortized per heaten.

"../geometry/Point.h" 147bce, 23 lines

```
using P = Point<ll>; // radial <; __int128 if cross overflows
struct KineticTournament {
  int n; vt<P> seg, mt; P t, tn;
  KineticTournament(vt<P> a, P t0, P tn)
    : n(size(a)), seg(2 * n), mt(2 * n, tn), t(t0), tn(tn) {
    assert(t0.cross(tn) > 0); // CCW turn < 180
    copy(all(a), n + begin(seg));
    ROF (i, 1, n) combine(i);
  }
  void combine(int i) { // rebuild i from its children
    P p = seg[2 * i], q = seg[2 * i + 1];
    if (q.dot(t) < p.dot(t)) swap(p, q); // p = argmin
    seg[i] = p, mt[i] = min(mt[2 * i], mt[2 * i + 1]);
    P e = (q - p).perp(); // where p, q tie; ahead of t
    if (!(tn < e)) mt[i] = min(mt[i], e);
  }
  void pull(int i) { // recurse into expired subtrees
    if (i >= n || t < mt[i]) return;
    pull(2 * i), pull(2 * i + 1), combine(i);
  }
  void upd(int i, P p) { for (seg[i+=n] = p; i /= 2; combine(i)); }
  void heaten(P nt) { assert(!(nt < t)); t = nt, pull(1); }
};
```

MonotonicCHT.h

Description: Convex hull trick when slopes are added non-decreasing and queries come at non-decreasing x, both amortized $O(1)$ (LineContainer is the general case). A line (m, c) is a point, a query is the dot with $(x, 1)$, h is the upper hull in slope order and p walks it. A line with the tail's slope replaces it if higher and is dropped otherwise, so the cross test never sees equal slopes, the one case it cannot decide. Returns the max; for the min negate m, c and the result. The cross is $\Delta m \cdot \Delta c$: if that can exceed $9 \cdot 10^{18}$ use Point<__int128>.
Usage: add(m, c), m non-decreasing; query(x), x non-decreasing, = max mx+c.
Time: $\mathcal{O}(1)$ amortized

"../geometry/Point.h" 418067, 18 lines

```
using P = Point<ll>; // __int128 if dm * dc overflows
struct MonotonicCHT {
  vt<P> h; int p = 0;
  void add(ll m, ll c) {
    P l{m, c};
    if (size(h) && h.back().x == m && h.back().y >= c) return;
    while (size(h) && (h.back().x == m || (size(h) > 1 &&
        end(h)[-2].cross(h.back(), l) >= 0)))
      h.pop_back();
    h.pb(l), p = min(p, size(h) - 1);
  }
  ll query(ll x) {
    assert(size(h)); P d{x, 1};
    while (p + 1 < size(h) && h[p+1].dot(d) > h[p].dot(d))
      p++;
    return h[p].dot(d);
  }
};
```

IncrementalMST.h

Description: Incremental minimum spanning forest keyed by weight.f (globally distinct). Interface: merge(u, v, w) inserts edge (u, v) and returns the edge that is now NOT in the forest (inf, -1 if none was displaced); find(u, w) = representative of u's component using only edges of weight $\leq w$ (default: all), so connectivity "as of" any threshold is a query, no deletions needed; max_edge(u, v) = vertex p whose parent edge (weight[p], to par[p]) is the heaviest on the u–v path (-1 if disconnected); delete_max_edge(u, v, w) removes the edge of weight w, valid only if w $\geq$ the weight of every live edge, including edges merge dropped. Offline dynacon: weight = -deletion time, answer queries at time t with find(u, -t-1) == find(v, -t-1).
Usage: DSU d(n); d.merge(u, v, {w, id}); // w < inf
Time: $\mathcal{O}(\log n)$ expected

55ac3e, 86 lines

```
struct DSU {
  vi par, pri;
  vt<pi> weight;

  DSU(int n): par(n), pri(n), weight(n) {
    for (int i = 0; i < n; ++i) {
      par[i] = pri[i] = i;
      weight[i] = pi{inf, -1};
    }
    shuffle(all(pri), mt19937(random_device{}()));
  } // 09d59d

  int parent(int u) {
    if (par[u] == u) return par[u];
    while (weight[par[u]].f <= weight[u].f) {
      par[u] = par[par[u]];
    }
    return par[u];
  }

  int find(int u, int w = inf - 1) {
    while (weight[u].f <= w) u = parent(u);
    return u;
  } // 610136

  int connect(int v, int w = inf - 1) {
    while (weight[v].f <= w) {
      v = par[v];
    }
    return v;
  }

  void add_edge(int u, int v, pi w) { // link two components
    while (u != v) {
      u = connect(u, w.f);
      v = connect(v, w.f);
      if (pri[u] < pri[v]) swap(u, v);
      swap(par[v], u);
      swap(weight[v], w);
    }
    connect(u);
  } // bdf117

  int max_edge(int u, int v) {
    if (find(u) != find(v)) return -1;
    while (true) {
      if (weight[u].f > weight[v].f) swap(u, v);
      if (par[u] == v) break;
      u = par[u];
    }
    return u;
  } // 4d639c

  void delete_edge(int v, int w) {
    while (par[v] != v) {
      if (weight[v].f == w) {
        par[v] = v;
        weight[v] = {inf, -1};
        return;
      }
      v = parent(v);
    }
  }

  // delete edge (u, v) of weight w; w must be the global max
  void delete_max_edge(int u, int v, int w) {
    delete_edge(u, w);
    delete_edge(v, w);
  } // 57b721

  // return weight of deleted edge; {inf, -1} otherwise
  pi merge(int u, int v, pi w) {
    if (u == v) return w;
    int p = max_edge(u, v);
    if (p == -1) {
      add_edge(u, v, w);
      return {inf, -1};
    } else if (weight[p].f > w.f) {
      pi res = weight[p];
      delete_edge(p, res.f);
      add_edge(u, v, w);
      return res;
    }
    return w;
  }
};
```

DynaconMST.h

Description: Offline dynamic connectivity via IncrementalMST: an edge gets weight $-$(deletion time), so at time t exactly the edges with weight $\leq -t-1$ are alive and deletions cost nothing. Events are processed in time order; simple graph (no parallel edges).
Usage: ev[t] = {0 add / 1 del / 2 query, u, v}, 0 <= u, v < n; each del matches a live add. Returns one bool per query, in order: u, v connected.
Time: $\mathcal{O}((N + Q) \log N)$ expected

"IncrementalMST.h" 6e418f, 18 lines

```
vt<bool> dynacon(int n, vt<array<int, 3>> &ev) {
  int q = size(ev);
  map<pi, int> at; vi del(q, q); vt<bool> res;
  F0R (t, q) { // pass 1: deletion time of each added edge
    auto [ty, u, v] = ev[t];
    if (u > v) swap(u, v);
    if (ty == 0) at[{u, v}] = t;
    if (ty == 1) del[at[{u, v}]] = t, at.erase({u, v});
  }
  DSU dsu(n);
  F0R (t, q) {
    auto [ty, u, v] = ev[t];
    if (ty == 0) dsu.merge(u, v, {-del[t], t});
    if (ty == 2)
      res.pb(dsu.find(u, -t - 1) == dsu.find(v, -t - 1));
  }
  return res;
}
```

DSURollback.h

Description: DSU with rollbacks.
Time: $\mathcal{O}(\log(N))$ 5507c7, 24 lines

```
struct DSU {
  vi e, stk;
  vt<pi> upds;
  DSU(int n) : e(n, -1) {}
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int unite(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return 0;
    if (e[x] < e[y]) swap(x, y);
    upds.pb({x, e[x]});
    e[y] += e[x];
    e[x] = y;
    return 1;
  }
  void push() { stk.pb(size(upds)); }
  void pop() {
    ROF (i, stk.back(), size(upds)) {
      auto [x, sz] = upds[i];
      e[e[x]] -= sz, e[x] = sz;
    }
    upds.resize(stk.back());
    stk.pop_back();
  }
};
```

Dynacon.h

Description: Dynamic connectivity; answers are component counts (for u–v connectivity queries see DynaconMST).
Usage: `init(n, q)` with `q >=` total `toggle()` + `query()` calls (each consumes one time slot). `toggle(u, v)` adds or removes an edge, `query()` records the component count, `ans()` returns all recorded answers in order.
Time: $\mathcal{O}(N \log^2 N)$ "DSURollback.h" dd841a, 37 lines

```
struct DynaCon {
  int n, q, t = 0;
  vt<vt<pi>> seg;
  map<pi, int> eds;
  DSU dsu{0};
  void init(int _n, int _q) {
    for (q = 1; q < _q; q *= 2);
    seg.resize(2 * q);
    dsu = DSU(n = _n);
  }
  void toggle(int u, int v, bool erase = true) {
    if (u > v) swap(u, v);
    if (eds.count({u, v})) {
      for (int l = eds[{u, v}] + q, r = min(++t + q, 2 * q); l < r; l
           /= 2, r /= 2) {
        if (l & 1) seg[l++].pb({u, v});
        if (r & 1) seg[--r].pb({u, v});
      }
      if (erase) eds.erase({u, v});
    } else eds[{u, v}] = t++;
  }
  void query() { assert(t < q); seg[q + t++].pb({-1, -1}); }
  void dfs(int i, vi &ans) {
    dsu.push();
    for (auto [u, v] : seg[i]) {
      if (u == -1) ans.pb(n - size(dsu.upds));
      else dsu.unite(u, v);
    }
    if (i < q) dfs(2 * i, ans), dfs(2 * i + 1, ans);
    dsu.pop();
  }
  vi ans() {
    for (auto [u, v] : eds) toggle(u.f, u.s, false);
    vi res;
    dfs(1, res);
    return res;
  }
};
```

Trie.h

Description: maintains a multiset of ints in $[0, 2^{lg})$ with queries related to xor. All lazy/push lines are optional: delete them if you don't need global xor.
Usage: `Node t{}; t.ins(x) (1 if new); t.del(x); t.qmin(x); t.count<0>(x, k) = #y with x^y < k (<1>: >); t.lazy ^= v; mex() needs ins-only tries. merge(a, b) with ptr a, b; b is consumed.`
Time: $\mathcal{O}(lg)$
Memory: 24 B/node, $\leq$ lg nodes per element, never freed. b58173, 76 lines

```
const int lg = 30;
using ptr = struct Node *;
struct Node {
  int cnt = 0, lazy = 0; // root->lazy ^= v xors all elements
  ptr _c[2] = {};
  ptr &c(int i) { return _c[i] ?: _c[i] = new Node {}; }

  int ins(int x, int i = lg) {
    push(i);
    return (i-- ? c(1 & x >> i)->ins(x, i) : !cnt) ? ++cnt : 0;
  }

  void multi_ins(int x, int i = lg) {
    push(i);
    if (i--) c(1 & x >> i)->multi_ins(x, i);
    cnt++;
  }

  int del(int x, int i = lg) { // wastes a bit of mem
    push(i);
    return (i-- ? c(1 & x >> i)->del(x, i) : cnt) ? cnt-- : 0;
  }

  // min of x ^ y over the set (nonempty)
  int qmin(int x, int i = lg) {
    push(i);
    if (!i--) return 0;
    int b = 1 & x >> i;
    return c(b)->cnt ? c(b)->qmin(x, i) :
      c(!b)->qmin(x, i) | (1 << i);
  }

  // max of x ^ y over the set (nonempty)
  int qmax(int x, int i = lg) {
    push(i);
    if (!i--) return 0;
    int b = 1 & ~x >> i;
    return c(b)->cnt ? c(b)->qmax(x, i) | (1 << i) :
      c(!b)->qmax(x, i);
  }

  // count y in n such that:
  template<int sgn> // sgn = 0 for x ^ y < k, sgn = 1 for x ^ y > k
  int count(int x, int k, int i = lg) {
    push(i);
    if (!i-- || !cnt) return 0;
    int b = 1 & (x ^ k) >> i;
    return ((1 & k >> i) ^ sgn ? c(!b)->cnt : 0)
      + c(b)->count<sgn>(x, k, i);
  }

  inline void push(int i) { // lazy xor
    if (i && 1 & lazy >> (i - 1)) swap(_c[0], _c[1]);
    for (int j = 2; j--; _c[j] && (_c[j]->lazy ^= lazy));
    lazy = 0;
  }

  int mex(int i = lg) { // ins-only tries
    if (!i) return 0;
    push(i);
    int b = c(0)->cnt == (1 << --i);
    return c(b)->mex(i) + (b << i);
  }
};

// multiset merge; for set semantics use l->cnt |= r->cnt at the leaf
int merge(ptr &l, ptr r, int i = lg) {
  if (!l) swap(l, r);
  if (!l) return 0;
  if (!r) return l->cnt;
  if (!i) return l->cnt += r->cnt;
  l->push(i), r->push(i);
  l->cnt = 0;
  F0R (j, 2) l->cnt += merge(l->_c[j], r->_c[j], i - 1);
  return l->cnt;
}
```

CaterpillowTree.h

Description: 64-ary set of ints in $[0, sz)$, $sz = 2^{18}$ (raise depth for more).
Usage: `Tree t; t.insert(x); t.erase(x); t[x]; next(x) = min elem >= x (sz if none), prev(x) = max elem <= x (-1 if none); min() / max() = sz / -1 if empty().`
Time: $\mathcal{O}(\log_{64} N)$. 426eac, 89 lines

```
using ull = unsigned long long;
const int depth = 3;
const int sz = 1 << (depth * 6);

struct Tree {
  vt<ull> seg[depth];

  Tree() {
    F0R (i, depth) seg[i].resize(1 << (6 * i));
  }

  void insert(int x) {
    ROF (d, 0, depth) {
      seg[d][x >> 6] |= 1ull << (x & 63);
      x >>= 6;
    }
  } // fbca82

  void erase(int x) {
    ull b = 0;
    ROF (d, 0, depth) {
      seg[d][x >> 6] &= ~(1ull << (x & 63));
      seg[d][x >> 6] |= b << (x & 63);
      x >>= 6;
      b = bool(seg[d][x]);
    }
  } // 9265ef

  int next(int x) {
    if (x >= sz) return sz;
    x = std::max(x, 0);
    int d = depth - 1;
    while (true) {
      if (ull m = seg[d][x >> 6] >> (x & 63)) {
        x += __builtin_ctzll(m);
        break;
```

```
    }
    x = (x >> 6) + 1;
    if (d == 0 || x >= (1 << (6 * d))) return sz;
    d--;
  } // 97fad3
  while (++d < depth) {
    x = (x << 6) + __builtin_ctzll(seg[d][x]);
  }
  return x;
}

int prev(int x) {
  if (x < 0) return -1;
  x = std::min(x, sz - 1);
  int d = depth - 1;
  while (true) {
    if (ull m = seg[d][x >> 6] << (63 - (x & 63))) {
      x -= __builtin_clzll(m);
      break;
    }
    x = (x >> 6) - 1;
    if (d == 0 || x == -1) return -1;
    d--;
  } // 80e6f1
  while (++d < depth) {
    x = (x << 6) + 63 - __builtin_clzll(seg[d][x]);
  }
  return x;
}

int min() {
  if (empty()) return sz;
  int ans = 0;
  F0R (d, depth) {
    ans <<= 6;
    ans += __builtin_ctzll(seg[d][ans >> 6]);
  }
  return ans;
} // f0b37a

int max() {
  if (empty()) return -1;
  int ans = 0;
  F0R (d, depth) {
    ans <<= 6;
    ans += 63 - __builtin_clzll(seg[d][ans >> 6]);
  }
  return ans;
}

inline bool empty() { return !seg[0][0]; }
inline int operator[] (int i) { return 1 & (seg[depth - 1][i >> 6] >
    > (i & 63)); }
};
```

Treap.h

Description: Treap with too many operations
Usage: ptr t = 0; every mutator returns the new root. Range update/query: split, set the piece's lazy / read its agg, merge. push(n) before reading a node's fields. order(x) = index of x, valid right after find/findi.
Time: $\mathcal{O}(\log N)$ 98211b, 184 lines

```
using K = ll;
random_device rd;
mt19937 mt(rd());

struct Lazy {
  ll v;
  bool inc, rev;
  void operator+=(const Lazy &b) {
    if (b.inc) v += b.v;
    else v = b.v, inc = false;
    rev ^= b.rev;
  }
};

struct Value {
  ll mx, sum;
  void upd(const Lazy &b, int sz) {
    if (!b.inc) mx = sum = 0;
    mx += b.v, sum += b.v * sz;
  }
  Value operator+(const Value &b) const {
    return {max(mx, b.mx), sum + b.sum};
  }
};

const Lazy LID = {0, true, false};
const Value VID = {-INF, 0};

using ptr = struct Node*;

struct Node {
  int pri;
  K key;
  ptr l, r, par;
  int sz;

  Value val, agg;
  Lazy lazy;

  Node(K key, Value val) : key(key), val(val), agg(val) {
    sz = 1;
    pri = mt();
    l = r = par = 0;
    lazy = LID;
  }

  ~Node() {
    delete l;
    delete r;
  }
};

int sz(ptr n) { return n ? n->sz : 0; }
Value val(ptr n) { return n ? n->val : VID; }
Value agg(ptr n) { return n ? n->agg : VID; }

ptr push(ptr n) {
  if (!n) return n;
  if (n->lazy.rev) swap(n->l, n->r);
  ptr l = n->l, r = n->r;
  n->val.upd(n->lazy, 1);
  n->agg.upd(n->lazy, n->sz);
  if (l) n->l->lazy += n->lazy;
  if (r) n->r->lazy += n->lazy;
  n->lazy = LID;
  return n;
}

ptr pull(ptr n) {
  ptr l = n->l, r = n->r;
  push(l), push(r);
  n->sz = sz(l) + 1 + sz(r);
  n->agg = agg(l) + n->val + agg(r);
  if (n->l) n->l->par = n;
  if (n->r) n->r->par = n;
  return n;
}

pair<ptr, ptr> split(ptr n, K k) {
  if (!n) return {n, n};
  push(n);
  n->par = 0;
  if (k <= n->key) {
    auto [l, r] = split(n->l, k);
    n->l = r;
    return {l, pull(n)};
  } else {
    auto [l, r] = split(n->r, k);
    n->r = l;
    return {pull(n), r};
  }
}

pair<ptr, ptr> spliti(ptr n, int i) {
  if (!n) return {n, n};
  push(n);
  n->par = 0;
  if (i <= sz(n->l)) {
    auto [l, r] = spliti(n->l, i);
    n->l = r;
    return {l, pull(n)};
  } else {
    auto [l, r] = spliti(n->r, i - sz(n->l) - 1);
    n->r = l;
    return {pull(n), r};
  }
}

ptr merge(ptr l, ptr r) {
  if (!l || !r) return l ? l : r;
  push(l), push(r);
  ptr t;
  if (l->pri > r->pri) l->r = merge(l->r, r), t = l;
  else r->l = merge(l, r->l), t = r;
  return pull(t);
}

ptr del(ptr n, K k) { // delete one copy of k, if present
  auto a = split(n, k), b = spliti(a.s, 1);
  if (b.f && b.f->key != k) // k was absent; reattach
    return merge(a.f, merge(b.f, b.s));
  return merge(a.f, b.s);
}

ptr find(ptr n, K k) {
  push(n);
  if (!n || n->key == k) return n;
  if (k < n->key) return find(n->l, k);
  else return find(n->r, k);
}

ptr findi(ptr n, int i) {
  push(n);
  if (!n || i == sz(n->l)) return n;
  if (i < sz(n->l)) return findi(n->l, i);
  else return findi(n->r, i - 1 - sz(n->l));
}

K mn(ptr n) {
  assert(n);
  push(n);
  if (n->l) return mn(n->l);
  else return n->key;
```

```
}

int order(ptr n, ptr from = nullptr) {
  if (!n) return -1;
  int res = order(n->par, n);
  if (from == n->r || !from) res += sz(n->l) + 1;
  return res;
}

ptr unite(ptr l, ptr r) {
  if (!l || !r) return l ? l : r;
  // l has the smallest key
  if (mn(l) > mn(r)) swap(l, r);
  ptr res = 0;
  while (r) {
    auto [lt, rt] = split(l, mn(r) + 1);
    res = merge(res, lt);
    tie(l, r) = make_pair(r, rt);
  }
  return merge(res, l);
}

void heapify(ptr n) {
  if (!n) return;
  ptr mx = n;
  if (n->l && n->l->pri > mx->pri) mx = n->l;
  if (n->r && n->r->pri > mx->pri) mx = n->r;
  if (mx != n) swap(n->pri, mx->pri), heapify(mx);
}

ptr build(int l, int r, vt<ptr>& ns) {
  if (l > r) return nullptr;
  if (l == r) return ns[l];
  int m = (r + l) / 2;
  ns[m]->l = build(l, m - 1, ns);
  ns[m]->r = build(m + 1, r, ns);
  heapify(ns[m]);
  return pull(ns[m]);
}

Node* tree;
```

TreapBeats.h

Description: Segtree beats on a treap: range chmin/chmax/add and range sum/min/max on half-open [lo, hi). Only updi/queryi: build the tree yourself (new Node(Value::make(x)), pull(n)).
Usage: updi(t, lo, hi, chmin_tag(x)); queryi(t, lo, hi).sum
Time: $\mathcal{O}\left(\log^2 N\right)$ amortized 9e6da6, 146 lines

```
struct Lazy {
  ll mn, mx, add;

  void operator+=(const Lazy oth) {
    if (oth.mn - add <= mx) mn = mx = oth.mn - add;
    else if (oth.mx - add >= mn) mn = mx = oth.mx - add;
    else {
      mn = min(mn, oth.mn - add);
      mx = max(mx, oth.mx - add);
    }
    add += oth.add;
  }
};

const Lazy lid = {INF, -INF, 0};

Lazy chmin_tag(ll x) { Lazy lazy = lid; lazy.mn = x; return lazy; }
Lazy chmax_tag(ll x) { Lazy lazy = lid; lazy.mx = x; return lazy; }
Lazy add_tag(ll x) { Lazy lazy = lid; lazy.add = x; return lazy; }

// You can implement your own monoid here for custom operations.
struct Value {
  ll sum;
  ll mx, mxcnt, mx2;
  ll mn, mncnt, mn2;

  static Value make(ll x, ll len = 1) {
    return {x * len, x, len, -INF, x, len, INF};
  }

  bool can_break(const Lazy& lazy) {
    return lazy.mn >= mx && lazy.mx <= mn && lazy.add == 0;
  }

  bool can_tag(const Lazy& lazy) {
    return mx2 < lazy.mn && mn2 > lazy.mx;
  }

  void upd(Lazy lazy, int sz) {
    if (mn == mx) {
      mn = mx = min(lazy.mn, mn);
      mn = mx = max(lazy.mx, mn);
      sum = (ll) mn * mncnt;
    } else if (mn == mx2) {
      if (lazy.mx > mn) mn = mx2 = lazy.mx;
      if (lazy.mn < mx) mx = mn2 = lazy.mn;
      sum = (ll) mn * mncnt + (ll) mx * mxcnt;
    } else {
      if (lazy.mn < mx) sum -= (ll) (mx - lazy.mn) * mxcnt, mx = lazy.
          mn;
      if (lazy.mx > mn) sum += (ll) (lazy.mx - mn) * mncnt, mn = lazy.
          mx;
    }
    sum += (ll) lazy.add * sz;
    mx += lazy.add, mx2 += lazy.add;
    mn += lazy.add, mn2 += lazy.add;
  }

  Value operator+(const Value& oth) const {
    Value res {};
    res.sum = sum + oth.sum;
    if (mx == oth.mx) res.mx = mx, res.mxcnt = mxcnt + oth.mxcnt, res.
        mx2 = max(mx2, oth.mx2);
    else if (mx > oth.mx) res.mx = mx, res.mxcnt = mxcnt, res.mx2 =
        max(mx2, oth.mx);
    else res.mx = oth.mx, res.mxcnt = oth.mxcnt, res.mx2 = max(mx, oth
        .mx2);
    if (mn == oth.mn) res.mn = mn, res.mncnt = mncnt + oth.mncnt, res.
        mn2 = min(mn2, oth.mn2);
    else if (mn < oth.mn) res.mn = mn, res.mncnt = mncnt, res.mn2 =
        min(mn2, oth.mn);
    else res.mn = oth.mn, res.mncnt = oth.mncnt, res.mn2 = min(mn, oth
        .mn2);
    return res;
  }
};

const Value vid = {0, -INF, 0, -INF, INF, 0, INF};

mt19937 mt(chrono::steady_clock::now().time_since_epoch().count());
using ptr = struct Node*;

struct Node {
  Value val;
  Value agg;
  Lazy lazy;

  int sz;
  int pri;
  ptr l, r;

  Node() {
    pri = mt();
    val = vid;
    agg = vid;
    lazy = lid;
    sz = 0;
    l = r = nullptr;
  }

  Node(Value val) : val(val), agg(val) {
    pri = mt();
    lazy = lid;
    sz = 1;
    l = r = nullptr;
  }

  ~Node() {
    delete l;
    delete r;
  }
};

int sz(ptr n) { return n ? n->sz : 0; };
Value agg(ptr n) { return n ? n->agg : vid; }

void push(ptr n) {
  n->val.upd(n->lazy, 1);
  n->agg.upd(n->lazy, sz(n));
  if (n->l) n->l->lazy += n->lazy;
  if (n->r) n->r->lazy += n->lazy;
  n->lazy = lid;
}

ptr pull(ptr n) {
  if (!n) return nullptr;
  if (n->l) push(n->l);
  if (n->r) push(n->r);
  n->sz = sz(n->l) + 1 + sz(n->r);
  n->agg = agg(n->l) + n->val + agg(n->r);
  return n;
}

void updi(ptr n, int lo, int hi, Lazy lazy) {
  if (!n) return;
  push(n);
  if (lo >= n->sz || hi <= 0 || n->agg.can_break(lazy)) return;
  if (lo <= 0 && n->sz <= hi && n->agg.can_tag(lazy)) {
    n->lazy += lazy;
    push(n);
    return;
  }
  if (lo <= sz(n->l) && sz(n->l) < hi) n->val.upd(lazy, 1);
  updi(n->l, lo, hi, lazy);
  updi(n->r, lo - 1 - sz(n->l), hi - 1 - sz(n->l), lazy);
  pull(n);
}

Value queryi(ptr n, int lo, int hi) {
  if (!n || lo >= sz(n) || hi <= 0) return vid;
  push(n);
  if (lo <= 0 && sz(n) <= hi) return n->agg;
  return queryi(n->l, lo, hi) + (lo <= sz(n->l) && sz(n->l) < hi ?
      n->val : vid) + queryi(n->r, lo - 1 - sz(n->l), hi - 1 - sz(n-
      >l));
}
```

DynamicBitset.h

Description: Workaround for dynamic bitsets. Adds compile time and memory factor. Reduce multiplying factor depending on desired tradeoff.

388241, 10 lines

```
const int MAX_LEN = 1 << 20;
template <int len = 1>
void solve(int n) {
  assert(n <= MAX_LEN);
  if (n > len) {
    solve<min(len * 2, MAX_LEN)>(n);
    return;
  }
  // solution using bitset<len>
}
```

BitsetFindPrev.h

Description: Adds `_Find_prev(i)` (last set index $< i$, -1 if none) and `Find_last()` to bitset; `Find_next` returns N if none.

2e6302, 28 lines

```
template <size_t Nb>
struct Bitset : bitset<Nb> {
  template <typename... Args>
  Bitset(Args... args) : bitset<Nb>(args...) {}

  static constexpr int N = Nb;
  static constexpr int array_size = (Nb + 63) / 64;

  union raw_cast {
  array<uint64_t, array_size> a;
  Bitset b;
  };

  int _Find_prev(size_t i) const {
  if (i == 0) return -1;
  if ((*this)[--i] == true) return i;
  size_t M = i / 64;
  const auto& a = ((raw_cast*)(this))->a;
  uint64_t buf = a[M] & ((1ull << (i & 63)) - 1);
  if (buf != 0) return M * 64 + 63 - __builtin_clzll(buf);
  while (M--) {
    if (a[M] != 0) return M * 64 + 63 - __builtin_clzll(a[M]);
  }
  return -1;
  }

  inline int _Find_last() const { return _Find_prev(N); }
};
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change `step` to add/remove the edge (a, c) and remove the initial `add` call (but keep `in`).

Usage: `mo: Q = {l, r} half-open. moTree: Q = {a, b} nodes, ed = adjacency lists, path a..b with both ends and the LCA. add/del(ind, end): end 0 = left, 1 = right. Set blk to about N / sqrt(Q) in both.`

Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

bd1b0c, 51 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vt<pi> Q) {
  int L = 0, R = 0, blk = 350; // set to N/sqrt(Q)
  vi s(size(Q)), res = s;
#define K(x) pi(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&] (int s, int t) { return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pi q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}

vi moTree(vt<array<int, 2>> Q, vt<vi>& ed, int root = 0) {
  int N = size(ed), pos[2] = {}, blk = 350; // set to N/sqrt(Q)
  vi s(size(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&] (int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) pi(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&] (int s, int t) { return K(Q[s]) < K(Q[t]); });
  for (int qi : s) F0R (end, 2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
          else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}
#undef K
#undef step
```

FastMo.h

Description: Mo's algorithm, queries are half-open [l, r).

Usage: `Fast_Mo m(N, Q); m.insert(l, r) Q times; m.run(addL, addR, delL, delR, rem): add/del take an element index, rem(q) answers the q-th insert. State starts empty; run once.`

Time: $\mathcal{O}\left(N\sqrt{Q}\right)$ but faster?

9f220c, 79 lines

```
struct Fast_Mo {
  int N, Q, width;
  vi L, R, order;
  bool is_build;
  int nl, nr;

  Fast_Mo(int _n, int _q) : N(_n), Q(_q), order(Q), is_build(false) {
  width = max<int>(1, 1.0 * N / max<db>(1.0, sqrt(Q / 2.0)));
  iota(begin(order), end(order), 0);
  }
  // [l, r)
  void insert(int l, int r) {
  assert(0 <= l and l <= r and r <= N);
  L.pb(l), R.pb(r);
  }

  void build() { sort(), climb(), is_build = true; }

  template <typename AL, typename AR, typename DL, typename DR,
       typename REM>
  void run(const AL &add_left, const AR &add_right, const DL &
       delete_left,
       const DR &delete_right, const REM &rem) {
  if (!is_build) build();
  nl = nr = 0;
  for (auto idx : order) {
    while (nl > L[idx]) add_left(--nl);
    while (nr < R[idx]) add_right(nr++);
    while (nl < L[idx]) delete_left(nl++);
    while (nr > R[idx]) delete_right(--nr);
    rem(idx);
  }
  }

private:
  void sort() {
  assert(size(L) == Q); // insert() exactly Q times
  vi cnt(N + 1), buf(Q);
  for (int i = 0; i < Q; i++) cnt[R[i]]++;
  for (int i = 1; i < size(cnt); i++) cnt[i] += cnt[i - 1];
  for (int i = 0; i < Q; i++) buf[--cnt[R[i]]] = i;
  vi b(Q);
  for (int i = 0; i < Q; i++) b[i] = L[i] / width;
  cnt.resize(N / width + 1);
  fill(begin(cnt), end(cnt), 0);
  for (int i = 0; i < Q; i++) cnt[b[i]]++;
  for (int i = 1; i < size(cnt); i++) cnt[i] += cnt[i - 1];
  for (int i = 0; i < Q; i++) order[--cnt[b[buf[i]]]] = buf[i];
  for (int i = 0, j = 0; i < Q; i = j) {
    int bi = b[order[i]];
    j = i + 1;
    while (j != Q and bi == b[order[j]]) j++;
    if (!(bi & 1)) reverse(begin(order) + i, begin(order) + j);
  }
  }

  int dist(int i, int j) { return abs(L[i] - L[j]) + abs(R[i] - R[j]);
       }

  void climb(int iter = 3, int interval = 5) {
  if (Q == 0) return;
  vi d(Q - 1);
  for (int i = 0; i < Q - 1; i++) d[i] = dist(order[i], order[i + 1]);
  while (iter--) {
    for (int i = 1; i < Q; i++) {
    int pre1 = d[i - 1];
    int js = i + 1, je = min<int>(i + interval, Q - 1);
    for (int j = je - 1; j >= js; j--) {
      int pre2 = d[j];
      int now1 = dist(order[i - 1], order[j]);
      int now2 = dist(order[i], order[j + 1]);
      if (now1 + now2 < pre1 + pre2) {
      reverse(begin(order) + i, begin(order) + j + 1);
      reverse(begin(d) + i, begin(d) + j);
      d[i - 1] = pre1 = now1;
      d[j] = now2;
      }
    }
    }
  }
  }
};
```

LeftistHeap.h

Description: Persistent meldable heap. Empty heap is `ph h = 0`, min is `h->v`; old versions stay valid.
Time: $\mathcal{O}(\log N)$ per meld
Memory: $\mathcal{O}(\log N)$ per meld af8a7c, 17 lines

```
using T = pair<ll, int>;
using ph = struct heap*;
struct heap { // min heap
  ph l = NULL, r = NULL;
  int s = 0; T v; // s: path to leaf
  heap(T _v):v(_v) {}
};

ph meld(ph p, ph q) {
  if (!p || !q) return p?:q;
  if (p->v > q->v) swap(p, q);
  ph P = new heap(*p); P->r = meld(P->r, q);
  if (!P->l || P->l->s < P->r->s) swap(P->l, P->r);
  P->s = (P->r?P->r->s:0) + 1; return P;
}
ph ins(ph p, T v) { return meld(p, new heap(v)); }
ph pop(ph p) { return meld(p->l, p->r); }
```

Numerical (4)

4.1 Polynomials and recurrences

QuadRoots.h

Description: Calculates the roots of a quadratic equation with better precision. Returns the number of roots.
Time: $\mathcal{O}(1)$ cda769, 8 lines

```
int quadRoots(db a, db b, db c, pair<db, db> &out) {
  assert(a != 0);
  db disc = b * b - 4 * a * c;
  if (disc < 0) return 0;
  db sum = (b >= 0) ? -b - sqrt(disc) : -b + sqrt(disc);
  out = {sum / (2 * a), sum == 0 ? 0 : (2 * c) / sum};
  return 1 + (disc > 0);
}
```

Polynomial.h

Description: $\sum a_i x^i$, `a[i]` the coefficient of x^i. `diff` and `divroot(x0)` (divide by $x - x_0$) work in place. dcb459, 17 lines

```
struct Poly {
  vt<db> a;
  db operator()(db x) const {
    db val = 0;
    for (int i = size(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    FOR (i, 1, size(a)) a[i - 1] = i * a[i];
    a.pop_back();
  }
  void divroot(db x0) {
    db b = a.back(), c; a.back() = 0;
    for (int i = size(a) - 1; i--;) c = a[i], a[i] = a[i + 1] * x0 + b
        , b = c;
    a.pop_back();
  }
};
```

PolyRoots.h

Description: Finds the distinct real roots in $[xmin, xmax]$. With double, use well-scaled inputs and simple roots separated by $\gg 10^{-8}\max(1, |x|)$. Multiple roots are unsafe.
Usage: `poly_roots({{2,-3,1}},-1e9,1e9) // x^2-3x+2 = 0`
Time: $\mathcal{O}(n^3)$, where n is the degree. "Polynomial.h" e424c1, 36 lines

```
vt<db> poly_roots(Poly p, db xmin, db xmax) {
  assert(xmin <= xmax);
  while (!p.a.empty() && p.a.back() == 0) p.a.pop_back();
  if (size(p.a) <= 1) return {};
  if (size(p.a) == 2) {
    db x = -p.a[0] / p.a[1];
    return xmin <= x && x <= xmax ? vt<db>{x} : vt<db>{};
  } // 57adaf
  Poly der = p;
  der.diff();
  auto dr = poly_roots(der, xmin, xmax);
  sort(all(dr));
  vt<db> xs{xmin};
  for (db x : dr) if (xmin < x && x < xmax) xs.pb(x);
  if (xmin < xmax) xs.pb(xmax);
  vt<db> ret;
  F0R (i, size(xs)) {
    db fl = p(xs[i]);
    if (fl == 0) ret.pb(xs[i]);
    if (i + 1 == size(xs)) break;
    db fh = p(xs[i + 1]);
    if (fl == 0 || fh == 0 || (fl < 0) == (fh < 0))
      continue; // 1d0311
    db l = xs[i], h = xs[i + 1];
    F0R (it, 100) {
      db m = l / 2 + h / 2;
      if (m == l || m == h) break;
      db fm = p(m);
      if (fm == 0) { l = h = m; break; }
      if ((fm < 0) == (fl < 0)) l = m, fl = fm;
      else h = m;
    }
    ret.pb(l / 2 + h / 2);
  }
  return ret;
}
```

DurandKerner.h

Description: Durand–Kerner: all n complex roots of $a_0 + a_1 x + \cdots + a_n x^n$ at once ($a_n \neq 0$). Quadratic convergence for simple roots; a root of multiplicity m is only accurate to $\approx 10^{-16/m}$, and $n > 15$ is ill-conditioned. Scale so roots are $O(1)$. Real roots: `abs(imag) < eps`.
Usage: `dk_roots({-2, 0, 1}) // x^2 - 2`
Time: $\mathcal{O}(n^2)$ per iteration, typically 50–500 iterations bebacb, 23 lines

```
vt<complex<db>> dk_roots(vt<db> a, int iters = 500) {
  while (size(a) > 1 && a.back() == 0) a.pop_back();
  int n = size(a) - 1;
  vt<complex<db>> z(n);
  F0R (i, n)
    z[i] = polar(1 + .2 * i / n, 2 * M_PI * i / n + .4);
  F0R (it, iters) {
    db mv = 0;
    F0R (i, n) {
      complex<db> v = a[n], d = 1;
      ROF (k, 0, n) v = v * z[i] + a[k];
      F0R (j, n) if (j != i) d *= z[i] - z[j];
      v /= a[n] * d, z[i] -= v, mv = max(mv, abs(v));
    }
    if (mv < 1e-14) break;
  }
  F0R (i, n) F0R (t, 3) { // Newton polish
    complex<db> v = 0, d = 0;
    ROF (k, 0, n + 1) d = d * z[i] + v, v = v * z[i] + a[k];
    if (abs(v) < abs(d) * 1e-9) z[i] -= v / d;
  }
  return z;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \ldots n-1$.
Usage: `interpolate(x, y, n) // x distinct`
Time: $\mathcal{O}(n^2)$ 1a7cc3, 14 lines

```
using vd = vt<db>;
vd interpolate(vd x, vd y, int n) {
  assert(n > 0);
  vd res(n), temp(n);
  F0R (k, n - 1) FOR (i, k + 1, n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  db last = 0; temp[0] = 1;
  F0R (k, n) F0R (i, n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

LagrangeInterpolation.h

Description: Given $y_i = f(i)$ for $i = 0..n$ and $\deg f \leq n$, evaluates $f(x) \bmod p$ in $O(n)$. Idea: $L_i(x) = \prod_{j \neq i} \frac{x-j}{i-j}$ is 1 at i and 0 at every other sample, so $f(x) = \sum_i y_i L_i(x)$; with samples $0..n$ the denominators are $\pm i!(n-i)!$ and the numerators are prefix/suffix products of $(x-j)$. Typical use: $\sum_{k \leq N} k^m$ is a polynomial of degree $m+1$ in N, so sample $m+2$ values. General points: PolyInterpolate.
Time: $\mathcal{O}(n)$ per call; factorials are cached ($\mathcal{O}(\log p)$ per new index)
"../number-theory/ModPow.h" c93753, 20 lines

```
ll lagrange(vl &y, ll x) { // y[i] = f(i), i in [0, n]
  int n = size(y) - 1; x = (x % mod + mod) % mod;
  static vl fac{1}, ifac{1}; // cached across calls
  while (size(fac) <= n) {
    fac.pb(fac.back() * size(fac) % mod);
    ifac.pb(mpow(fac.back()));
  }
  vl pre(n + 2, 1), suf(n + 2, 1);
  F0R (i, n + 1)
    pre[i + 1] = pre[i] * ((x - i + mod) % mod) % mod;
  ROF (i, 0, n + 1)
    suf[i] = suf[i + 1] * ((x - i + mod) % mod) % mod;
  ll res = 0;
  F0R (i, n + 1) {
    ll t = y[i] % mod * pre[i] % mod * suf[i + 1] % mod
      * ifac[i] % mod * ifac[n - i] % mod;
    res = (res + ((n - i) & 1 ? mod - t : t)) % mod;
  }
  return (res + mod) % mod;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Output C (size $\leq n$): $s[i] = \sum_j C[j]\, s[i-j-1]$ for $i \geq |C|$; the $s[0 \ldots |C|-1]$ are the initial values. Needs $\geq 2|C|$ terms, the prime global mod (ModPow.h) and every s[i] in $[0, mod)$. Matrix form: M is $n \times n$ with first row C and $M_{i,i-1} = 1$ below it, $v = (s[n-1], \ldots, s[0])$; $s[k]$ is the last entry of $M^k v$. $O(n^3 \log k)$, versus $O(n^2 \log k)$ for linearRec.
Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
`linearRec(s, berlekampMassey(s), k) // s[k]`
Time: $\mathcal{O}(N^2)$ "../number-theory/ModPow.h" ce4d42, 20 lines

```
vt<ll> berlekampMassey(vt<ll> s) {
  int n = size(s), L = 0, m = 0;
  vt<ll> C(n), B(n), T;
  C[0] = B[0] = 1;

  ll b = 1;
  F0R (i, n) { ++m;
    ll d = s[i] % mod;
    FOR (j, 1, L + 1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * mpow(b, mod - 2) % mod;
    FOR (j, m, n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }

  C.resize(L + 1); C.erase(C.begin());
  for (ll &x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \ldots \geq n-1]$ and $tr[0 \ldots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
`vl s; F0R (i, 40) s.pb(brute(i) % mod); // >= 2n terms`
`vl c = berlekampMassey(s); // check 2 size(c) < size(s)`
`ll x = linearRec(s, c, k); // s[k], k up to 1e18`
Time: $\mathcal{O}(n^2 \log k)$ 366168, 28 lines

```
using Poly = vt<ll>;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = size(tr);
  assert(size(S) >= n);
  if (!n) return 0;

  auto combine = [&] (Poly a, Poly b) {
    Poly res(n * 2 + 1);
    F0R (i, n + 1) F0R (j, n + 1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) F0R (j, n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }

  ll res = 0;
  F0R (i, n) res = (res + pol[i + 1] * S[i]) % mod;
  return (res + mod) % mod;
}
```

LinearRecurrenceNTT.h

Description: Same contract as linearRec: k'th term of $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \ldots \geq n-1]$ and tr. $s[k] = [x^k]P/Q$ with $Q = 1 - \sum_j tr[j]x^{j+1}$; each step multiplies by $Q(-x)$ and keeps the half of P matching k's parity. NTT mod (998244353); for other mods use convMod from FFTMod.
Usage: `linearRecNTT({0, 1}, {1, 1}, k) // k'th Fibonacci number`
Time: $\mathcal{O}(n \log n \log k)$ "NumberTheoreticTransform.h" 1c24ab, 18 lines

```
ll linearRecNTT(vl S, vl tr, ll k) {
  int n = size(tr);
  assert(size(S) >= n);
  vl Q(n + 1); Q[0] = 1;
  F0R (i, n) Q[i + 1] = (mod - tr[i] % mod) % mod;
  S.resize(n);
  vl P = conv(S, Q); P.resize(n);
  for (; k; k >>= 1) {
    vl R = Q;
    for (int i = 1; i < size(R); i += 2)
      R[i] = (mod - R[i]) % mod;
    vl A = conv(P, R), B = conv(Q, R);
    P.clear(), Q.clear();
    for (int i = k & 1; i < size(A); i += 2) P.pb(A[i]);
    for (int i = 0; i < size(B); i += 2) Q.pb(B[i]);
  }
  return P.empty() ? 0 : P[0]; // Q[0] stays 1
}
```

4.2 Optimization

4.2.1 Newton vs ternary search

Newton ($x \leftarrow x - f/f'$) converges quadratically near a simple root: ~6 iterations for 10^{-12} from a decent start, versus $\log_{1.5}(\text{range}/\epsilon) \approx 70$ iterations (two evaluations each) for ternary search. It needs f' (finite differences lose half the digits), and a bracket or a good start – raw Newton diverges or cycles otherwise (the template falls back to bisection). At a root of multiplicity m it is only linear: use $x \leftarrow x - mf/f'$, or Newton on f/f'. Newton finds zeros, so to minimise g you run it on g' and supply g'' in closed form – worth it when g is smooth and evaluations are expensive or high precision is needed. With only values of a unimodal g, use ternary/golden-section search instead; finite-difference derivatives $(g'(x) \approx (g(x+h) - g(x-h))/2h)$ lose half the digits and make Newton no better than golden section. For roots without f', bracketed secant (Illinois) converges superlinearly. Integer Newton: $x \leftarrow (x + n/x)/2$ from any $x \geq \sqrt{n}$; iterate while it decreases, the last x is $\lfloor \sqrt{n} \rfloor$ (same idea for k-th roots).

Newton.h

Description: Safeguarded Newton: a root of f in $[lo, hi]$ where $f(lo)$, $f(hi)$ have opposite signs. Takes the Newton step when it stays inside the bracket and bisects otherwise, so it cannot diverge; quadratic convergence near simple roots (~6 iterations), linear at multiple roots. To minimise a convex g, pass g' and g''.
Usage: `newton(0, 2, f, df) with f = [](db x) { return x*x - 2; }`
`and df = [](db x) { return 2*x; }` e1d4df, 13 lines

```
template<class F, class D>
db newton(db lo, db hi, F f, D df) {
  db x = (lo + hi) / 2, fl = f(lo);
  F0R (it, 200) {
    db fx = f(x);
    if ((fx < 0) == (fl < 0)) lo = x, fl = fx; else hi = x;
    db nx = x - fx / df(x);
    if (!(lo < nx && nx < hi)) nx = (lo + hi) / 2; // bisect
    if (abs(nx - x) <= 1e-13 * (1 + abs(x))) return nx;
    x = nx;
  }
  return x;
}
```

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.
Usage: `db func(db x) { return 4+x+.3*x*x; }`
`db xmin = gss(-1000,1000,func);`
Time: 100 evaluations of f (relative precision 10^{-21}) 013d82, 14 lines

```
template<class F> db gss(db a, db b, F f) {
  db r = (sqrt(5) - 1) / 2;
  db x1 = b - r * (b - a), x2 = a + r * (b - a);
  db f1 = f(x1), f2 = f(x2);
  F0R (it, 100)
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r * (b - a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r * (b - a); f2 = f(x2);
    }
  return a;
}
```

HillClimbing.h

Description: Poor man's optimization for unimodal functions. Minimizes f over the plane, returns (min, argmin).
Usage: `auto [v, p] = hillClimb({0, 0}, f); // f(P) -> db`
Time: 87301 calls to f 7ecb06, 14 lines

```
using P = array<db, 2>;

template<class F> pair<db, P> hillClimb(P start, F f) {
  pair<db, P> cur(f(start), start);
  for (db jmp = 1e9; jmp > 1e-20; jmp /= 2) {
    F0R (j, 100) FOR (dx, -1, 2) FOR (dy, -1, 2) {
      P p = cur.second;
      p[0] += dx * jmp;
      p[1] += dy * jmp;
      db v = f(p); if (v < cur.f) cur = {v, p};
    }
  }
  return cur;
}
```

4.2.2 Numerical integration

Simpson (`Integrate.h`): fixed $2n$ slices, error $O(h^4 f^{(4)})$, exact for cubics; double n until stable. Adaptive Simpson (`IntegrateAdaptive.h`): `eps` is the absolute error budget of the whole interval (halves get `eps/2`); it stops splitting below width 10^{-10}, so rescale tiny domains, and scale `eps` by the answer's magnitude for relative precision. Gauss–Legendre (`GaussLegendre.h`): n nodes at the roots of P_n, weights

$2/((1-x_i^2)P_n'(x_i)^2)$, exact for polynomials of degree $< 2n$; far more accurate per evaluation on smooth integrands, useless across kinks.

Tips: split at kinks, discontinuities and (oscillatory) periods so each piece is smooth. Endpoint singularity at a: substitute $x = a + t^2$, $dx = 2t\,dt$. Infinite range: $x = \tan t$ on $(-\pi/2, \pi/2)$, or $x = t/(1-t)$ on $[0, 1)$. Flatten an integrand with very different scales by a change of variable (e.g. $x = e^u$ for $\int f(x)\,dx/x$). Simpson evaluates both endpoints, so use `gauss` if f is undefined there. Nested integrals multiply the evaluation counts: loosen the inner `eps`. Use `long double`.

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule with $2n$ slices ($2n+1$ evaluations). Error is $O(h^4 f^{(4)})$, so double n until the result stops changing. Exact for cubics; useless across kinks or singularities – split the interval there, and substitute $x = a + t^2$ near an endpoint singularity.

Usage: `quad(a, b, f, n) // n = number of double-slices` 715302, 7 lines

```
template<class F>
db quad(db a, db b, F f, const int n = 1000) {
  db h = (b - a) / 2 / n, v = f(a) + f(b);
  FOR (i, 1, n * 2)
    v += f(a + i * h) * (i & 1 ? 4 : 2);
  return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule. eps is the absolute error budget for the whole interval (halves get eps/2). Stop test $|T - S| \le 15\,\text{eps}$: the two-half estimate T has error $\approx (T-S)/15$, which is also the correction returned. Width $< 10^{-10}$ also stops, so rescale tiny domains. Cost grows with $|f|$: scale eps with the answer (the sphere below takes 10^7 evaluations). Split at kinks and discontinuities; for endpoint singularities substitute $x = a + t^2$ ($dx = 2t\,dt$; the endpoints are evaluated, so guard $t = 0$), for infinite ranges $x = \tan t$ or $x = t/(1-t)$.

Usage:
```
quad(a, b, f, eps = 1e-8) // e.g.
db sphereVolume = quad(-1, 1, [](db x) {
return quad(-1, 1, [&](db y) {
return quad(-1, 1, [&](db z) {
return x*x + y*y + z*z < 1; });});});
```
426d0e, 15 lines

```
using d = db;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || abs(b - a) < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}
```

GaussLegendre.h

Description: n-point Gauss–Legendre quadrature on $[a, b]$: exact for polynomials of degree $< 2n$, converges very fast for smooth f. Oscillating f needs about 2 nodes per half-period: raise n or split $[a, b]$. No error estimate. $n \le 500$ is fine.

Usage: `gauss(0, 1, [](db x) { return x * x; }, 20)`

Time: $\mathcal{O}(n^2)$ setup, n evaluations 41140c, 18 lines

```
template<class F>
db gauss(db a, db b, F f, int n = 20) {
  db res = 0;
  F0R (i, n) {
    db x = cos(M_PI * (i + .75) / (n + .5)), dp = 1;
    F0R (it, 100) { // Newton on P_n(x)
      db p0 = 1, p1 = x;
      FOR (k, 2, n + 1) tie(p0, p1) =
        pair{p1, ((2 * k - 1) * x * p1 - (k - 1) * p0) / k};
      dp = n * (x * p1 - p0) / (x * x - 1);
      db dx = p1 / dp; x -= dx;
      if (abs(dx) < 1e-15) break;
    }
    db w = 2 / ((1 - x * x) * dp * dp);
    res += w * f((a + b) / 2 + (b - a) / 2 * x);
  }
  return res * (b - a) / 2;
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$. Returns IEEE -inf if there is no solution, +inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable. To minimise, negate c and the result. Equality: two rows, $ax \le b$ and $-ax \le -b$. Free x: $x^+ - x^-$.

Usage:
```
vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
```

Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case. 55299f, 62 lines

```
#define mp make_pair
using T = db;
using vd = vt<db>;
using vvd = vt<vd>;
const db eps = 1e-9;
const db linf = 1 / .0;

#define ltj(X) if (s == -1 || mp(X[j], N[j]) < mp(X[s], N[s])) s = j
struct LPSolver {
  int m, n;
  vt<int> N, B;
  vvd D;
  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(size(b)), n(size(c)), N(n + 1), B(m), D(m + 2, vd(n + 2)) {
    F0R (i, m) F0R (j, n) D[i][j] = A[i][j];
    F0R (i, m) B[i] = n + i, D[i][n] = -1, D[i][n + 1] = b[i];
    F0R (j, n) N[j] = j, D[m][j] = -c[j];
    N[n] = -1; D[m + 1][n] = 1;
  } // d4d8ce
  void pivot(int r, int s) {
    T inv = 1 / D[r][s];
    F0R (i, m + 2) if (i != r && abs(D[i][s]) > eps) {
      T binv = D[i][s] * inv;
      F0R (j, n + 2) if (j != s) D[i][j] -= D[r][j] * binv;
      D[i][s] = -binv;
    }
    D[r][s] = 1; F0R (j, n + 2) D[r][j] *= inv; // scale r-th row
    swap(B[r], N[s]);
  } // fdde58
  bool simplex(int phase) {
    int x = m + phase - 1;
    while (1) {
      int s = -1; F0R (j, n + 1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return 1;
      int r = -1;
      F0R (i, m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || mp(D[i][n + 1] / D[i][s], B[i])
              < mp(D[r][n + 1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return 0;
      pivot(r, s);
    }
  } // 8aaff3
  T solve(vd &x) {
    int r = 0; FOR (i, 1, m) if (D[i][n + 1] < D[r][n + 1]) r = i;
    if (D[r][n + 1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m + 1][n + 1] < -eps) return -linf;
      F0R (i, m) if (B[i] == -1) {
        int s = 0; FOR (j, 1, n + 1) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    F0R (i, m) if (B[i] < n) x[B[i]] = D[i][n + 1];
    return ok ? D[m][n + 1] : linf;
  }
};

#undef mp
#undef ltj
```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}(N^3)$ 7485d1, 15 lines

```
db det(vt<vt<db>>& a) {
  int n = size(a); db res = 1;
  F0R (i, n) {
    int b = i;
    FOR (j, i + 1, n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    FOR (j, i + 1, n) {
      db v = a[j][i] / a[i][i];
      if (v != 0) FOR (k, i + 1, n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version. Pass entries already reduced into $(-mod, mod)$ and keep $mod < \sim 3e9$: intermediates reach mod^2. Works for composite mod. Destroys the matrix.

Time: $\mathcal{O}(N^3)$ 6ff634, 18 lines

```
// needs a global ll mod, e.g. from "../number-theory/ModPow.h"
ll det(vt<vl> &a) {
  int n = size(a); ll ans = 1;
  F0R (i, n) {
    FOR (j, i + 1, n) {
      while (a[j][i] != 0) { // gcd step
```

```
        ll t = a[i][i] / a[j][i];
        if (t) FOR (k, i, n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Usage: `vd x(m); int r = solveLinear(A, b, x);`
Time: $\mathcal{O}\left(n^2 m\right)$ 4a5a49, 38 lines

```
using vd = vt<db>;
const db eps = 1e-12; // absolute, tune to entries

int solveLinear(vt<vd>& A, vd& b, vd& x) {
  int n = size(A), m = size(x), rank = 0, br, bc;
  if (n) assert(size(A[0]) == m);
  vi col(m); iota(all(col), 0);

  F0R (i, n) {
    db v, bv = 0;
    FOR (r, i, n) FOR (c, i, m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      FOR (j, i, n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    F0R (j, n) swap(A[j][i], A[j][bc]);
    bv = 1 / A[i][i];
    FOR (j, i + 1, n) {
      db fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      FOR (k, i + 1, m) A[j][k] -= fac * A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    F0R (j, i) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes (the second block REPLACES the back-substitution; define `undefined`, e.g. NAN): "SolveLinear.h" 77b9bb, 7 lines

```
F0R (j, n) if (j != i) // instead of FOR (j, i + 1, n)
// ... then at the end:
x.assign(m, undefined);
F0R (i, rank) {
  FOR (j, rank, m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.
Usage: `solveLinear(A, b, x, m); // b[i] is 0 or 1`
Time: $\mathcal{O}\left(n^2 m\right)$ 0cfcfa, 34 lines

```
using bs = bitset<1000>;

int solveLinear(vt<bs>& A, vi& b, bs& x, int m) {
  int n = size(A), rank = 0, br;
  assert(m <= size(x));
  vi col(m); iota(all(col), 0);
  F0R (i, n) {
    for (br = i; br < n; br++) if (A[br].any()) break;
    if (br == n) {
      FOR (j, i, n) if (b[j]) return -1;
      break;
    }
    int bc = (int) A[br]._Find_next(i - 1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    F0R (j, n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    FOR (j, i + 1, n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    F0R (j, i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinearMod.h

Description: Solves $Ax = b \pmod{\texttt{mod}}$ (prime), A is $n \times m$. Returns the rank, or -1 if there is no solution; free variables are 0. The solution is unique iff rank $= m$.
Usage: `vl x; int r = solve_mod(A, b, x);`
Time: $\mathcal{O}\left(nm \min(n, m)\right)$ "../number-theory/ModPow.h" 814f93, 24 lines

```
int solve_mod(vt<vl> A, vl b, vl& x) {
  int n = size(A), m = size(A[0]), rk = 0;
  vi piv;
  F0R (c, m) {
    int r = rk;
    while (r < n && !A[r][c]) r++;
    if (r == n) continue;
    swap(A[rk], A[r]), swap(b[rk], b[r]);
    ll v = mpow(A[rk][c]);
    F0R (j, m) A[rk][j] = A[rk][j] * v % mod;
    b[rk] = b[rk] * v % mod;
    F0R (i, n) if (i != rk && A[i][c]) {
      ll f = A[i][c];
      F0R (j, m)
        A[i][j] = (A[i][j] - f * A[rk][j] % mod + mod) % mod;
      b[i] = (b[i] - f * b[rk] % mod + mod) % mod;
    }
    piv.pb(c), rk++;
  }
  FOR (i, rk, n) if (b[i]) return -1;
  x.assign(m, 0);
  F0R (i, rk) x[piv[i]] = b[i];
  return rk;
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$ 013a34, 36 lines

```
int matInv(vt<vt<db>>& A) {
  int n = size(A); vi col(n);
  vt<vt<db>> tmp(n, vt<db>(n));
  F0R (i, n) tmp[i][i] = 1, col[i] = i;

  F0R (i, n) {
    // pick largest non-zero pivot; return any non-zero if under mod
    int r = i, c = i;
    FOR (j, i, n) FOR (k, i, n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i; // absolute eps
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    F0R (j, n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    db v = A[i][i];
    FOR (j, i + 1, n) {
      db f = A[j][i] / v; // replace with mod inv
      A[j][i] = 0;
      FOR (k, i + 1, n) A[j][k] -= f * A[i][k];
      F0R (k, n) tmp[j][k] -= f * tmp[i][k];
    }
    FOR (j, i + 1, n) A[i][j] /= v;
    F0R (j, n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n - 1; i > 0; --i) F0R (j, i) {
    db v = A[j][i];
    F0R (k, n) tmp[j][k] -= v * tmp[i][k];
  }

  F0R (i, n) F0R (j, n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). Entries must be in $[0, \text{mod})$. For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$ "../number-theory/ModPow.h" f57615, 37 lines

```
int matInv(vt<vl>& A) {
  int n = size(A); vi col(n);
  vt<vl> tmp(n, vl(n));
  F0R (i, n) tmp[i][i] = 1, col[i] = i;

  F0R (i, n) {
    int r = i, c = i;
    FOR (j, i, n) FOR (k, i, n) if (A[j][k]) {
      r = j; c = k; goto found;
    }
    return i;
found:
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    F0R (j, n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
```

```
    swap(col[i], col[c]);
    ll v = mpow(A[i][i]);
    FOR (j, i + 1, n) {
      ll f = A[j][i] * v % mod;
      A[j][i] = 0;
      FOR (k, i + 1, n) A[j][k] = (A[j][k] - f * A[i][k]) % mod;
      FOR (k, n) tmp[j][k] = (tmp[j][k] - f * tmp[i][k]) % mod;
    }
    FOR (j, i + 1, n) A[i][j] = A[i][j] * v % mod;
    FOR (j, n) tmp[i][j] = tmp[i][j] * v % mod;
    A[i][i] = 1;
  }

  for (int i = n - 1; i > 0; --i) FOR (j, i) {
    ll v = A[j][i];
    FOR (k, n) tmp[j][k] = (tmp[j][k] - v * tmp[i][k]) % mod;
  }

  FOR (i, n) FOR (j, n)
    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] < 0) * mod;
  return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, -d_1, -d_2, \ldots, -d_n, a_{n+1}\}).$$

Fails if the solution is not unique. p and q have length $n-1$.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$ c3dc24, 26 lines

```
using T = db;
vt<T> tridiagonal(vt<T> diag, const vt<T>& super,
    const vt<T>& sub, vt<T> b) {
  int n = size(b); vi tr(n);
  FOR (i, n - 1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
      b[i + 1] -= b[i] * diag[i + 1] / super[i];
      if (i + 2 < n) b[i + 2] -= b[i] * sub[i + 1] / super[i];
      diag[i + 1] = sub[i]; tr[++i] = 1;
    } else {
      diag[i + 1] -= super[i] * sub[i] / diag[i];
      b[i + 1] -= b[i] * sub[i] / diag[i];
    }
  }
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i - 1]);
      diag[i - 1] = diag[i];
      b[i] /= super[i - 1];
    } else {
      b[i] /= diag[i];
      if (i) b[i - 1] -= b[i] * super[i - 1];
    }
  }
  return b;
}
```

4.4 Convolutions

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$ (doubles: round with `llround`). For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 0.2s$ for $N = 2^{20}$) 636a47, 36 lines

```
using C = complex<db>;
using vd = vt<db>;

void fft(vt<C> &a) {
  int n = size(a), L = 31 - __builtin_clz(n);
  static vt<complex<long double>> R(2, 1);
  static vt<C> rt(2, 1); // (^ 10% faster if db)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    FOR (i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
  } // e8103a
  vi rev(n);
  FOR (i, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  FOR (i, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) FOR (j, k) {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
} // 4a8318
vd conv(const vd &a, const vd &b) {
  if (a.empty() || b.empty()) return {};
  vd res(size(a) + size(b) - 1);
  int L = 32 - __builtin_clz(size(res)), n = 1 << L;
  vt<C> in(n), out(n);
  copy(all(a), begin(in));
  FOR (i, size(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  FOR (i, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  FOR (i, size(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Usage: `convMod<M>(a, b) // M = the modulus`

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT) (but seemed +10% on yosupo?) "FastFourierTransform.h" 25d865, 21 lines

```
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(size(a) + size(b) - 1);
  int B = 32 - __builtin_clz(size(res)), n = 1 << B, cut = int(sqrt(M)
      );
  vt<C> L(n), R(n), outs(n), outl(n);
  FOR (i, size(a)) L[i] = C((int) a[i] / cut, (int) a[i] % cut);
  FOR (i, size(b)) R[i] = C((int) b[i] / cut, (int) b[i] % cut);
  fft(L), fft(R);
  FOR (i, n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  FOR (i, size(res)) {
    ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5);
    ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size less than 2^a. For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 0.2s$ for $N = 2^{20}$)
"../number-theory/ModPow.h" 033d55, 34 lines

```
const ll root = 62; // mod = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.

void ntt(vl &a) {
  int n = size(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, mpow(root, mod >> s)};
    FOR (i, k, 2 * k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  } // 235155
  vi rev(n);
  FOR (i, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  FOR (i, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) FOR (j, k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }
} // 669d22

vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = size(a) + size(b) - 1, B = 32 - __builtin_clz(s);
  int n = 1 << B, inv = mpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  FOR (i, n) out[-i & (n - 1)] = L[i] * R[i] % mod * inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

OnlineFFT.h

Description: Online convolution with a fixed kernel b: the t-th call `push(a[t])` returns $c[t] = \sum_{i=0}^{t} a[i]\, b[t-i]$, so $a[t]$ may depend on $c[0..t-1]$. E.g. for $f_{t+1} = \sum_{i \le t} f_i g_{t+1-i}$, use kernel $g_1, g_2, \ldots$; then `push(`f_t`)` returns f_{t+1}. At most n pushes; the kernel is cut/padded to length n. Any linear `conv` mod `mod` works in place of NTT's. Values must be in $[0, \text{mod})$.
Usage: `OnlineFFT o(n, kernel); o.push(x);`
Time: $\mathcal{O}\left(N \log^2 N\right)$ "NumberTheoreticTransform.h" 3eae75, 16 lines

```
struct OnlineFFT {
  vl a, b, c;
  OnlineFFT(int n, vl kernel) : b(move(kernel)), c(n) {
    b.resize(n);
    a.reserve(n);
  }
  ll push(ll x) {
    int t = size(a), w = (t + 1) & -(t + 1);
    int len = min(w, size(c) - t);
    a.pb(x);
    auto d = conv(vl(a.end() - w, a.end()),
            vl(b.begin(), b.begin() + w + len - 1));
    F0R (j, len) (c[t + j] += d[w - 1 + j]) %= mod;
    return c[t];
  }
};
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. a, b must have the same size, a power of two. Replace with long longs (care pi -> pl) and do operations under mod if needed.
Time: $\mathcal{O}\left(N \log N\right)$ 00ce7a, 17 lines

```
// don't forget MOD!
void FST(vi &a, bool inv) {
  for (int n = size(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) FOR (j, i, i + step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pi(v - u, u) : pi(v, u + v); // AND
        inv ? pi(v, u - v) : pi(u + v, u); // OR
        pi(u + v, u - v);                  // XOR
    }
  }
  if (inv) for (int& x : a) x /= size(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  F0R (i, size(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

SubsetConvolution.h

Description: $c[S] = \sum_{T \subseteq S} a[T]\, b[S \setminus T]$, arrays of size 2^n, values mod `mod`. Zeta transform by popcount rank, multiply the rank polynomials, invert, keep rank = popcount. Unlike OR-convolution it forbids overlaps.
Usage: `vl c = subset_conv(a, b, n);`
Time: $\mathcal{O}\left(2^n n^2\right)$
Memory: $24(n+1)2^n$ bytes $(n \le 18)$ "../number-theory/ModPow.h" 395803, 19 lines

```
vl subset_conv(vl a, vl b, int n) {
  int M = 1 << n;
  vt<vl> A(n + 1, vl(M)), B = A, C = A;
  F0R (i, M) {
    int k = __builtin_popcount(i);
    A[k][i] = a[i], B[k][i] = b[i];
  }
  F0R (k, n + 1) F0R (j, n) F0R (i, M) if (i >> j & 1) {
    A[k][i] = (A[k][i] + A[k][i ^ 1 << j]) % mod;
    B[k][i] = (B[k][i] + B[k][i ^ 1 << j]) % mod;
  }
  F0R (i, M) F0R (x, n + 1) F0R (y, n + 1 - x)
    C[x + y][i] = (C[x + y][i] + A[x][i] * B[y][i]) % mod;
  F0R (k, n + 1) F0R (j, n) F0R (i, M) if (i >> j & 1)
    C[k][i] = (C[k][i] - C[k][i ^ 1 << j] + mod) % mod;
  vl c(M);
  F0R (i, M) c[i] = C[__builtin_popcount(i)][i];
  return c;
}
```

MinPlusConvolution.h

Description: Min-plus convolution with one convex or concave input. SMAWK handles arbitrary/convex; the border algorithm handles arbitrary/-concave. Here, convex = smiley and concave = frowny. `min_smawk(f,r,c)` returns a minimizing column for each row of the matrix f. Totally monotone means every submatrix's leftmost row-minimum indices are nondecreasing. The matrix is queried through f, not stored.
Usage: `c = min_plus_smawk(a, b); // b convex`
`c = min_plus_concave_one(a, b); // b concave`
`vi col = min_smawk(f, R, C); // f(r, c), R rows, C cols`
Time: $\mathcal{O}\left(N + M\right)$ convex, $\mathcal{O}\left(N \log M + M\right)$ concave 13bef4, 88 lines

```
// Compute row-minima indices for totally monotone f(x,y). O(N + M)
template<class Fn>
vi min_smawk_rec(Fn& f, const vi &row, vi col) {
  int n = size(row);
  if (!n) return {};
  vi red;
  for (int c : col) {
    while (size(red) &&
      f(row[size(red) - 1], c) < f(row[size(red) - 1], red.back()))
      red.pop_back();
    if (size(red) < n) red.pb(c);
  } // eb45c6
  col = move(red);

  vi odd;
  for (int i = 1; i < n; i += 2) odd.pb(row[i]);
  vi oddans = min_smawk_rec(f, odd, col), ans(n);
  F0R (i, size(odd)) ans[2 * i + 1] = oddans[i];
  for (int i = 0, j = 0; i < n; i += 2) {
    ans[i] = col[j];
    int last = i + 1 < n ? ans[i + 1] : col.back();
    while (col[j] != last) {
      j++;
      if (f(row[i], col[j]) < f(row[i], ans[i])) ans[i] = col[j];
    }
  } // 40a846
  return ans;
}
template<class Fn>
vi min_smawk(Fn f, int r, int c) {
  vi row(r), col(c);
  iota(all(row), 0), iota(all(col), 0);
  return min_smawk_rec(f, row, col);
} // 167c2f

// Compute min plus convolution c[k] = min{i+j=k}(a[i]+b[j]) for
//     convex b. O(N + M)
vl min_plus_smawk(const vl& a, const vl& b) {
  int n = size(a), m = size(b);
  if (!n || !m) return n ? a : b;
  auto f = [&] (int r, int c) -> tuple<int, ll, int> {
    if (r < c) return {1, 0ll, c};
    if (r - c >= m) return {1, 0ll, -c};
    return {0, a[c] + b[r - c], 0};
  };
  vi cols = min_smawk(f, n + m - 1, n);
  vl d(n + m - 1);
  F0R (r, n + m - 1) d[r] = a[cols[r]] + b[r - cols[r]];
  return d;
} // 981ed3

// Compute min plus convolution c[k] = min{i+j=k}(a[i]+b[j]) for
//     concave b. O(N log M + M)
vl min_plus_concave_one(const vl& a, const vl& b) {
  int n = size(a), m = size(b), z = n + m - 1;
  if (!n || !m) return n ? a : b;
  vl c(z, INF);
  auto solve = [&] (int l, int r, bool rev) {
    auto val = [&] (int j, int k) {
      if (rev) j = n - 1 - j, k = z - 1 - k;
      return a[j] + b[k - j];
    }; // 7f52b0
#define better(i,j,k) val(i,k) <= val(j,k)
    auto improve = [&] (int u, int v, int l, int r) {
      while (r - l > 1) {
        int m = (l + r) / 2;
        (better(u, v, m) ? r : l) = m;
      }
      return l;
    };
    vt<array<int, 2>> stk;
    for (int i = l, k = l; k < r; i++, k++) {
      while (size(stk) && i < n && better(i, stk.back()[0], stk.back()
          [1]))
        stk.pop_back();
      if (i < n) {
        int t = stk.empty() ? r - 1 : improve(stk.back()[0], i, k - 1,
            stk.back()[1]);
        if (t >= k) stk.pb({i, t});
      } // 70045d
      int out = rev ? z - 1 - k : k;
      c[out] = min(c[out], val(stk.back()[0], k));
      if (stk.back()[1] == k) stk.pop_back();
    }
  };

  solve(0, m, 0), solve(0, m, 1);
  for (int k = m; k < z - m; k += m + 1)
    solve(k + 1, k + m + 1, 0), solve(z - k - m, z - k, 1);
#undef better
  return c;
}
```

Number theory (5)

5.1 Modular arithmetic

ModInverse.h

Description: Pre-computation of modular inverses `inv[1..LIM-1]`. Assumes LIM $\le$ mod and that mod is a prime. e403d4, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
FOR (i, 2, LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

Description: b^e mod `mod`; `mpow(b)` is b^{-1} for prime `mod`. e69235, 8 lines

```
const ll mod = 1000000007; // faster if const

ll mpow(ll b, ll e = mod - 2) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
```

```
  return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a. Pass a, b already reduced mod m, and keep $m < \sim 3e9$ ($(m-1)^2$ must fit in ll).
Time: $\mathcal{O}\left(\sqrt{m}\right)$ f052cb, 11 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    FOR (i, 2, n + 2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
`modsum(to, c, k, m)` $= \sum_{i=0}^{to-1} (ki + c) \% m$. `divsum` is similar but for floored division ($c, k \geq 0$).
Time: $\log(m)$, with a large constant. aba7f7, 16 lines

```
using ull = unsigned long long;
ull sumsq(ull to) { return to / 2 * ((to - 1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for `mmul`, $\mathcal{O}(\log b)$ for `mpow` b65e13, 11 lines

```
using ull = unsigned long long;
ull mmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll) M);
}
ull mpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = mmul(b, b, mod), e /= 2)
    if (e & 1) ans = mmul(ans, b, mod);
  return ans;
}
```

FixedModPow.h

Description: $b^e \bmod M$ for fixed b, $0 \leq e < 2^B$, via $\lceil B/L \rceil$ table lookups. vs binary exponentiation: $\sim$17x for $M = 2^{61}-1$, $\sim$35x for $M \approx 10^9$ ($B = 30$, ll `mul`). $L = 16$: $\sim$2x faster, $\leq$2 MiB.
Time: $\mathcal{O}\left(B/L \cdot 2^L\right)$ setup, $\mathcal{O}(B/L)$ query. 9db5cf, 20 lines

```
template<ll M, int L = 8, int B = 64> struct FixedModPow {
  static_assert(M > 1 && 1 <= L && L <= 16 && 1 <= B && B <= 64);
  static constexpr int N = 1 << L, R = (B+L-1)/L;
  ll pw[R][N]{};
  static constexpr ll mul(ll a, ll b) { // M <= 2^32: (ull)a * b % M
    return (ll)((__uint128_t)a * b % M);
  }
  constexpr FixedModPow(ll b) {
    for (auto &row : pw) {
      row[0] = 1;
      FOR (i, 1, N) row[i] = mul(row[i-1], b);
      b = mul(row[N-1], b);
    }
  }
  constexpr ll operator()(unsigned long long e) const {
    ll r = pw[0][e & (N-1)];
    FOR (i, 1, R) r = mul(r, pw[i][e>>(L*i) & (N-1)]);
    return r;
  }
};
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution), p prime. Internal products cap p at $\sim$3e9; swap the * % for mmul (already included) for larger p.
Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p "ModMulLL.h" 4ba904, 24 lines

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(mpow(a, (p - 1) / 2, p) == 1); // else no solution
  if (p % 4 == 3) return mpow(a, (p + 1) / 4, p);
  // p%8==5: x=a^((p+3)/8); if x*x%p!=a, x*=2^((p-1)/4)
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (mpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = mpow(a, (s + 1) / 2, p);
  ll b = mpow(a, s, p), g = mpow(n, s, p);
  for (; ; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = mpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

5.2 Primality

Sieve.h

Description: Linear sieve implementation. Around 0.5s for 1e8. Also computes smallest prime divisor for each number in lp. Memory: 4(mx+1) bytes (mx=1e8 $\rightarrow$ $\sim$400MB;). Prime iff `lp[x] == x`. `factor(x)` returns {p, e} pairs, $1 < x <$ mx. d24f05, 21 lines

```
const int mx = 1e8; // ~0.5s
vi lp(mx + 1), primes;

FOR (i, 2, mx + 1) {
  if (lp[i] == 0) lp[i] = i, primes.pb(i);
  for (int j = 0; i * primes[j] <= mx; j++) {
    lp[i * primes[j]] = primes[j];
    if (primes[j] == lp[i]) break;
  }
}

auto factor = [&] (int x) {
  vt<pi> res;
  while (x > 1) {
    if (!size(res) || res.back().first != lp[x])
      res.pb({lp[x], 0});
    res.back().second++;
    x /= lp[x]; // can avoid division by storing extra array
  }
  return res;
};
```

FactorSqrt.h

Description: Easy factorisation. Returns pairs of prime factor, multiplicity
Time: $\mathcal{O}\left(\sqrt{N}\right)$. 4a65d9, 15 lines

```
vt<pi> factor(int x) {
  if (x == 1) return {};
  vt<pi> res;
  for (ll i = 2; i * i <= x; i++) { // ll: i * i overflows near
       INT_MAX
    if (x % i == 0) {
      res.pb({(int) i, 0});
      while (x % i == 0) {
        res.back().second++;
        x /= i;
      }
    }
  }
  if (x > 1) res.pb({x, 1});
  return res;
}
```

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Usage: `vi pr = eratosthenes(); // also sets isPrime`
Time: LIM=1e9 $\approx$ 1.5s
Memory: LIM/8 + 4 per prime bytes ($\approx$325MB at 1e9) df71e2, 20 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int) round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S + 1); pr.reserve(int(LIM / log(LIM) * 1.1));
  vt<pi> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.pb({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i = idx; i < S + L; idx = (i += p)) block[i - L] = 1;
    FOR (i, min(S, R - L))
      if (!block[i]) pr.pb((L + i) * 2 + 1);
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$. "ModMulLL.h" 168797, 12 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n - 1), d = n >> s;
  for (ull a : A) { // ^ count trailing zeroes
    ull p = mpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
```

```
    p = mmul(p, p, n);
    if (p != n - 1 && i != s) return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}). Consider manually trying small primes for better runtime. Needs $1 \le n < 7 \cdot 10^{18}$. In one second: average 350k numbers at 1e9, 200k numbers at 1e12, and 100k at 1e16. Worst-case semiprimes near 1e18: ~0.6ms each (~1500/s).

Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d100a4, 18 lines

```
ull pollard(ull n) { // returns random factor
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&] (ull x) { return mmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = mmul(prd, max(x, y) - min(x, y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vt<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

PrimeCnt.h

Description: Counts number of primes up to N.

Time: $\mathcal{O}\left(N^{3/4}/\log N\right)$, 60ms for $N = 10^{11}$, 2.5s for $N = 10^{13}$

a4ce83, 20 lines

```
ll count_primes(ll N) { // count_primes(1e13) == 346065536839
  if (N <= 1) return 0;
  int sq = (int) sqrt(N);
  vl big_ans((sq + 1) / 2), small_ans(sq + 1);
  FOR (i, 1, sq + 1) small_ans[i] = (i - 1) / 2;
  F0R (i, size(big_ans)) big_ans[i] = (N / (2 * i + 1) - 1) / 2;
  vt<bool> skip(sq + 1); int prime_cnt = 0;
  for (int p = 3; p <= sq; p += 2) if (!skip[p]) { // primes
    for (int j = p; j <= sq; j += 2 * p) skip[j] = 1;
    F0R (j, min((ll) size(big_ans), (N / p / p + 1) / 2)) {
      ll prod = (ll) (2 * j + 1) * p;
      big_ans[j] -= (prod > sq ? small_ans[(db) N / prod]
            : big_ans[prod / 2]) - prime_cnt;
    }
    for (int j = sq, q = sq / p; q >= p; --q) for (; j >= q * p; --j)
      small_ans[j] -= small_ans[q] - prime_cnt;
    ++prime_cnt;
  }
  return big_ans[0] + 1;
}
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then $x \equiv a^{-1} \pmod b$, $|x| \le b$.

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a / b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.

`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$ and $a \equiv b \pmod{\gcd(m, n)}$.

Time: $\log(n)$

"euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m * n / g : x;
}
```

5.3.1 Bézout's identity

For a, b not both 0, $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) :=$ # of positive integers $\le n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \ldots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1 - 1} \ldots (p_r - 1)p_r^{k_r - 1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$

Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$. Any a, $e \ge \log_2 n$: $a^e \equiv a^{e \bmod \phi(n) + \phi(n)} \pmod n$.

Fermat's little thm: p prime $\Rightarrow a^p \equiv a \pmod p$ $\forall a$; $a^{p-1} \equiv 1$ if $p \nmid a$.

Usage: `calculatePhi(); // phi[n] for n < LIM`

5bf43c, 8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  F0R (i, LIM) phi[i] = i & 1 ? i : i / 2;
  for (int i = 3; i < LIM; i += 2) if (phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

5.4.1 Continued Fractions

Let $a_0, a_1, \ldots, a_n \in \mathbb{Z}$ and $a_1, a_2, \ldots, a_n \ge 1$. Then the expression

$$r = a_0 + \cfrac{1}{a_1 + \cfrac{1}{\ddots + \cfrac{1}{a_n}}},$$

is called the *continued fraction representation* of the rational number r and is denoted shortly as

$$r = [a_0; a_1, a_2, \ldots, a_k].$$

If $a_0 > 0$ then $\frac{1}{r}$ is given by sticking a 0 in front. If $a_0 = 0$ then ditch the first term.

$[a_0, a_1, \ldots, a_n - 1]$ gives the run-length encoding $R^{a_0} L^{a_1} R^{a_2} \cdots$ of r's path from the root 1/1 in the Stern-Brocot tree.

$r_k = [a_0; a_1, a_2, \ldots, a_k] = \frac{p_k}{q_k}$ is called the k-th *convergent* of r.

$p_k = a_k p_{k-1} + p_{k-2}$, $q_k = a_k q_{k-1} + q_{k-2}$, $(p_{-1}, q_{-1}) = (1, 0)$, $(p_{-2}, q_{-2}) = (0, 1)$.

Semiconvergents are convergents but you can lower the last number down to 1.

Consider the convex hull of points above and below $y = rx$. Odd convergents $(q_k; p_k)$ give vertices of the upper hull, while even convergents give the vertices of the lower hull.

The set of semiconvergents = set of lattice points on the hulls.

Combine these facts with Pick's theorem ($A = I + B/2 - 1$) for funny stuff.

SBT.h

Description: Given p/q with $p \ge 0$, $q > 0$, find the shorter of its two unique continued fraction representations. (Negative p breaks: C++ division truncates instead of flooring.) The other representation is given by `a.back()-, a.pb(1);`

Time: $\mathcal{O}(\log N)$

0f01ea, 8 lines

```
vl cont_frac(ll p, ll q) {
  vl a;
  while (q) {
    a.pb(p / q);
    tie(p, q) = make_pair(q, p % q);
  }
  return a;
}
```

ContinuedFractions.h

Description: Given N and a real number $x \ge 0$, finds the closest rational approximation p/q with $p, q \le N$. It will obey $|p/q - x| \le 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.

Usage: `pl r = approximate(M_PI, 1000); // {355, 113}`

Time: $\mathcal{O}(\log N)$

56b36c, 21 lines

```
using d = db; // for N ~ 1e7; long double for N ~ 1e9
pl approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
  for (; ;) {
    ll lim = min(P ? (N - LP) / P : inf, Q ? (N - LQ) / Q : inf),
       a = (ll) floor(y), b = min(a, lim),
       NP = b * P + LP, NQ = b * Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // Return {P, Q} here for a more canonical approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1 / (y - (d)a)) > 3 * N) {
      return {NP, NQ};
    }
```

```
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}
```

FracBinarySearch.h

Description: Given f and N, finds the smallest fraction $p/q \in [0,1]$ such that $f(p/q)$ is true, and $p, q \le N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed. Needs f monotone with $f(1/1)$ true.
Usage: `fracBS([](Frac f) { return 3*f.p>=f.q; }, 10); // {1,3}`
Time: $\mathcal{O}(\log(N))$ 27ab3e, 25 lines

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$ (primitive root 7), which may be useful. For hashing use 970592641 (30-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.6.1 NTT primes

$p = c \cdot 2^a + 1$, result length $\le 2^a$. `root` must be a primitive root g of p (62 in NTT.h only fits the first five):
998244353 (c=119, a=23, g=3), 167772161 (5, 25, 3), 469762049 (7, 26, 3), 1004535809 (479, 21, 3), 2013265921 (15, 27, 31), 754974721 (45, 24, 11), 962592769 (459, 21, 7).

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most 100 for $n < 5e4$, 240 for $n < 1e6$, 768 for $n < 1e8$, 6720 for $n < 1e12$, 103 680 for $n < 1e18$.

Max $d(n)$ for $n < 10^k$: 100 for $5 \cdot 10^4$, 240 for 10^6 (720720), 768 for 10^8, 1344 for 10^9 (735134400), 6720 for 10^{12} (963761198400), 26880 for 10^{15} (866421317361600), 103 680 for 10^{18} (897612484786617600). Primorials: 2, 6, 30, 210, 2310, 30030, 510510, 9699690, 223092870, 6469693230, ...; $\omega(n) \le 9, 11, 15$ for $n < 10^9, 10^{12}, 10^{18}$.

k	3	4	5	6	7	8	9
$\pi(10^k)$	168	1229	9592	78498	664579	5761455	50847534

$\pi(10^{12}) = 37607912018$. Largest prime gap below 10^9: 282 (after 436273009), 10^{12}: 540, 10^{18}: 1442.

5.8 Floors

$\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/ab \rfloor$, $\lceil a/b \rceil = \lfloor (a+b-1)/b \rfloor$ for $a \ge 0, b > 0$.
$\lfloor n/i \rfloor$ takes $\le 2\sqrt{n}$ values, constant on blocks:

```
for (l = 1; l <= n; l = r + 1) {
  r = n / (n / l); /* i in [l,r] */ }
```

$\sum_{i \le n} \lfloor n/i \rfloor = \sum_{k \le n} d(k) = 2 \sum_{i \le \sqrt{n}} \lfloor n/i \rfloor - \lfloor \sqrt{n} \rfloor^2$,
$\sum_{i \le n} i \lfloor n/i \rfloor = \sum_{k \le n} \sigma(k)$.
Lattice points under a line: `divsum` (ModSum.h).

5.9 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

5.9.1 Multiplicative functions

$(f * g)(n) = \sum_{d|n} f(d) g(n/d)$. $\phi = \mu * \mathrm{id}$, $d = 1 * 1$, $\sigma = \mathrm{id} * 1$,
$\sigma(n) = \prod \frac{p_i^{e_i+1}-1}{p_i-1}$, $\prod_{d|n} d = n^{d(n)/2}$.

$$\sum_{i \le n, j \le m} [\gcd(i,j) = 1] = \sum_d \mu(d) \lfloor \tfrac{n}{d} \rfloor \lfloor \tfrac{m}{d} \rfloor$$
$$\sum_{i,j \le n} \gcd(i,j) = \sum_d \phi(d) \lfloor \tfrac{n}{d} \rfloor^2$$
$$\sum_{i \le n} \gcd(i,n) = \sum_{d|n} d\, \phi(n/d)$$
$$\sum_{i \le n} \mathrm{lcm}(i,n) = \tfrac{n}{2}\left(1 + \sum_{d|n} d\, \phi(d)\right)$$

Coprime pairs in $[1,n]^2$: $2\Phi(n) - 1$, $\Phi = \sum_{i \le n} \phi(i)$.
$\Phi(n) = \frac{n(n+1)}{2} - \sum_{d \ge 2} \Phi(\lfloor n/d \rfloor)$ in $O(n^{2/3})$ with a sieve up to $n^{2/3}$.

5.9.2 Ballot and reflection

Paths with a up- and b down-steps, $a \ge b$, never below 0: $\binom{a+b}{a} - \binom{a+b}{a+1}$; strictly above 0 after the start ($a > b$): $\frac{a-b}{a+b}\binom{a+b}{a}$.
Touching the line $y = -k$ ($k > 0$, $a - b > -k$): $\binom{a+b}{b-k}$ paths (reflect the part before the first touch).

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11		12		13		14	15	16	17
$n!$	4.0e7		4.8e8		6.2e9		8.7e10	1.3e12	2.1e13	3.6e14
n	20		25		30		40	50	100	150	171
$n!$	2e18		2e25		3e32		8e47	3e64	9e157	6e262	>DBL_MAX

IntPerm.h

Description: Permutation of $0..n-1$ ($n \le 12$) -> integer in $[0, n!)$. (Not order preserving.) Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$ 9a369b, 15 lines

```
int permToInt(vi &v) {
  int use = 0, i = 0, r = 0;
  for (int x : v) r = r * ++i + __builtin_popcount(use & -(1 << x)),
    use |= 1 << x; // (note: minus, not ~!)
  return r;
}
vi intToPerm(int r, int n) {
  vi a(n), v(n); iota(all(a), 0);
  for (int i = n; i; i--) {
    int c = r % i; r /= i;
    v[i - 1] = a[i - 1 - c];
    a.erase(begin(a) + i - 1 - c);
  }
  return v;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

$D(0) = 1,\ D(1) = 0$; the rounded form holds for $n \geq 1$.

6.1.4 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

6.1.5 Necklaces and bracelets

k colours, length n. Necklaces (rotations): $N = \frac{1}{n}\sum_{d|n}\phi(d)k^{n/d}$. Bracelets (also reflections): $\frac{1}{2}(N + k^{(n+1)/2})$ for odd n, $\frac{1}{2}\left(N + \frac{k+1}{2}k^{n/2}\right)$ for even n. Aperiodic necklaces (Lyndon words): $\frac{1}{n}\sum_{d|n}\mu(d)k^{n/d}$.

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(m<0) = 0,\ p(n) = \sum_{k\neq 0}(-1)^{k+1}p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

6.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \ldots + n_1 p + n_0$ and $m = m_k p^k + \ldots + m_1 p + m_0$ in base p. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

6.2.3 Inclusion–exclusion

With $s_i = \sum_{|S|=i}|\bigcap_S A|$, the number of elements in *exactly* j sets is $\sum_{i\geq j}(-1)^{i-j}\binom{i}{j}s_i$. Binomial inversion: $g_n = \sum_k \binom{n}{k}f_k \iff f_n = \sum_k (-1)^{n-k}\binom{n}{k}g_k$, e.g. surjections $[n]\to[k]$: $\sum_j (-1)^{k-j}\binom{k}{j}j^n$. $x_1 + \cdots + x_k = n$ with $0 \leq x_i \leq u$: $\sum_j (-1)^j\binom{k}{j}\binom{n-j(u+1)+k-1}{k-1}$. $\sum_{k\leq m}(-1)^k\binom{n}{k} = (-1)^m\binom{n-1}{m}$.

6.2.4 Binomials

multinomial.h

Description: Computes $\binom{k_1+\cdots+k_n}{k_1,k_2,\ldots,k_n} = \frac{(\sum k_i)!}{k_1!k_2!\ldots k_n!}$. ef3db0, 5 lines

```
ll multinomial(const vi& v) {
  __int128 c = 1; ll m = v.empty() ? 1 : v[0];
  FOR (i, 1, size(v)) F0R (j, v[i]) c = c * ++m / (j + 1);
  return c;
}
```

$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$; hockey stick $\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$; Vandermonde $\sum_j \binom{m}{j}\binom{n}{k-j} = \binom{m+n}{k}$; stars and bars: $\binom{n+k-1}{k-1}$ solutions of $x_1 + \cdots + x_k = n$, $x_i \geq 0$.

6.2.5 Lindström–Gessel–Viennot

Acyclic digraph, sources $a_1..a_k$, sinks $b_1..b_k$, M_{ij} = number of paths $a_i \to b_j$ (or sum of weights):

$$\det M = \sum_{\sigma}\operatorname{sgn}(\sigma)\cdot\#\{\text{vertex-disjoint tuples } a_i \to b_{\sigma(i)}\}.$$

If disjointness forces $\sigma = \mathrm{id}$ (e.g. sources and sinks on the boundary of a planar grid in matching order), $\det M$ counts the disjoint tuples. Grid paths: $M_{ij} = \binom{\Delta x+\Delta y}{\Delta x}$.

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able). $B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k\cdot(n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$

$$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Stirling identities

$x^n = \sum_k S(n,k)\,x^{\underline{k}}$, $x^{\overline{n}} = \sum_k c(n,k)x^k$, $x^{\underline{n}} = \sum_k(-1)^{n-k}c(n,k)x^k$. $\sum_{i<N} i^m = \sum_k S(m,k)\frac{N^{\underline{k+1}}}{k+1}$; $S(n,2) = 2^{n-1}-1$; $c(n,2) = (n-1)!H_{n-1}$. Row $S(n,\cdot)$: convolve $j^n/j!$ with $(-1)^i/i!$; row $c(n,\cdot)$: $\prod_{i<n}(x+i)$ by divide and conquer.

6.3.6 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

6.3.7 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.8 Prüfer codes

Labeled trees on n vertices $\leftrightarrow$ sequences in $\{0..n-1\}^{n-2}$ (a bijection, hence n^{n-2} trees); vertex v appears $\deg(v)-1$ times. Encode: repeatedly remove the smallest leaf and append its

neighbour. Decode: the leaf removed at step i is the smallest label that is not yet removed and does not occur in $a_i, \dots, a_{n-2}$; join it to a_i; finally join the two survivors. Both $O(n)$ with a pointer walk (`PruferCode.h`). Rooted forests with k trees rooted at $1..k$: $k\,n^{n-k-1}$. Uniform random tree: random sequence, decode.

PruferCode.h

Description: Prüfer code of a labeled tree, 0-indexed. encode takes the adjacency list ($n \geq 2$) and returns the $n-2$ codes; decode returns the $n-1$ edges. Bijection between trees on n vertices and sequences in $[0,n)^{n-2}$.
Time: $\mathcal{O}(n)$ eb3575, 31 lines

```
vi prufer_encode(vt<vi>& adj) {
  int n = size(adj), ptr = 0;
  vi par(n, -1), deg(n), code(n - 2), st{n - 1};
  while (size(st)) { // parents towards root n-1
    int u = st.back(); st.pop_back();
    for (int v : adj[u]) if (v != par[u]) par[v] = u, st.pb(v);
  }
  F0R (i, n) deg[i] = size(adj[i]);
  while (deg[ptr] != 1) ptr++;
  int leaf = ptr;
  F0R (i, n - 2) {
    int nxt = code[i] = par[leaf];
    if (--deg[nxt] == 1 && nxt < ptr) leaf = nxt;
    else { while (deg[++ptr] != 1); leaf = ptr; }
  }
  return code;
}

vt<pi> prufer_decode(vi& code) {
  int n = size(code) + 2, ptr = 0;
  vi deg(n, 1); for (int v : code) deg[v]++;
  while (deg[ptr] != 1) ptr++;
  int leaf = ptr; vt<pi> edges;
  for (int v : code) {
    edges.pb({leaf, v});
    if (--deg[v] == 1 && v < ptr) leaf = v;
    else { while (deg[++ptr] != 1); leaf = ptr; }
  }
  edges.pb({leaf, n - 1});
  return edges;
}
```

6.3.9 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.0.1 Theorems

A matching M is maximum **iff** there is no M-augmenting path.

7.0.2 Hall's Marriage Theorem

$G = (L, R; E)$ has a matching covering all of L iff $|N(S)| \geq |S|$ for every $S \subseteq L$.
Corollary: Let $\delta = \max_{S \subseteq L}\big(|S| - |N(S)|\big)$. The size of the maximum matching is $\nu = |L| - \delta$.

7.0.3 Kőnig's Theorem

In bipartite graphs: **max matching = min vertex cover**, i.e. $\nu(G) = \tau(G)$. Hence max independent set size $\alpha(G) = |V| - \tau(G) = |V| - \nu(G)$.
To recover min vertex cover: From all unmatched vertices in L, BFS alternating paths (use non-matching edges $L \to R$; matching edges $R \to L$). Let Z be reachable vertices. Min vertex cover is $(L \setminus Z) \cup (R \cap Z)$.

7.0.4 Dilworth's Theorem (posets)

Run Floyd-Warshall for transitivity.
Width (max antichain size) = min number of chains in a partition.
Reduction: make bipartite graph with two copies of elements; edge $x_L - y_R$ if $x < y$. Then width = min chain decomposition size = $|P| - \nu$.
Mirsky's Theorem (dual): Height (max chain size) = min number of antichains in a partition.
Max antichain: from a min vertex cover, take the x with neither x_L nor x_R in it.
Min vertex-disjoint path cover of a DAG = $|V| - \nu$, with an edge $x_L - y_R$ per DAG edge.

7.0.5 Tutte's 1-Factor Theorem

$\text{odd}(H)$ is the number of components of a graph H with an odd number of vertices. G has a perfect matching iff for all $S \subseteq V$, $\text{odd}(G - S) \leq |S|$. Tutte–Berge formula:
$\nu(G) = \frac{1}{2}\min_{S \subseteq V}\big(|V| - \text{odd}(G - S) + |S|\big)$.

7.0.6 Edge cover

In any graph without isolated vertices, the size of the minimum edge cover (set of edges that touch every vertex) is given by:

$$\rho'(G) = |V| - \nu(G)$$

7.1 Math

7.1.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if G is undirected). Remove the ith row and column and take the determinant; this yields the number of directed spanning trees rooted at i with edges pointing away from i (if G is undirected, remove any row/column).

7.1.2 Erdős-Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

7.1.3 Euler's formula (planar graphs)

A connected planar graph has $V - E + F = 2$ (counting the outer face); with C components, $V - E + F = 1 + C$. Simple planar with $V \geq 3$: $E \leq 3V - 6$ (and $E \leq 2V - 4$ if triangle-free), so average degree < 6.

7.1.4 BEST theorem

Connected digraph with indeg = outdeg = $\deg(v)$ everywhere: the number of Eulerian circuits (cyclic sequences of distinct edges) is $t_w(G)\prod_v(\deg(v)-1)!$, where t_w is the number of spanning arborescences rooted at w (any w; the matrix of Number of Spanning Trees). Euler trails $s \to t$: add an arc $t \to s$ first.

7.2 Fundamentals

Dijkstra.h

Description: Decrease-key keeps exactly one entry per vertex. 1.2–1.4x faster on dense graphs and when a vertex is relaxed often, up to 1.4x slower on sparse random and grid graphs.
Usage: `dijkstra(s) returns d; needs w >= 0, INF if unreachable`
Time: $\mathcal{O}(E + V \log V)$ 304af2, 22 lines

```
#include <bits/extc++.h>

using pq_t = __gnu_pbds::priority_queue<pl, greater<pl>>;
vt<vt<pl>> adj; // adj[u] = {{v, w}, ...}

vl dijkstra(int s) {
  vl d(size(adj), INF);
  vt<pq_t::point_iterator> it(size(adj));
  pq_t pq;
  d[s] = 0, it[s] = pq.push({0, s});
  while (!pq.empty()) {
    ll u = pq.top().s; pq.pop();
    // a popped u is final, so v below is still queued
    for (auto [v, w] : adj[u]) if (d[u] + w < d[v]) {
      bool queued = d[v] < INF;
      d[v] = d[u] + w;
      if (queued) pq.modify(it[v], {d[v], v});
      else it[v] = pq.push({d[v], v});
    }
  }
  return d;
}
```

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = INF; nodes reachable through negative-weight cycles get dist = -INF. Assumes $V^2 \max|w_i| < \sim 2^{63}$.
Usage: `vt<Node> nd(n); vt<Ed> eds = {{a, b, w}}; // directed bellmanFord(nd, eds, s); // dist in nd[v].dist`
Time: $\mathcal{O}(VE)$ 2ed625, 22 lines

```
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = INF; int prev = -1; };

void bellmanFord(vt<Node> &nodes, vt<Ed> &eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [] (Ed a, Ed b) { return a.s() < b.s(); });

  int lim = size(nodes) / 2 + 2; // /3 + 100 with shuffled vertices
  F0R (i, lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == INF) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim - 1 ? d : -INF);
    }
  }
  F0R (i, lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -INF)
      nodes[e.b].dist = -INF;
  }
}
```

DEsopoPape.h

Description: Mystery shortest path algorithm with exponential breaking cases. Works with negative weights; loops forever on a negative cycle.
Time: Linear to exponential, depending on how good data is.
031730, 23 lines

```
vt<vt<pair<int, ll>>> adj; // INF comes from the template

void shortest_paths(int s, vt<ll> &d, vi &p) {
  d.assign(n, INF);
  d[s] = 0;
  vi m(n, 2);
  deque<int> q {s};
  p.assign(n, -1);

  while (!q.empty()) {
    int u = q.front(); q.pop_front();
    m[u] = 0;
    for (auto [v, w] : adj[u]) {
      if (d[v] > d[u] + w) {
        d[v] = d[u] + w;
        p[v] = u;
        if (m[v] == 2) q.pb(v);
        else if (m[v] == 0) q.push_front(v);
        m[v] = 1;
      }
    }
  }
}
```

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m, where $m[i][j] =$ `INF` if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j, `INF` if no path, or `-INF` if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$ a025bf, 9 lines

```
void floydWarshall(vt<vt<ll>>& m) {
  int n = size(m);
  F0R (i, n) m[i][i] = min(m[i][i], 0LL);
  F0R (k, n) F0R (i, n) F0R (j, n)
    if (m[i][k] != INF && m[k][j] != INF)
      m[i][j] = min(m[i][j], max(m[i][k] + m[k][j], -INF));
  F0R (k, n) if (m[k][k] < 0) F0R (i, n) F0R (j, n)
    if (m[i][k] != INF && m[k][j] != INF) m[i][j] = -INF;
}
```

DirectedCycle.h

Description: Finds a directed cycle with a DFS stack.
Usage: `DirCyc<N> g; g.adj[u].pb(v); vi c = g.init(n); // c: cycle in order (c.back() -> c[0]), empty if acyclic`
81fac0, 17 lines

```
template<int sz> struct DirCyc {
  vi adj[sz], stk, cyc; vt<bool> in_stk, vis;
  void dfs(int x) {
    stk.pb(x); in_stk[x] = vis[x] = 1;
    for (int i : adj[x]) {
      if (in_stk[i]) cyc = {find(all(stk), i), end(stk)};
      else if (!vis[i]) dfs(i);
      if (size(cyc)) return;
    }
    stk.pop_back(); in_stk[x] = 0;
  }
  vi init(int n) {
    in_stk.resize(n), vis.resize(n);
    F0R (i, n) if (!vis[i] && !size(cyc)) dfs(i);
    return cyc;
  }
};
```

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V| + |E|)$ ce6987, 8 lines

```
vi topoSort(const vt<vi>& gr) {
  vi indeg(size(gr)), q;
  for (auto& li : gr) for (int x : li) indeg[x]++;
  F0R (i, size(gr)) if (indeg[i] == 0) q.pb(i);
  F0R (j, size(q)) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.pb(x);
  return q;
}
```

7.3 Network flow

7.3.1 Reductions

Min cut reduction

Consider min cut as an instance of this problem: Given $A[N]$ and $B[N]$ (any sign) and $C[N][N]$ (non-negative), find a binary string S of length N that minimises the following:

- Pay $A[i]$ if $S[i] = 1$
- Pay $B[i]$ if $S[i] = 0$
- Pay $C[i][j]$ if $S[i] = 1$ and $S[j] = 0$.

To calculate the answer ($S[i] = 1$ means i is on the s side):

- Add edges s to i of capacity $B[i] +$ `inf`.
- Add edges i to t of capacity $A[i] +$ `inf`.
- Add edges i to j of capacity $C[i][j]$.
- The answer is the maximum flow in this graph minus $N \times$ `inf`, where `inf` $\geq \max_i(-A[i], -B[i])$.

7.3.2 Circulation & Lower Bounds

Each edge $e = (u \to v)$ has bounds $[\ell_e, u_e]$. Find flows f_e such that:

$$\ell_e \leq f_e \leq u_e, \qquad \sum f_{\text{in}} = \sum f_{\text{out}} \text{ at every node.}$$

Let $f'_e = f_e - \ell_e$, so $0 \leq f'_e \leq u_e - \ell_e$. Define node demand:

$$d[v] = \sum_{e=(v\to w)} \ell_e - \sum_{e=(u\to v)} \ell_e$$

Then residual flow must satisfy

$$\sum f'_{\text{in}} - \sum f'_{\text{out}} = d[v].$$

Build auxiliary graph:

- For each edge $(u \to v)$, add cap $u_e - \ell_e$
- Add super source S, super sink T
- If $d[v] > 0$: add $v \to T$ cap $d[v]$
- If $d[v] < 0$: add $S \to v$ cap $-d[v]$

Let $D = \sum_{d[v]>0} d[v]$. Feasible circulation iff max $S \to T$ flow equals D.

To recover original flow, use $f_e = f'_e + \ell_e$. We can model max $s \to t$ flow with lower bounds by adding edge (t, s) with bounds $[L, \infty)$ and binary searching the largest feasible L.

FordFulkerson.h

Description: Short algo for computing maximum flows with a bounded answer.
Usage: `adj[a][b] += cap; flow(s, t);`
Time: $\mathcal{O}(FM)$ 4493f3, 17 lines

```
const int mx = 2000;
int seen[mx], tim;
unordered_map<int, int> adj[mx];

int flow(int s, int t) {
  assert(s != t);
  auto dfs = [&] (auto &&self, int u) {
    if (u == t) return 1;
    if (exchange(seen[u], tim) == tim) return 0;
    for (auto &[v, c] : adj[u])
      if (c && self(self, v)) return --adj[u][v], ++adj[v][u];
    return 0;
  };
  int flow = 0;
  while (tim++, dfs(dfs, s)) flow++;
```

```
  return flow;
}
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $O(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.
Usage: `vt<unordered_map<int, ll>> adj(n); adj[a][b] += cap;`
`edmondsKarp(adj, s, t); // adj becomes the residual graph`

249427, 32 lines

```
template<class T> T edmondsKarp(vt<unordered_map<int, T>>&
    adj, int s, int t) {
  assert(s != t);
  T flow = 0;
  vi par(size(adj)), q = par;
  while (1) {
    fill(all(par), -1);
    int ptr = 1;
    q[0] = s, par[s] = 0;
    F0R (i, ptr) {
      int u = q[i];
      for (auto &[v, c] : adj[u]) {
        if (par[v] == -1 && c) {
          par[v] = u, q[ptr++] = v;
          if (v == t) goto out;
        }
      }
    } // 73adb5
    return flow;
  out:
    T inc = numeric_limits<T>::max();
    for (int y = t; y != s; y = par[y])
      inc = min(inc, adj[par[y]][y]);

    flow += inc;
    for (int y = t; y != s; y = par[y]) {
      int p = par[y];
      if ((adj[p][y] -= inc) <= 0) adj[p].erase(y);
      adj[y][p] += inc;
    }
  }
}
```

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.
Usage: `init(n); ae(a, b, c, rcap = 0); // rcap = c: undirected`
`calc(s, t) once per graph.`

84ac12, 48 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    // ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vt<vt<Edge>> adj;

  void init(int n) {
    lvl = ptr = q = vi(n);
    adj.resize(n);
  }

  void ae(int a, int b, ll c, ll rcap = 0) {
    adj[a].pb({b, size(adj[b]), c, c});
    adj[b].pb({a, size(adj[a]) - 1, rcap, rcap});
  } // bb548e
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < size(adj[v]); i++) {
      auto &[to, rev, c, _] = adj[v][i];
      if (lvl[to] == lvl[v] + 1) {
        if (ll p = dfs(to, t, min(f, c))) {
          c -= p, adj[to][rev].c += p;
          return p;
        }
      }
    }
    return 0;
  } // fc8ac0
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    F0R (L, 31) do { // 'int L = 30' maybe faster for random data
      lvl = ptr = vi(size(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  // bool left_of_min_cut(int a) { return lvl[a] != 0; }
};
```

Dinic2.h

Description: Dinic.h, faster: 16-byte edges, one DFS per phase that keeps pushing after a success and kills dead ends, no capacity scaling. `calc(s, t, lim)` pushes at most lim more units from the current state and returns them, so between calls the residual graph may be edited as long as it stays a valid flow: both residuals of a pair ≥ 0 with a constant sum, and conservation at every vertex but s, t. Raise the cap of edge i of a by d: `adj[a][i].c += d`, then `calc(s, t)`. Lower it below its flow f (= the reverse residual if rcap = 0) to c': $x = f - c'$, `adj[a][i].c = 0`, reverse `.c -= x`, then `z = x - calc(a, b, x); calc(a, s, z); calc(t, b, z); flow -= z; flow += calc(s, t);`. `left_of_min_cut` is for the last `calc(s, t)`. A calc that finds nothing is one BFS, $O(V+E)$; one that finds F costs $O(E)$ per augmenting path and phase, so incremental use is never worse than from scratch.
Time: $\mathcal{O}\left(V^2E\right)$, $\mathcal{O}\left(E\sqrt{V}\right)$ on unit graphs. 1.5–2x Dinic.h without its scaling, 2–16x with it, 1.3–14x PushRelabel.h: doc/dinic.md

f3109e, 37 lines

```
struct Dinic {
  struct Edge { int to, rev; ll c; }; // rcap 0: flow = rev's c
  vt<vt<Edge>> adj; vi lvl, ptr, q;
  void init(int n) { adj.resize(n), lvl = ptr = q = vi(n); }
  void ae(int a, int b, ll c, ll rcap = 0) {
    adj[a].pb({b, size(adj[b]), c});
    adj[b].pb({a, size(adj[a]) - 1, rcap});
  } // d7b68a
  ll dfs(int v, int t, ll f) { // pushes up to f
    if (v == t) return f;
    ll g = 0;
    for (int &i = ptr[v]; i < size(adj[v]); i++) {
      auto &[to, rev, c] = adj[v][i];
      if (lvl[to] != lvl[v] + 1 || !c) continue;
      ll w = min(f - g, c), p = dfs(to, t, w);
      if (p < w) lvl[to] = -1; // nothing left behind it
      c -= p, adj[to][rev].c += p, g += p;
      if (g == f) break; // not i++: edge may have room
    }
    return g;
  } // 18040c
  ll calc(int s, int t, ll lim = LLONG_MAX) {
    ll flow = 0; q[0] = s;
    do {
      fill(all(lvl), 0), fill(all(ptr), 0);
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v]) if (!lvl[e.to] && e.c)
          q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      ll p = dfs(s, t, lim); flow += p, lim -= p;
    } while (lvl[t] && lim);
    return flow;
  }
  bool left_of_min_cut(int a) { return lvl[a]; }
};
```

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.
Usage: `PushRelabel<> D; D.init(n); D.ae(a, b, cap, rcap);`
`D.calc(s, t); // once per graph`
`D.leftOfMinCut(v); // v on source side of a min cut`
Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$

e04a50, 66 lines

```
template<typename flow_t = ll>
struct PushRelabel {
  struct Edge {
    int to, rev;
    flow_t f, c;
  };
  vt<vt<Edge>> g;
  vt<flow_t> ec;
  vt<Edge*> cur;
  vt<vi> hs;
  vi h;

  void init(int n) {
    g.resize(n);
    ec.resize(n);
    cur.resize(n);
    hs.resize(2 * n);
    h.resize(n);
  } // 1f1aa3

  void ae(int s, int t, flow_t cap, flow_t rcap = 0) {
    if (s == t) return;
    g[s].pb({t, size(g[t]), 0, cap});
    g[t].pb({s, size(g[s]) - 1, 0, rcap});
  }
  void add_flow(Edge& e, flow_t f) {
    Edge &back = g[e.to][e.rev];
    if (!ec[e.to] && f)
      hs[h[e.to]].pb(e.to);
    e.f += f; e.c -= f;
    ec[e.to] += f;
    back.f -= f; back.c += f;
    ec[back.to] -= f;
  } // aacd76
  flow_t calc(int s, int t) {
    int v = size(g);
    h[s] = v;
    ec[t] = 1;
    vi co(2 * v);
    co[0] = v - 1;
    F0R (i, v) cur[i] = g[i].data();
    for (auto &e : g[s]) add_flow(e, e.c); // a3e44f
```

```
    if (size(hs[0]))
    for (int hi = 0; hi >= 0;) {
      int u = hs[hi].back();
      hs[hi].pop_back();
      while (ec[u] > 0) // discharge u
        if (cur[u] == g[u].data() + size(g[u])) {
          h[u] = 1e9;
          for (auto &e : g[u])
            if (e.c && h[u] > h[e.to] + 1)
              h[u] = h[e.to] + 1, cur[u] = &e;
          if (++co[h[u]], !--co[hi] && hi < v)
            F0R (i, v)
              if (hi < h[i] && h[i] < v)
                --co[h[i]], h[i] = v + 1;
          hi = h[u]; // 286095
        } else if (cur[u]->c && h[u] == h[cur[u]->to] + 1)
          add_flow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
      while (hi >= 0 && hs[hi].empty()) --hi;
    }
    return -ec[s];
  }
  bool leftOfMinCut(int a) { return h[a] >= size(g); }
};
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only. The edge added by `ae(u, ...)` is `ed[u][k]`, where `k = size(ed[u])` before the call.
Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

850ba5, 79 lines

```
#include <bits/extc++.h>

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int n;
  vt<vt<edge>> ed;
  vi seen;
  vl dist, pi;
  vt<edge*> par; // 416844

  MCMF(int n) : n(n), ed(n), seen(n), dist(n), pi(n), par(n) {}

  void ae(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].pb({from, to, size(ed[to]),
      cap, cost, 0});
    ed[to].pb({ to, from, size(ed[from]) - 1,
      0, -cost, 0});
  }

  void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vt<decltype(q)::point_iterator> its(n);
    q.push({ 0, s }); // b23e94

    while (!q.empty()) {
      s = q.top().second; q.pop();
      seen[s] = 1; di = dist[s] + pi[s];
      for (edge& e : ed[s]) if (!seen[e.to]) {
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
          dist[e.to] = val;
          par[e.to] = &e;
          if (its[e.to] == q.end())
            its[e.to] = q.push({ -dist[e.to], e.to });
          else
            q.modify(its[e.to], { -dist[e.to], e.to });
        }
      }
    }
    F0R (i, n) pi[i] = min(pi[i] + dist[i], INF);
  } // 5ba20b

  pl maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    F0R (i, n) for (edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost / 2};
  }

  // If some costs can be negative, call this before maxflow:
  // void setpi(int s) { // (otherwise, leave this out)
  //   fill(all(pi), INF); pi[s] = 0;
  //   int it = n, ch = 1; ll v;
  //   while (ch-- && it--)
  //     F0R (i, n) if (pi[i] != INF)
  //       for (edge& e : ed[i]) if (e.cap)
  //         if ((v = pi[i] + e.cost) < pi[e.to])
  //           pi[e.to] = v, ch = 1;
  //   assert(it >= 0); // negative cost cycle
  // }
};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity. Implemented as `left_of_min_cut(a)` in Dinic (commented out) and Dinic2, `leftOfMinCut(a)` in PushRelabel.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix. Weights are ll; keep the total graph weight below ~1e17.
Usage: `mat[a][b] += w, mat[b][a] += w; returns {weight, one side}`
Time: $\mathcal{O}(V^3)$

5fae9b, 21 lines

```
pair<ll, vi> globalMinCut(vt<vl> mat) {
  pair<ll, vi> best = {INF, {}};
  int n = size(mat);
  vt<vi> co(n);
  F0R (i, n) co[i] = {i};
  FOR (ph, 1, n) {
    vl w = mat[0];
    size_t s = 0, t = 0;
    F0R (it, n - ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = -INF;
      s = t, t = max_element(all(w)) - w.begin();
      F0R (i, n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    F0R (i, n) mat[s][i] += mat[t][i];
    F0R (i, n) mat[i][s] = mat[s][i];
    mat[0][t] = -INF;
  }
  return best;
}
```

NetworkSimplex.h

Description: Solves minimum cost circulation problem. To convert to a "normal" mcmf, add an edge from t -> s of big capacity and cost below $-(V \cdot \max|cost|)$ (NOT -1e18: costs multiply flows and accumulate into duals). Add it back onto the answer! If you don't need max flow, give the t -> s edge cost 0. Edge i (one indexed) is ns.edges[2 * i], its flow is ns.edges[2 * i + 1].cap. Works with negative cost cycles. Flow is int (infinite capacity: inf). Set Flow = ll and use INF if capacities/flows exceed 2^{31}. Total |cost x flow| must fit in ll.
Time: $\mathcal{O}(VE)$ on average maybe

b06bb1, 94 lines

```
struct NetworkSimplex {
  using Flow = int;
  using Cost = ll;
  struct Edge {
    int nxt, to;
    Flow cap;
    Cost cost;
  };
  vt<Edge> edges;
  vi head, fa, fe, mark, cyc;
  vt<Cost> dual;
  int ti; // fbbb5f

  NetworkSimplex(int n)
      : head(n, 0), fa(n), fe(n), mark(n), cyc(n + 1), dual(n), ti(0)
          {
    edges.pb({0, 0, 0, 0});
    edges.pb({0, 0, 0, 0});
  }

  int ae(int u, int v, Flow cap, Cost cost) {
    assert(size(edges) % 2 == 0);
    int e = size(edges);
    edges.pb({head[u], v, cap, cost});
    head[u] = e;
    edges.pb({head[v], u, 0, -cost});
    head[v] = e + 1;
    return e;
  } // d99a54

  void init_tree(int x) {
    mark[x] = 1;
    for (int i = head[x]; i; i = edges[i].nxt) {
      int v = edges[i].to;
      if (!mark[v] and edges[i].cap) {
        fa[v] = x, fe[v] = i;
        init_tree(v);
      }
    }
  }

  Cost phi(int x) {
    if (mark[x] == ti) return dual[x];
    return mark[x] = ti, dual[x] = phi(fa[x]) - edges[fe[x]].cost;
  } // c4bf9a
```

```
  void push_flow(int e, Cost &cost) {
    int pen = edges[e ^ 1].to, lca = edges[e].to;
    ti++;
    while (pen) mark[pen] = ti, pen = fa[pen];
    while (mark[lca] != ti) mark[lca] = ti, lca = fa[lca];

    int e2 = 0, path = 2, clen = 0; Flow f = edges[e].cap;
    for (int i = edges[e ^ 1].to; i != lca; i = fa[i]) {
      cyc[++clen] = fe[i];
      if (edges[fe[i]].cap < f)
        f = edges[fe[e2 = i] ^ (path = 0)].cap;
    }
    for (int i = edges[e].to; i != lca; i = fa[i]) {
      cyc[++clen] = fe[i] ^ 1;
      if (edges[fe[i] ^ 1].cap <= f)
        f = edges[fe[e2 = i] ^ (path = 1)].cap;
    } // db6cfc
    cyc[++clen] = e;

    for (int i = 1; i <= clen; ++i) {
      edges[cyc[i]].cap -= f, edges[cyc[i] ^ 1].cap += f;
      cost += edges[cyc[i]].cost * f;
    }
    if (path == 2) return;

    int laste = e ^ path, last = edges[laste].to, cur = edges[laste ^
        1].to;
    while (last != e2) {
      mark[cur]--;
      laste ^= 1;
      swap(laste, fe[cur]);
      swap(last, fa[cur]); swap(last, cur);
    }
  } // fe9ef2

  Cost compute() {
    Cost cost = 0;
    if (size(edges) <= 2) return 0; // no ae() calls
    init_tree(0);
    mark[0] = ti = 2, fa[0] = cost = 0;
    int ncnt = size(edges) - 1;
    for (int i = 2, pre = ncnt; i != pre; i = i == ncnt ? 2 : i + 1)
      if (edges[i].cap and edges[i].cost < phi(edges[i ^ 1].to) - phi(
          edges[i].to))
        push_flow(pre = i, cost);
    ti++;
    for (int u = 0; u < size(dual); ++u)
      phi(u);
    return cost;
  }
};
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Usage: gomoryHu(N, {{a, b, w}, ...}) returns {v, parent, w} edges
Time: $\mathcal{O}(V)$ Flow Computations "PushRelabel.h" 292d55, 14 lines

```
using Edge = array<ll, 3>;
vt<Edge> gomoryHu(int N, vt<Edge> ed) {
  vt<Edge> tree;
  vi par(N);
  FOR (i, 1, N) {
    PushRelabel D; // Dinic also works
    D.init(N);
    for (Edge t : ed) D.ae(t[0], t[1], t[2], t[2]);
    tree.pb({i, par[i], D.calc(i, par[i])});
    FOR (j, i + 1, N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
}
```

7.4 Matching

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: vi btoa(m, -1); dfsMatching(g, btoa);
Time: $\mathcal{O}(VE)$ c2ed08, 22 lines

```
bool find(int j, vt<vi> &g, vi &btoa, vi &vis) {
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vt<vi> &g, vi &btoa) {
  vi vis;
  F0R (i, size(g)) {
    vis.assign(size(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return size(btoa) - (int) count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.
Usage: vi c = cover(g, n, m); // g[i]: right nbrs of left i
// n, m = left, right sizes; right j is returned as n + j
"DFSMatching.h" 3a064b, 20 lines

```
vi cover(vt<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vt<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  F0R (i, n) if (lfound[i]) q.pb(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.pb(match[e]);
    }
  }
  F0R (i, n) if (!lfound[i]) cover.pb(i);
  F0R (i, m) if (seen[i]) cover.pb(n + i);
  assert(size(cover) == res);
  return cover;
}
```

hopcroftKarp.h

Description: Fast incremental bipartite matching. Zero-indexed.
Usage: Matching M(m); // m = size of rhs
M.add(nbrs); // next lhs node (0, 1, ..); 1 if matching grew
M[j]; // lhs node matched to rhs j, or -1
Time: $\mathcal{O}\left(\sqrt{V}E\right)$ fc3e44, 40 lines

```
struct Matching : vi {
  vt<vi> adj;
  vi rank, low, pos, vis, seen;
  int k{0};
  // n = size of rhs
  Matching(int n) : vi(n, -1), rank(n) {}
  bool add(vi vec) {
    adj.pb(std::move(vec));
    low.pb(0), pos.pb(0), vis.pb(0); // 26ebbf
    if (size(adj.back())) {
      int i = k;
    nxt:
      seen.clear();
      if (dfs(size(adj) - 1, ++k - i)) return 1;
      for (auto &v : seen) for (auto &e : adj[v]) {
        if (rank[e] < inf && vis[at(e)] < k) goto nxt;
      }
      for (auto &v : seen) {
        low[v] = inf;
        for (auto &w : adj[v]) rank[w] = inf;
      }
    }
    return 0;
  } // e73b61
  bool dfs(int v, int g) {
    if (vis[v] < k) vis[v] = k, seen.pb(v);
    while (low[v] < g) {
      int e = adj[v][pos[v]];
      if (at(e) != v && low[v] == rank[e]) {
        rank[e]++;
        if (at(e) == -1 || dfs(at(e), rank[e])) {
          return at(e) = v, 1;
        }
      } else if (++pos[v] == size(adj[v])) {
        pos[v] = 0; low[v]++;
      }
    }
    return 0;
  }
};
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$. Costs must be < INF; forbid a pair with e.g. 10^{12}.
Usage: auto [cost, match] = hungarian(a); // a is vt<vl>
Time: $\mathcal{O}\left(N^2M\right)$ 27de0f, 31 lines

```
pair<ll, vi> hungarian(const vt<vl> &a) {
  if (a.empty()) return {0, {}};
  int n = size(a) + 1, m = size(a[0]) + 1;
  vl u(n), v(m); vi p(m), ans(n - 1);
  FOR (i, 1, n) {
    p[0] = i;
    int j0 = 0;
    vl dist(m, INF); vi pre(m, -1);
    vt<bool> done(m + 1);
    do {
      done[j0] = true;
      int i0 = p[j0], j1; ll delta = INF;
```

```
      FOR (j, 1, m) if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
        if (dist[j] < delta) delta = dist[j], j1 = j;
      } // f9de1e
      FOR (j, m) {
        if (done[j]) u[p[j]] += delta, v[j] -= delta;
        else dist[j] -= delta;
      }
      j0 = j1;
    } while (p[j0]);
    while (j0) { // update alternating path
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;
    }
  }
  FOR (j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans}; // min cost
}
```

Blossom.h

Description: Matching for general graphs. 1-indexed!
Usage: `Blossom b; b.init(n); b.ae(u, v); // once per edge`
`b.solve() = matching size; b.mate[v] = partner, 0 if none.`
Time: $\mathcal{O}(NM)$ ffee1f, 60 lines

```
// 1-indexed!
struct Blossom {
  int n, h, t, cnt = 0;
  vt<pi> edges;
  vi vis, q, mate, col, fa, pre, he;
  void ae(int u, int v) {
    assert(u && v);
    edges.pb({he[u], v}); he[u] = size(edges) - 1;
    edges.pb({he[v], u}); he[v] = size(edges) - 1;
  } // 12b664
  inline int get(int u) { return fa[u] == u ? u : fa[u] = get(fa[u]);
      }
  void aug(int u, int v) {
    for (int p; u; u = p, v = pre[p])
      p = mate[v], mate[mate[u] = v] = u;
  }
  void init(int _n) {
    n = _n, cnt = 0;
    edges.assign(1, {});
    vis = q = mate = col = fa = pre = he = vi(n + 1);
  } // 64180e
  int lca(int u, int v) {
    for (cnt++; ; u = pre[mate[u]]) {
      if (v) swap(u, v);
      if (vis[u = get(u)] == cnt) return u;
      vis[u] = cnt;
    }
  }
  void blo(int u, int v, int f) {
    for (int p; get(u) != f; v = p, u = pre[p]) {
      p = mate[u]; pre[u] = v; fa[u] = fa[p] = f;
      if (col[p] != 1) col[q[++t] = p] = 1;
    }
  } // 003757
  bool bfs(int u) {
    FOR (i, 1, n + 1) col[i] = 0, fa[i] = i;
    h = 0; q[t = 1] = u; col[u] = 1;
    while (h != t) {
      int x = q[++h];
      for (int i = he[x]; i; i = edges[i].f) {
        int y = edges[i].s;
        if (!col[y]) {
          if (!mate[y]) { aug(y, x); return 1; }
          pre[y] = x;
          col[y] = 2;
          col[q[++t] = mate[y]] = 1; // 8bf5a6
        } else if (col[y] == 1 && get(x) != get(y)) {
          int p = lca(x, y);
          blo(x, y, p);
          blo(y, x, p);
        }
      }
    }
    return 0;
  }
  int solve() {
    int ans = 0;
    FOR (i, 1, n + 1) if (!mate[i]) ans += bfs(i);
    return ans;
  }
};
```

GabowMatching.h

Description: Maximum matching in a general graph, 1-indexed; same interface as Blossom. Self-loops and multi-edges are fine.
Usage: `GabowMatching g; g.init(n); g.ae(u, v); // once per edge`
`g.solve() = matching size; g.mate[v] = partner, 0 if none.`
Time: $\mathcal{O}\left(\sqrt{N}M \log N\right)$; 10^6 vertices, $2 \cdot 10^6$ random edges: 1s. Odd cycles hanging off a long cycle: 2s at $3 \cdot 10^5$. 607aea, 153 lines

```
struct GabowMatching {
  int n, nh, tc, ta, oid, qh, lgs;
  vt<vi> g, bl;
  vt<vt<pi>> e;
  vi mate, pot, lab, par, q, st;
  vt<pi> lk, lg;
  void init(int _n) { n = _n, g.assign(n + 1, {}), bl = g; }
  void ae(int u, int v) { g[u].pb(v), g[v].pb(u); }
  int find(int u) {
    return par[u] == u ? u : par[u] = find(par[u]);
  }
  void join(int x, int y) {
    if (!lab[y]) {
      int z = mate[y];
      lab[y] = -1, pot[y] = tc, lk[z] = {x, y};
      lab[z] = lab[x], pot[z] = tc + 1, q.pb(z);
      return;
    }
    int bx = find(x), by = find(y), h = -2 - size(lg), l;
    lab[mate[bx]] = lab[mate[by]] = h;
    while (1) {
      if (mate[by]) swap(bx, by);
      bx = l = find(lk[bx].f);
      if (lab[mate[bx]] == h) break;
      lab[mate[bx]] = h;
    }
    for (int b : {par[x], par[y]})
      for (; b != l; b = par[lk[b].f]) {
        int m = mate[b];
        lk[m] = {x, y}, lab[m] = lab[x], q.pb(m);
        pot[m] = 2 * tc + 1 - pot[m];
        par[b] = par[m] = l, lg.pb({b, l}), lg.pb({m, l});
      }
  } // 85cb41
  bool path() {
    while (qh < size(q)) {
      int x = q[qh++], lx = lab[x], px = pot[x];
      int bx = find(x);
      for (int y : g[x]) {
        if (lab[y] < 0) continue;
        int t = lab[y] ? (px + pot[y]) >> 1 : px + 1;
        if (lab[y] > 0 && lx != lab[y]) {
          if (t == tc) return 1;
          ta = min(ta, t);
        } else if (!lab[y] || bx != find(y)) {
          if (t == tc) join(x, y), bx = find(x);
          else if (t <= nh) e[t].pb({x, y});
        }
      }
    }
    return 0;
  } // 43650e
  bool dual() { // 1 if the matching is maximum
    int lim = min(nh + 1, ta);
    for (++tc; lgs = size(lg), tc < lim; ++tc) {
      bool up = 0;
      for (auto [x, y] : e[tc]) {
        if (lab[y] > 0) {
          if (pot[x] + pot[y] != 2 * tc) continue;
          if (find(x) == find(y)) continue;
          if (lab[x] != lab[y]) return ta = tc, 0;
        } else if (lab[y]) continue;
        join(x, y), up = 1;
      }
      if (up) return 0;
    }
    return tc > nh;
  } // e55d27
  void rematch(int v, int w) {
    int t = mate[v]; mate[v] = w;
    if (mate[t] != v) return;
    auto [x, y] = lk[v];
    if (y == find(y)) mate[t] = x, rematch(x, t);
    else rematch(x, y), rematch(y, x);
  }
  bool dfs(int x, int bx) {
    int px = pot[x], lx = lab[bx];
    for (int y : g[x]) {
      if (px + pot[y]) continue;
      int by = find(y), ly = lab[by];
      if (ly > 0) {
        if (lx >= ly) continue;
        int b = size(st);
        for (int v = by; v != bx; v = find(lk[v].f)) {
          int w = find(mate[v]);
          st.pb(w), lk[w] = {x, y}, par[v] = par[w] = bx;
        }
        for (int i = size(st); i-- > b; st.pop_back())
          for (int u : bl[st[i]])
            if (dfs(u, bx)) return 1;
      } else if (!ly) {
        lab[by] = -1;
        int z = mate[by];
        if (!z) return rematch(x, y), rematch(y, x), 1;
        int bz = find(z);
        lk[bz] = {x, y}, lab[bz] = oid++;
        for (int u : bl[bz]) if (dfs(u, bz)) return 1;
      }
    }
    return 0;
  } // 16fb9c
  int init_match() { // greedy, degree-1 vertices first
    int r = 0;
    vi d(n + 1), s, t;
    auto take = [&](int u) {
      int b = 0;
      for (int v : g[u])
        if (v != u && !mate[v] && (!b || d[v] < d[b])) b = v;
      if (!b) return;
```

```
    mate[u] = b, mate[b] = u, r++;
    for (int w : {u, b})
      for (int x : g[w])
        if (!mate[x]) (--d[x] == 1 ? s : t).pb(x);
  };
  FOR (u, 1, n + 1) if ((d[u] = size(g[u])) == 1) s.pb(u);
  FOR (u, 1, n + 1) for (t.pb(u); size(s) + size(t); ) {
    auto& z = size(s) ? s : t;
    int x = z.back(); z.pop_back();
    if (!mate[x]) take(x);
  }
  return r;
 }
 int solve() {
  nh = n / 2, mate = pot = lab = par = vi(n + 1);
  lk.assign(n + 1, {}), e.assign(n + 1, {});
  int r = init_match(); // optional, or r = 0
  while (2 * r + 1 < n) {
    tc = qh = lgs = 0, ta = inf, oid = 1;
    q.clear(), lg.clear(), st.clear();
    FOR (u, 0, n + 1) {
      pot[u] = 1, par[u] = u, bl[u].clear(), e[u].clear();
      lab[u] = u && !mate[u] ? (q.pb(u), u) : 0;
    }
    while (!path()) {
      if (dual()) return r;
      if (tc == ta) break;
    }
    FOR (u, 1, n + 1) {
      if (lab[u] > 0) pot[u] -= tc;
      else if (lab[u] < 0) pot[u] = 1 + tc - pot[u];
      lab[u] = 0, par[u] = u;
    }
    F0R (i, lgs) par[lg[i].f] = lg[i].s;
    FOR (u, 1, n + 1) bl[find(u)].pb(u);
    FOR (u, 1, n + 1) if (!mate[u] && !lab[par[u]]) {
      int b = par[u];
      lab[b] = oid++;
      for (int v : bl[b]) if (dfs(v, b)) { r++; break; }
    }
  }
  return r;
 } // cb501d
};
```

WeightedBlossom.h

Description: Maximum weight matching in a general graph (not necessarily perfect), 1-indexed, adjacency matrix. Edges with $w \leq 0$ and self-loops are never used; parallel edges keep the max.
Usage: `WeightedBlossom g; g.init(n); g.ae(u, v, w);`
`ll best = g.solve(); g.mate[v] = partner, 0 if none.`
Time: $\mathcal{O}\left(N^3\right)$
Memory: $(2N)^2$ edges of 16 bytes — 005a60, 160 lines

```
struct WeightedBlossom {
  struct E { int u, v; ll w; };
  int n, nx, tm = 0;
  vt<vt<E>> g;
  vl lab;
  vi mate, sl, st, pa, S, vis;
  vt<vi> fl, ff; // cycle of b; ff[b][x] = child of b with x
  queue<int> q;
  void init(int _n) {
    n = _n; int m = 2 * n + 1;
    g.assign(m, vt<E>(m));
    F0R (u, m) F0R (v, m) g[u][v] = {u, v, 0};
    lab = vl(m), mate = sl = st = pa = S = vis = vi(m);
    fl.assign(m, {}), ff.assign(m, vi(n + 1));
  }
  void ae(int u, int v, ll w) {
    if (u != v) g[u][v].w = g[v][u].w = max(g[u][v].w, w);
  }
  ll d(E e) { return lab[e.u] + lab[e.v] - 2 * g[e.u][e.v].w; }
  void upd(int u, int x) {
    if (!sl[x] || d(g[u][x]) < d(g[sl[x]][x])) sl[x] = u;
  }
  void setsl(int x) {
    sl[x] = 0;
    FOR (u, 1, n + 1)
      if (g[u][x].w > 0 && st[u] != x && !S[st[u]]) upd(u, x);
  }
  void push(int x) {
    if (x <= n) q.push(x);
    else for (int y : fl[x]) push(y);
  }
  void setst(int x, int b) {
    st[x] = b;
    if (x > n) for (int y : fl[x]) setst(y, b);
  }
  int pos(int b, int x) { // even position of x in fl[b]
    int p = int(find(all(fl[b]), x) - begin(fl[b]));
    if (p % 2 == 0) return p;
    reverse(begin(fl[b]) + 1, end(fl[b]));
    return size(fl[b]) - p;
  } // e8ed53
  void setm(int u, int v) {
    mate[u] = g[u][v].v;
    if (u <= n) return;
    int x = ff[u][g[u][v].u], p = pos(u, x);
    F0R (i, p) setm(fl[u][i], fl[u][i ^ 1]);
    setm(x, v);
    rotate(begin(fl[u]), begin(fl[u]) + p, end(fl[u]));
  }
  void aug(int u, int v) {
    for (int w; (w = st[mate[u]]); u = st[pa[w]], v = w)
      setm(u, v), setm(w, st[pa[w]]);
    setm(u, v);
  }
  int lca(int u, int v) {
    for (++tm; u || v; swap(u, v)) {
      if (!u) continue;
      if (vis[u] == tm) return u;
      vis[u] = tm, u = st[mate[u]];
      if (u) u = st[pa[u]];
    }
    return 0;
  } // 85e19e
  void add(int u, int l, int v) {
    int b = n + 1;
    while (b <= nx && st[b]) b++;
    if (b > nx) nx++;
    lab[b] = S[b] = 0, mate[b] = mate[l], fl[b] = {l};
    for (int x = u, y; x != l; x = st[pa[y]])
      fl[b].pb(x), fl[b].pb(y = st[mate[x]]), push(y);
    reverse(begin(fl[b]) + 1, end(fl[b]));
    for (int x = v, y; x != l; x = st[pa[y]])
      fl[b].pb(x), fl[b].pb(y = st[mate[x]]), push(y);
    setst(b, b);
    FOR (x, 1, nx + 1) g[b][x].w = g[x][b].w = 0;
    FOR (x, 1, n + 1) ff[b][x] = 0;
    for (int c : fl[b]) {
      FOR (x, 1, nx + 1)
        if (!g[b][x].w || d(g[c][x]) < d(g[b][x]))
          g[b][x] = g[c][x], g[x][b] = g[x][c];
      FOR (x, 1, n + 1) if (ff[c][x]) ff[b][x] = c;
    }
    setsl(b);
  }
  void expand(int b) {
    for (int x : fl[b]) setst(x, x);
    int r = ff[b][g[b][pa[b]].u], p = pos(b, r);
    for (int i = 0; i < p; i += 2) {
      int x = fl[b][i], y = fl[b][i + 1];
      pa[x] = g[y][x].u, S[x] = 1, S[y] = 0;
      sl[x] = 0, setsl(y), push(y);
    }
    S[r] = 1, pa[r] = pa[b];
    FOR (i, p + 1, size(fl[b]))
      S[fl[b][i]] = -1, setsl(fl[b][i]);
    st[b] = 0;
  } // 86e358
  bool found(E e) { // tight edge e from an outer vertex
    int u = st[e.u], v = st[e.v];
    if (S[v] == -1) {
      int w = st[mate[v]];
      pa[v] = e.u, S[v] = 1, sl[v] = sl[w] = S[w] = 0, push(w);
    } else if (!S[v]) {
      int l = lca(u, v);
      if (!l) return aug(u, v), aug(v, u), 1;
      add(u, l, v);
    }
    return 0;
  }
  bool phase() { // 1 if the matching grew
    q = {};
    FOR (x, 1, nx + 1) S[x] = -1, sl[x] = 0;
    FOR (x, 1, nx + 1)
      if (st[x] == x && !mate[x]) pa[x] = S[x] = 0, push(x);
    if (q.empty()) return 0;
    while (1) {
      for (; size(q); q.pop()) {
        int u = q.front();
        if (S[st[u]] == 1) continue;
        FOR (v, 1, n + 1) {
          if (g[u][v].w <= 0 || st[u] == st[v]) continue;
          if (d(g[u][v])) upd(u, st[v]);
          else if (found(g[u][v])) return 1;
        }
      }
      ll t = INF;
      FOR (b, n + 1, nx + 1)
        if (st[b] == b && S[b] == 1) t = min(t, lab[b] / 2);
      FOR (x, 1, nx + 1) if (st[x] == x && sl[x] && S[x] != 1)
        t = min(t, d(g[sl[x]][x]) / (S[x] ? 1 : 2));
      FOR (u, 1, n + 1) {
        if (!S[st[u]]) {
          if (lab[u] <= t) return 0;
          lab[u] -= t;
        } else if (S[st[u]] == 1) lab[u] += t;
      }
      FOR (b, n + 1, nx + 1) if (st[b] == b && S[b] >= 0)
        lab[b] += S[b] ? -2 * t : 2 * t;
      q = {};
      FOR (x, 1, nx + 1)
        if (st[x] == x && sl[x] && st[sl[x]] != x &&
            !d(g[sl[x]][x]) && found(g[sl[x]][x])) return 1;
      FOR (b, n + 1, nx + 1)
        if (st[b] == b && S[b] == 1 && !lab[b]) expand(b);
    }
  } // 61abdd
  ll solve() {
    int m = 2 * n + 1;
    mate = st = vi(m), nx = n;
    F0R (u, n + 1) st[u] = u, fl[u].clear();
    ll mx = 0;
```

```
    FOR (u, 1, n + 1) FOR (v, 1, n + 1)
      ff[u][v] = u == v ? u : 0, mx = max(mx, g[u][v].w);
    FOR (u, 1, n + 1) lab[u] = mx;
    while (phase());
    ll r = 0;
    FOR (u, 1, n + 1) if (mate[u] > u) r += g[u][mate[u]].w;
    return r;
  } // dbd982
};
```

GabowWeighted.h

Description: Maximum weight matching in a general graph (not necessarily perfect), 1-indexed. Weights $|w| \leq 10^9$; negative edges and self-loops are never used.
Usage: `GabowWeighted g; g.init(n); g.ae(u, v, w); ll best = g.solve(); g.mate[v] = partner, 0 if none.`
Time: $\mathcal{O}(NM \log N)$. Include `bits/extc++.h` before the template.

f62260, 253 lines

```
#include <bits/extc++.h>

struct GabowWeighted {
  using A = array<ll, 4>; // {time, from, to, id}
  using PQ = __gnu_pbds::priority_queue<A, greater<A>>;
  struct Heaps { // item i: in heap w[i] (-1: none) at it[i]
    vt<PQ> s; vt<PQ::point_iterator> it; vi w;
    void init(int h, int m) {
      s.assign(h, {}), it.assign(m, {}), w.assign(m, -1);
    }
    void er(int i) {
      if (w[i] >= 0) s[w[i]].erase(it[i]), w[i] = -1;
    }
    void put(int i, A v, int h = 0) {
      v[3] = i;
      if (w[i] == h) return s[h].modify(it[i], v);
      er(i), it[i] = s[h].push(v), w[i] = h;
    }
    A top(int h = 0) {
      return s[h].empty() ? A{INF} : s[h].top();
    }
    void clear(int h = 0) {
      for (A a : s[h]) w[a[3]] = -1;
      s[h].clear();
    }
  };
  int n, qh = 0, cc = 0; ll T = 0; pl e1 = {INF, 0};
  vi mate, sf, bs, lab, hv, gp, bf, fr, q, sz;
  vl pot, tc, lz, sl; Heaps h2, h2s, h4;
  priority_queue<A, vt<A>, greater<A>> h3;
  vt<vt<pl>> g; vt<pi> lk; vt<array<pi, 2>> nd;
  void init(int _n) { n = _n, g.assign(n + 1, {}); }
  void ae(int u, int v, ll w) {
    g[u].pb({v, 2 * w}), g[v].pb({u, 2 * w});
  }
  void rematch(int v, int w) {
    int t = mate[v]; mate[v] = w; if (mate[t] != v) return;
    auto [x, y] = lk[v];
    if (y == sf[y]) mate[t] = x, rematch(x, t);
    else rematch(x, y), rematch(y, x);
  }
  void fix(int b) { // mate and base of b after augmenting
    if (b <= n) return;
    int v = bs[b], d = nd[nd[v][0].f][1].s != mate[nd[v][0].s];
    for (int m; nd[m = nd[v][d].f][!d].s == mate[nd[v][d].s];
        v = nd[m][d].f) fix(v), fix(m);
    fix(bs[b] = v), mate[b] = mate[v];
  }
  ll settle(int b, bool in) { // move lazy potential into b
    ll k = in * (T - tc[b]), d = lz[b] + k;
    return lz[b] = 0, pot[b] -= 2 * k * (b > n), d;
  }
  void upd(int x, int y, int by, ll t, bool inner) {
    if (t >= sl[y]) return;
    sl[y] = t, bf[y] = x;
    if (y != by) {
      int gy = gp[y];
      if (gy != y && t >= sl[gy]) return;
      sl[gy] = t, h2s.put(gy, {t, x, y}, by);
    }
    if (inner) return;
    A m = y == by ? A{t, x, y} : h2s.top(by); m[0] += lz[by];
    if (h2.w[by] < 0 || m[0] < (*h2.it[by])[0]) h2.put(by, m);
  } // 583460
  void swp(int a, int b) { // b is a maximal blossom
    auto w = [&](auto&... v) { (swap(v[a], v[b]), ...); };
    F0R (d, 2) nd[nd[a][d].f][!d].f = b;
    w(lk, nd, mate, sz, pot, lz, tc, bs, hv);
    for (auto v : {&bs, &hv}) if ((*v)[a] == a) (*v)[a] = b;
  }
  void setsf(int b, int s, int g2) {
    sf[b] = s, gp[b] = g2;
    for (int c = bs[b]; sf[c] != s; c = nd[c][0].f)
      setsf(c, s, g2);
  }
  void contract(int x, int y) {
    int bx = sf[x], by = sf[y], h = -1 - cc++, l;
    auto mk = [&](int b) -> int& { return lk[sf[mate[b]]].f; };
    mk(bx) = mk(by) = h;
    do {
      if (mate[by]) swap(bx, by);
      bx = l = sf[lk[bx].f];
    } while (mk(bx) - h && (mk(bx) = h));
    int id = fr.back(), ts = 0; fr.pop_back();
    F0R (d, 2) {
      for (int bv = sf[x]; bv != l;) {
        int mv = mate[bv], bm = sf[mv], v = mate[mv];
        auto [f, t] = lk[v];
        ts += sz[bv] + sz[bm], lk[mv] = {x, y}, h4.er(bm);
        if (bv > n) pot[bv] += 2 * (T - tc[bv]);
        outer(bm, settle(bm, 1));
        nd[bv][d] = {bm, mv}, nd[bm][!d] = {bv, v};
        nd[bm][d] = {bv = sf[f], f}, nd[bv][!d] = {bm, t};
      }
      nd[sf[x]][!d] = {sf[y], y};
      swap(x, y);
    }
    if (l > n) pot[l] += 2 * (T - tc[l]);
    sz[id] = ts + sz[l], bs[id] = l, lk[id] = lk[l];
    mate[id] = mate[l], lab[id] = 1, sf[id] = id;
    tc[id] = T, pot[id] = lz[id] = hv[id] = 0;
    int big = id, bsz = 1, c = l; // heavy child keeps its id
    do if (sz[c] > bsz) bsz = sz[c], big = c;
    while ((c = nd[c][0].f) != l);
    do if (c != big) setsf(c, big, c);
    while ((c = nd[c][0].f) != l);
    if (bsz > 1) sf[id] = hv[id] = big, swp(big, id);
  } // 43448b
  void sched(int b) { h4.put(b, {T + (pot[b] >> 1)}); }
  void link(int v, pi p) {
    lk[v] = p;
    if (v <= n) return;
    int b = bs[v]; link(b, p);
    p = {nd[nd[b][1].f][0].s, nd[b][1].s};
    for (int c = b, w; (w = nd[c][0].f) != b; c = nd[w][0].f) {
      link(w, p);
      link(nd[w][0].f, {nd[w][1].s, nd[c][0].s});
    }
  }
  void outer(int v, ll d) {
    lab[v] = 1;
    if (v > n)
      for (int b = bs[v]; lab[b] != 1; b = nd[b][0].f)
        outer(b, d);
    else {
      pot[v] += T + d, q.pb(v);
      if (pot[v] < e1.f) e1 = {pot[v], v};
    }
  }
  bool grow(int x, int y) {
    int by = sf[y], vis = lab[by], z = mate[by], bz = sf[z];
    if (!vis) link(by, {0, 0});
    lab[by] = -1, tc[by] = T, h2.er(by);
    if (y != by) sched(by);
    if (!z) return rematch(x, y), rematch(y, x), 1;
    if (!vis) link(bz, {x, y}); else lk[bz] = lk[z] = {x, y};
    return outer(bz, settle(bz, 0)), tc[bz] = T, h2.er(bz), 0;
  } // c79a57
  int ms(int b) { // vertex under b with least slack
    if (b <= n) return b;
    int v = 0, c = bs[b], w;
    do if (sl[w = ms(c)] < sl[v]) v = w;
    while ((c = nd[c][0].f) != bs[b]);
    return v;
  }
  void split(int b, int s, int g2) {
    sf[b] = s, gp[b] = g2;
    for (int c = bs[b]; sf[c] != s; c = nd[c][0].f)
      if (c == hv[b]) split(c, s, g2);
      else {
        setsf(c, s, c);
        int to = ms(c);
        if ((sl[c] = sl[to]) < INF)
          h2s.put(c, {sl[c], bf[to], to}, s);
      }
  }
  void expand(int id) {
    int mv = mate[bs[id]], h = hv[id], c = bs[id];
    ll d = settle(id, 1);
    do {
      tc[c] = T, lz[c] = d;
      if (c != h) split(c, c, c), h2s.er(c);
    } while ((c = nd[c][0].f) != bs[id]);
    if (h > 0) swp(h, id), id = h;
    fr.pb(id), bs[id] = id, h2s.clear(id);
    pi ol = lk[mv]; int ob = sf[mate[mv]], rt = sf[ol.s];
    int e = mate[rt] == nd[rt][0].s;
    for (int b = nd[ob][!e].f; b != rt; b = nd[b][!e].f) {
      lab[b] = -2;
      A m = b <= n ? A{sl[b], bf[b], b} : h2s.top(b);
      if (m[0] < INF) m[0] += lz[b], h2.put(b, m);
    }
    for (int b = ob;; b = nd[b][e].f) {
      lab[b] = -1;
      int nb = nd[b][e].f;
      pi p = b == rt ? ol : pi{nd[b][e].s, nd[nb][!e].s};
      lk[mate[b]] = lk[sf[mate[b]]] = p;
      if (b > n) pot[b] ? sched(b) : expand(b);
      if (b == rt) break;
      outer(nb, settle(b = nb, 1));
    }
  } // 54c034
  bool path() {
    while (qh < size(q)) {
      int x = q[qh++], bx = sf[x];
      if (pot[x] == T) return rematch(x, 0), 1;
      for (auto [y, c] : g[x]) {
```

```
      int by = sf[y];
      ll l = lab[by], t = pot[x] + pot[y] - c;
      if (bx == by) continue;
      if (l == 1) {
        if (t >> 1 == T) contract(x, y), bx = sf[x];
        else if (t >> 1 < e1.f) h3.push({t >> 1, x, y});
      } else if (l + 1 && t + lz[by] == T) {
        if (grow(x, y)) return 1;
      } else if (l + 1 || mate[x] != y)
        upd(x, y, by, t, l == -1);
    }
  }
  return 0;
 }
 A pk() { // best live blossom-merging edge, {INF} if none
  while (size(h3) && sf[h3.top()[1]] == sf[h3.top()[2]])
    h3.pop();
  return size(h3) ? h3.top() : A{INF};
 }
 bool dual() { // 1: root matched or can stay unmatched
  T = min({e1.f, h2.top()[0], pk()[0], h4.top()[0]});
  if (T == e1.f) return rematch(e1.s, 0), 1;
  for (A e; (e = h2.top())[0] == T;)
    if (grow(e[1], e[2])) return 1;
  for (A e; (e = pk())[0] == T; h3.pop())
    contract(e[1], e[2]);
  for (A e; (e = h4.top())[0] == T;)
    h4.er(e[3]), expand(e[3]);
  return 0;
 } // 2b197d
 ll solve() {
  int S = n + (n - 1) / 2 + 1;
  mate = lab = hv = bf = vi(S), sz = vi(S, 1);
  sf = vi(S), iota(all(sf), 0), bs = gp = sf;
  fr = vi(sf.rbegin(), sf.rend() - n - 1);
  lk.assign(S, {}), nd.assign(S, {}), sl.assign(S, INF);
  pot = tc = lz = vl(S);
  h2.init(1, S), h2s.init(S, S), h4.init(1, S);
  FOR (u, 1, n + 1) for (auto [v, c] : g[u])
    pot[u] = max(pot[u], c / 2);
  FOR (u, 1, n + 1) if (!mate[u] && pot[u]) {
    link(sf[u], {0, 0}), outer(sf[u], 0);
    while (!path() && !dual());
    FOR (v, 0, n + 1) {
      int b = sf[v];
      if (lab[v] == 1) pot[v] -= T;
      else pot[v] += lz[b] + (lab[b] == -1) * (T - tc[b]);
      lab[v] = lk[v].f = lz[v] = 0, sl[v] = INF;
    }
    for (int b = n + 1, r = S - n - 1 - size(fr);
        r && b < S; b++)
      if (bs[b] != b) {
        if (sf[b] == b) fix(b), pot[b] += 2 * (T - tc[b]) *
          ((lab[b] == 1) - (lab[b] == -1));
        h2s.clear(b), r--;
        lab[b] = lk[b].f = lz[b] = 0, sl[b] = INF;
      }
    q.clear(), qh = cc = 0, T = 0, e1 = {INF, 0};
    h2.clear(), h4.clear(), h3 = {};
  }
  vl m(n + 1);
  FOR (u, 1, n + 1) for (auto [v, w] : g[u])
    if (v == mate[u]) m[u] = max(m[u], w);
  return accumulate(all(m), 0LL) / 4;
 } // e511a4
};
```

7.5 DFS algorithms

SCC2.h

Description: Finds strongly connected components in a directed graph. `comps` lists one representative per SCC in topological order (condensation edges go left to right).
Usage: `SCC sc(n, adj); // sc.comp[v] = representative of v`
Time: $\mathcal{O}(E+V)$ d3e36d, 21 lines

```
using G = vt<vi>; // vt<basic_string<int>> faster
struct SCC {
  int ti = 0; G &adj;
  vi disc, comp, stk, comps;
  SCC(int n, G &adj) : adj(adj), disc(n), comp(n, -1) {
    F0R (i, n) if (!disc[i]) dfs(i);
    reverse(all(comps));
  }
  int dfs(int u) {
    int low = disc[u] = ++ti;
    stk.pb(u);
    for (int v : adj[u]) if (comp[v] == -1)
      low = min(low, disc[v] ?: dfs(v));
    if (low == disc[u]) {
      comps.pb(u);
      for (int y = -1; y != u;)
        comp[y = stk.back()] = u, stk.pop_back();
    }
    return low;
  }
};
```

ArticulationPoints.h

Description: Finds all articulation points of an undirected graph.
Usage: `Arts a(n, adj); // a[u] == 1 iff u is a cut vertex`
Time: $\mathcal{O}(E+V)$ 707c6d, 21 lines

```
struct Arts : vi {
  int t;
  vi tin;
  Arts(int n, vt<vi> &adj) : vi(n), t(0), tin(n) {
    F0R (u, n) if (!tin[u]) dfs(u, adj, 0);
  }

  int dfs(int u, vt<vi> &adj, int cnt = 1) {
    int dp = tin[u] = ++t;
    for (int v : adj[u]) {
      if (tin[v]) dp = min(dp, tin[v]);
      else {
        int up = dfs(v, adj);
        dp = min(dp, up);
        cnt += up >= tin[u];
      }
    }
    at(u) = cnt >= 2;
    return dp;
  }
};
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge in no component is a bridge or a self-loop. Note that degree 0 nodes are not considered components.
Usage: `BCC b; b.init(n, E); // E: vt<pi> lvalue, 0-indexed`
`b.comps[k]: indices into E of block k; b.is_bridge[i]: E[i].`
Time: $\mathcal{O}(E+V)$ 1e39bf, 47 lines

```
struct BCC {
  int t;
  vt<vt<pi>> adj;
  vt<vi> comps; // lists of edges of bcc
  vi tin, stk, is_bridge; // stk for bcc only
  // vi is_art;

  void init(int n, vt<pi> &edges) {
    int m = size(edges);
    adj.resize(n);
    F0R (i, m) {
      auto [u, v] = edges[i];
      adj[u].pb({v, i});
      adj[v].pb({u, i});
    }
    t = 0;
    tin.resize(n);
    // is_art.resize(n);
    is_bridge.resize(m);
    F0R (u, n) if (!tin[u]) dfs(u, -1);
    // if we include bridges as 2-node bcc
    // F0R (i, m) if (is_bridge[i]) comps.pb({i});
  }

  int dfs(int u, int par) {
    int me = tin[u] = ++t, dp = me;
    // int cnt = (par != -1); // art
    for (auto [v, ei] : adj[u]) if (ei != par) {
      if (tin[v]) {
        dp = min(dp, tin[v]);
        if (tin[v] < me) stk.pb(ei); // bcc
      } else {
        int si = size(stk), up = dfs(v, ei);
        dp = min(dp, up);
        // cnt += up >= me; // art
        if (up == me) { // bcc
          stk.pb(ei);
          comps.pb({si + all(stk)});
          stk.resize(si);
        } else if (up < me) stk.pb(ei); // bcc
        if (up > me) is_bridge[ei] = 1; // bridges
      }
    }
    // is_art[u] = cnt >= 2; // art
    return dp;
  }
};
```

BlockCutTree.h

Description: Run BiconnectedComponents with the bridge line in `init` uncommented (bridges become 1-edge blocks) to locate VERTEX components. A block's vertices are the endpoints of its edges. Degree 0 nodes have no block. To build a block cut tree, make a bipartite graph: Put all the normal nodes on the left, and make a new node for each bcc on the right. Draw edges from normal nodes to BCC that contain them. Note that the graph may be disconnected. f38aa7, 6 lines

```
// E = edges, b = BCC after b.init(n, E), bridge line on
vt<vi> T(n + size(b.comps)); // block c is node n + c
vi mk(n, -1);
F0R (c, size(b.comps)) for (int e : b.comps[c])
  for (int x : {E[e].f, E[e].s})
    if (mk[x] != c) mk[x] = c, T[x].pb(n + c), T[n + c].pb(x);
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: `TwoSAT ts{number of boolean variables};`
`ts.either(0, ~3); // var 0 is true or var 3 is false`
`ts.force(2); // var 2 is true`
`ts.at_most_one({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // returns the assignment; empty iff unsatisfiable`
`ts.implies(a, b); ts.tie(a, b); // a => b; a == b`
`ts.exactly_one(a, b); ts.nand(a, b); // a xor b; not both`
`int v = ts.add(); // new variable id`
Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses. "SCC2.h" 7a653d, 49 lines

```
struct TwoSAT {
  int n;
  vt<pi> edges;
  int add() { return n++; }
  void either(int x, int y) { // x | y
    x = max(2 * x, -1 - 2 * x); // ~(2 * x)
    y = max(2 * y, -1 - 2 * y); // ~(2 * y)
    edges.pb({x, y});
  }
  void implies(int x, int y) { either(~x, y); }
  void force(int x) { either(x, x); }
  void exactly_one(int x, int y) {
    either(x, y), either(~x, ~y);
  }
  void tie(int x, int y) {
    implies(x, y), implies(~x, ~y);
  }
  void nand(int x, int y ) { either(~x, ~y); }
  void at_most_one(const vi &li) {
    if (size(li) <= 1) return;
    int cur = ~li[0];
    FOR (i, 2, size(li)) {
      int next = add();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }
  vt<bool> solve() {
    G adj(2 * n);
    for (auto& e : edges) {
      adj[e.f ^ 1].pb(e.s);
      adj[e.s ^ 1].pb(e.f);
    }
    SCC scc(2 * n, adj); // f66e8c
    reverse(all(scc.comps)); // reverse topo order
    for (int i = 0; i < 2 * n; i += 2)
      if (scc.comp[i] == scc.comp[i ^ 1]) return {};
    vi tmp(2 * n);
    for (auto i : scc.comps) {
      if (!tmp[i]) tmp[i] = 1, tmp[scc.comp[i ^ 1]] = -1;
    }
    vt<bool> ans(n);
    F0R (i, n) ans[i] = tmp[scc.comp[2 * i]] == 1;
    return ans;
  }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. For edges, push the edge u came from instead of current node. First, the graph (after removing directivity) must be connected. For undirected graphs, a tour exists when all nodes have even degree. For directed graphs, a tour exists when all nodes have equal in and out degree. For trails, the condition is the same as if you added an edge from t -> s.
Usage: `adj.assign(n, {}); used.assign(m, 0);`
`adj[u].pb({v, id}); // id in [0, m), undirected: add both ends`
`dfs(s); reverse(all(ret)); // walk from s, s = trail start`
Time: $\mathcal{O}(V+E)$ 4b88d2, 14 lines

```
int n, m;
vt<vt<pi>> adj;
vi ret, used;

//
void dfs(int u) {
  while (size(adj[u])) {
    auto [v, ei] = adj[u].back();
    adj[u].pop_back();
    if (used[ei]++) continue;
    dfs(v);
  }
  ret.pb(u);
}
```

DominatorTree.h

Description: Used only a few times. Assuming that all nodes are reachable from *root*, a dominates b iff every path from *root* to b passes through a. `ans[a]` lists the children of a in the dominator tree (immediate dominees).
Usage: `Dominator d; d.init(n); d.ae(a, b); d.build(root);`
Time: $\mathcal{O}(M \log N)$ 0b1894, 46 lines

```
struct Dominator {
  int co;
  vt<vi> adj, ans; // input edges, edges of dominator tree
  vt<vi> radj, child, sdomChild;
  vi label, rlabel, sdom, dom, par, bes;
  void init(int n) {
    co = 0;
    adj = ans = radj = child = sdomChild = vt<vi>(n + 1);
    label = rlabel = sdom = dom = par = bes = vi(n + 1);
  }
  void ae(int a, int b) { adj[a].pb(b); }
  int get(int x) { // DSU with path compression
    // get vertex with smallest sdom on path to root
    if (par[x] != x) {
      int t = get(par[x]); par[x] = par[par[x]];
      if (sdom[t] < sdom[bes[x]]) bes[x] = t;
    }
    return bes[x];
  }
  void dfs(int x) { // create DFS tree
    label[x] = ++co; rlabel[co] = x;
    sdom[co] = par[co] = bes[co] = co;
    for (int y : adj[x]) {
      if (!label[y])
        dfs(y), child[label[x]].pb(label[y]);
      radj[label[y]].pb(label[x]);
    }
  }
  void build(int root) {
    dfs(root);
    ROF (i, 1, co + 1) {
      for (int j : radj[i])
        sdom[i] = min(sdom[i], sdom[get(j)]);
      if (i > 1) sdomChild[sdom[i]].pb(i);
      for (int j : sdomChild[i]) {
        int k = get(j);
        dom[j] = sdom[j] == sdom[k] ? sdom[j] : k;
      }
      for (int j : child[i]) par[j] = i;
    }
    FOR (i, 2, co + 1) {
      if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
      ans[rlabel[dom[i]]].pb(rlabel[i]);
    }
  }
};
```

7.6 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Usage: `vi c = edgeColoring(N, eds); // vertices 0..N-1`
`// c[i] in [0, D] is the color of eds[i]`
Time: $\mathcal{O}(NM)$
Memory: $\mathcal{O}(ND)$ 5f7543, 31 lines

```
vi edgeColoring(int N, vt<pi> eds) {
  vi cc(N + 1), ret(size(eds)), fan(N), free(N), loc;
  for (pi e : eds) ++cc[e.f], ++cc[e.s];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vt<vi> adj(N, vi(ncols, -1));
  for (pi e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  F0R (i, size(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
  return ret;
}
```

Chordal.h

Description: A graph is chordal (every cycle of length ≥ 4 has a chord) iff it has a perfect elimination ordering: each vertex's neighbours later in the order form a clique. `mcs` returns an order that is perfect iff one exists: maximum cardinality search (always take the unvisited vertex with the most visited neighbours) reversed. `isPeo` checks any order: with p the earliest later neighbour of v, the other later neighbours of v must be adjacent to p. If chordal: max clique $= \max_v 1 + |\text{later neighbours}| =$ chromatic number, and greedy colouring from the back is optimal. Simple undirected graphs only.
Usage: `bool chordal = isPeo(g, mcs(g));`
Time: $\mathcal{O}(V+E)$ 2afcb0, 31 lines

```
vi mcs(vt<vi> &g) {
  int n = size(g), hi = 0; vi w(n), ord; vt<vi> b(n + 1);
```

```
  F0R (v, n) b[0].pb(v);
  while (size(ord) < n) {
    while (b[hi].empty()) hi--;
    int v = b[hi].back(); b[hi].pop_back();
    if (w[v] < 0) continue; // stale: visited from higher up
    w[v] = -1, ord.pb(v);
    for (int u : g[v]) if (w[u] >= 0)
      b[++w[u]].pb(u), hi = max(hi, w[u]);
  }
  reverse(all(ord));
  return ord;
} // d4ab34

bool isPeo(vt<vi> &g, const vi &ord) {
  int n = size(g); vi pos(n), mk(n, -1); vt<vi> ch(n);
  F0R (i, n) pos[ord[i]] = i;
  F0R (v, n) {
    int p = -1;
    for (int u : g[v]) if (pos[u] > pos[v] &&
      (p < 0 || pos[u] < pos[p])) p = u;
    if (p >= 0) ch[p].pb(v);
  }
  F0R (p, n) {
    for (int u : g[p]) mk[u] = p;
    for (int v : ch[p]) for (int u : g[v])
      if (pos[u] > pos[v] && u != p && mk[u] != p) return 0;
  }
  return 1;
}
```

7.7 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs — c47f64, 12 lines

```
using B = bitset<128>;
template<class F>
void cliques(vt<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  F0R (i, size(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Usage: `vb g(n); g[a][b] = g[b][a] = 1; // n <= 200`
`vi c = Maxclique(g).maxClique(); // vertex ids`

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs. — 22ed14, 51 lines

```
using vb = vt<bitset<200>>;
struct Maxclique {
  db limit = 0.025, pk = 0;
  struct Vertex { int i, d = 0; };
  using vv = vt<Vertex>;
  vb e;
  vv V;
  vt<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [] (auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    F0R (i, size(r)) r[i].d = min(i, mxD) + 1;
  } // 7805a6
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (size(R)) {
      if (size(q) + R.back().d <= size(qmax)) return;
      q.pb(R.back().i);
      vv T;
      for (auto v:R) if (e[R.back().i][v.i]) T.pb({v.i});
      if (size(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1;
    int mnk = max(size(qmax) - size(q) + 1, 1);
        C[1].clear(), C[2].clear(); // ec774d
        for (auto v : T) {
          int k = 1;
          auto f = [&] (int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].pb(v.i);
        } // 86a60d
        if (j > 0) T[j - 1].d = 0;
        F0R (k, mnk, mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (size(q) > size(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn)
    : e(conn), C(size(e) + 1), S(size(C)), old(S) {
    F0R (i, size(e)) V.pb({i});
  }
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

Usage: `h[i][j] = i != j && !g[i][j]; // complement, i, j < n`

7.8 Trees

7.8.1 Tree facts

For vertex sets A, B with diameter endpoints $(a_1, a_2), (b_1, b_2)$, the diameter of $A \cup B$ has endpoints among those four. The farthest vertex from any vertex is a diameter endpoint. Centre = middle of a diameter (one vertex if its length D is even, else two adjacent), radius $\lceil D/2 \rceil$. Centroid: every component after removal has size $\leq n/2$; one or two (adjacent).

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.

Usage: `t = build_table(par); jump(t, u, k); lca(t, depth, a, b);`
`jump needs k <= depth[u]; depth[v] = distance from root.`

Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$ — 9fc465, 25 lines

```
vt<vi> build_table(vi &par) {
  int d = 1;
  while ((1 << d) < size(par)) d++;
  vt<vt<int>> jmp(d, par);
  FOR (i, 1, d) FOR (j, 0, size(par))
    jmp[i][j] = jmp[i - 1][jmp[i - 1][j]];
  return jmp;
}

int jump(vt<vi>& jmp, int u, int d) {
  F0R (i, size(jmp))
    if (1 & d >> i) u = jmp[i][u];
  return u;
}

int lca(vt<vi> &jmp, vi &depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jump(jmp, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = size(jmp); i--;) {
    int c = jmp[i][a], d = jmp[i][b];
    if (c != d) a = c, b = d;
  }
  return jmp[0][a];
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). adj is an adjacency list of the tree, either directed or undirected.

Usage: `LCA lca(adj); lca(u, v);`

Time: $\mathcal{O}(N \log N + Q)$ — "../data-structures/SparseTable.h" 4beec0, 23 lines

```
struct LCA {
  int t = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vt<vi>& adj) : time(size(adj)) {
    dfs(0, -1, adj);
    if (size(ret)) rmq.init(ret);
  }
  void dfs(int u, int p, vt<vi> &adj) {
    time[u] = t++;
    for (int v : adj[u]) if (v != p) {
      path.pb(u), ret.pb(time[u]);
      dfs(v, u, adj);
    }
  }

  int operator()(int u, int v) {
    if (u == v) return u;
    tie(u, v) = minmax(time[u], time[v]);
    return path[rmq.query(u, v)];
  }
};
```

VirtualTree.h

Description: Computes virtual tree. Needs a global `pos` (dfs entry time) and callable `lca(u, v)` in scope; returns pairs of `(par, child)`. Overwrites `nodes` with the virtual nodes; root is `nodes[0]`.

Time: $\mathcal{O}(|S| \log |S|)$ — "LCA.h" 9b6ff7, 14 lines

```
// pos is dfs time
// pairs of {ancestor, child}
vt<pl> virtual_tree(vt<ll>& nodes) {
  auto cmp = [&] (ll u, ll v) { return pos[u] < pos[v]; };
  sort(all(nodes), cmp);
  int sz = size(nodes);
  F0R (i, sz - 1) nodes.pb(lca(nodes[i], nodes[i + 1]));
```

```
  sort(all(nodes), cmp);
  nodes.erase(unique(all(nodes)), nodes.end());
  vt<pl> res;
  F0R (i, size(nodes) - 1)
  res.pb({lca(nodes[i], nodes[i + 1]), nodes[i + 1]});
  return res;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. `process(u, v, op)` calls `op(l, r)` on $O(\log N)$ half-open ranges of positions covering the u–v path; pair it with any range structure indexed by `pos`; op must be commutative. Subtree of u is `[pos[u] + in_edges, pos[u] + sz[u])`. in_edges true stores edge (par[v], v) at `pos[v]` (the range for a path skips the LCA). Takes the full adjacency list; root must be 0.

Time: $\mathcal{O}\left((\log N)^2\right)$ 781381, 40 lines

```
template<bool in_edges> struct HLD {
  int n, time;
  vt<vi> adj;
  vi par, root, sz, pos;
  HLD(vt<vi> &adj) : n(size(adj)), time(0), adj(adj), par(n, -1), root
       (n), sz(n), pos(n) {
    dfs_sz(0);
    dfs_hld(0);
  } // c667bb
  void dfs_sz(int u) {
    sz[u] = 1;
    for (int& v : adj[u]) {
      par[v] = u;
      adj[v].erase(find(all(adj[v]), u));
      dfs_sz(v);
      sz[u] += sz[v];
      if (sz[v] > sz[adj[u][0]]) swap(v, adj[u][0]);
    }
  } // db31af
  void dfs_hld(int u) {
    pos[u] = time++;
    for (int& v : adj[u]) {
      root[v] = (v == adj[u][0] ? root[u] : v);
      dfs_hld(v);
    }
  }
  template <class Op>
  void process(int u, int v, Op op) {
    for (; ; v = par[root[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (root[u] == root[v]) break;
      op(pos[root[v]], pos[v] + 1);
    }
    op(pos[u] + in_edges, pos[v] + 1); // u is lca
  }
  int lca(int u, int v) {
    for (; root[u] != root[v]; v = par[root[v]])
      if (pos[u] > pos[v]) swap(u, v);
    return pos[u] < pos[v] ? u : v;
  }
};
```

CentroidDecomp.h

Description: Order to process vertices in: dfs from each, stopping at already processed ones.

Time: $\mathcal{O}(N)$ c78ebc, 14 lines

```
vt<pi> ord;
auto dfs1 = [&] (auto &&self, int u, int p) -> int {
  int msk1 = 0, msk2 = 0;
  for (int v : adj[u]) if (v - p) {
    int res = self(self, v, u);
    msk2 |= msk1 & res;
    msk1 |= res;
  }
  int res = (msk1 | ((1 << __lg(2 * msk2 + 1)) - 1)) + 1;
  ord.pb({-__builtin_ctz(res), u});
  return res;
};
dfs1(dfs1, 0, -1);
sort(all(ord));
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Usage: `LinkCut lc(N); lc.link(u, v); lc.cut(u, v); // 0-indexed`
`lc.makeRoot(&lc.node[u]); // makeRoot, access take Node*`

Time: All operations take amortized $\mathcal{O}(\log N)$. 755a2e, 92 lines

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  } // d94cfc
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  } // 966070
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
}; // 225109

struct LinkCut {
  vt<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  } // 035156
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  } // 4cbb0c
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if (u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  } // 7829f0
  Node* access(Node* u) {
    u->splay();
    if (Node* r = u->c[1]) // drop deeper path
      r->p = 0, r->pp = u, u->c[1] = 0, u->fix();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. Edges go from a to b with weight w. Returns (cost, par), par[v] = tail of v's tree edge, par[r] = -1; par is empty if no MST exists.

Time: $\mathcal{O}(E \log V)$ "../data-structures/DSURollback.h" 617c32, 61 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
```

```
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vt<Edge>& g) {
  DSU uf(n);
  vt<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vt<Edge> Q(n), in(n, {-1, -1}), comp;
  deque<pair<int, vt<Edge>>> cycs;
  F0R (s, n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1, {}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi;
        uf.push(); // checkpoint, undone in the restore loop
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.unite(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, {&Q[qi], &Q[end]}});
      }
    }
    F0R (i, qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u, comp] : cycs) { // restore sol (optional)
    uf.pop();
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  F0R (i, n) par[i] = in[i].a;
  return {res, par};
}
```

LongestPath.h

Description: Standalone program (needs the template): reads n and $n-1$ edges (1-indexed), prints for every node its maximum distance to another node.

<bits/stdc++.h> 761cf9, 36 lines

```
using namespace std;

int main() {
  cin.tie(0)->sync_with_stdio(0);

  int n;
  cin >> n;
  vt<vi> adj(n);
  for (int i = n - 1; i--;) {
    int u, v;
    cin >> u >> v;
    adj[--u].pb(--v);
    adj[v].pb(u);
  }
  vi dp1(n, 0), dp2(n);
  auto dfs1 = [&] (auto &&self, int u, int p) -> void {
    for (int v : adj[u]) if (v != p) {
      self(self, v, u);
      dp1[u] = max(dp1[u], dp1[v] + 1);
    }
  };
  auto dfs2 = [&] (auto &&self, int u, int p) -> void {
    multiset<int> dps {dp2[u]};
    for (int v : adj[u]) if (v != p) dps.insert(dp1[v] + 1);
    for (int v : adj[u]) if (v != p) {
      if (*dps.rbegin() == dp1[v] + 1) dp2[v] = *next(dps.rbegin()) +
          1;
      else dp2[v] = *dps.rbegin() + 1;
      self(self, v, u);
    }
  };
  dfs1(dfs1, 0, -1);
  dfs2(dfs2, 0, -1);
  for (int i = 0; i < n; i++) {
    cout << max(dp1[i], dp2[i]) << " \n"[i == n - 1];
  }
}
```

7.9 Other

KthWalk.h

Description: K-th shortest walk from `src` to `des` in digraph. All edge weights must be non-negative.
Usage: `solve()` reads `N M src des K`, then `M` lines `u v w` (0-indexed); prints `K` lengths, `-1` when fewer walks exist
Time: $\mathcal{O}\left((M+N)\log N + K\log K\right)$
Memory: $\mathcal{O}\left((M+N)\log N + K\right)$

"../data-structures/LeftistHeap.h" a76ed9, 63 lines

```
int N, M, src, des, K;

ph cand[MX];
vt<array<int, 3>> adj[MX], radj[MX];
pi pre[MX];
ll dist[MX];

struct state {
  int vert; ph p; ll cost;
  bool operator<(const state& s) const { return cost > s.cost; }
};
priority_queue<state> ans;

void genHeaps() {
  F0R (i, N) dist[i] = INF, pre[i] = {-1, -1};
  priority_queue<T, vt<T>, greater<T>> pq;
  auto ad = [&] (int a, ll b, pi ind) {
    if (dist[a] <= b) return;
    pre[a] = ind; pq.push({dist[a] = b, a});
  };
  ad(des, 0, {-1, -1});
  vi seq; // reachable vertices in order of dist
  while (size(pq)) {
    auto a = pq.top(); pq.pop();
    if (a.f > dist[a.s]) continue;
    seq.pb(a.s); // edge index, vert
    for (auto& t : radj[a.s]) ad(t[0], a.f + t[1], {t[2], a.s});
  }
  for (auto& t : seq) {
    for (auto& u : adj[t])
      if (u[2] != pre[t].f && dist[u[0]] != INF) {
      ll cost = dist[u[0]] + u[1] - dist[t]; assert(cost >= 0);
      cand[t] = ins(cand[t], {cost, u[0]});
    }
    if (pre[t].f != -1) cand[t] = meld(cand[t], cand[pre[t].s]);
    if (t == src) {
      cout << dist[t] << "\n"; K --;
      if (cand[t]) ans.push(state{t, cand[t], dist[t] + cand[t]->v.f})
          ;
    }
  }
}
```

```
void solve() {
  cin >> N >> M >> src >> des >> K;
  F0R (i, M) {
    int u, v, w; cin >> u >> v >> w;
    adj[u].pb({v, w, i}); radj[v].pb({u, w, i}); // vert, weight,
        label
  }
  genHeaps();
  F0R (i, K) {
    if (!size(ans)) {
      cout << -1 << "\n";
      continue;
    }
    auto a = ans.top(); ans.pop();
    int vert = a.vert;
    cout << a.cost << "\n";
    if (a.p->l) ans.push(state{vert, a.p->l, a.cost + a.p->l->v.f - a.
        p->v.f});
    if (a.p->r) ans.push(state{vert, a.p->r, a.cost + a.p->r->v.f - a.
        p->v.f});
    int V = a.p->v.s;
    if (cand[V]) ans.push(state{V, cand[V], a.cost + cand[V]->v.f});
  }
}
```

MatroidIntersection.h

Description: Given two matroids, finds the largest common independent set. For the color and graph matroids, this would be the largest forest where no two edges are the same color. A matroid has 3 functions - check(int x): returns if current matroid can add x without becoming dependent - add(int x): adds an element to the matroid (guaranteed to never make it dependent) - clear(): sets the matroid to the empty matroid The matroid is given an int representing the element, and is expected to convert it (e.g: the color or the endpoints) Pass the matroid with more expensive add/clear operations to M1.
Usage: `MatroidIsect<GraphMat, ColorMat> mi(GraphMat(V, e), ColorMat(C, clr), m); vi a = mi.solve(); // chosen elements // e[i] = {u, v} in [0, V), clr[i] in [0, C), m = #elements`
Time: $R^2N(M2.add+M1.check+M2.check)+R^3M1.add+R^2M1.clear+RNM2.clear$

"../data-structures/UnionFind.h" 74f3e6, 60 lines

```
struct ColorMat {
  vi cnt, clr;
  ColorMat(int n, vi clr) : cnt(n), clr(clr) {}
  bool check(int x) { return !cnt[clr[x]]; }
  void add(int x) { cnt[clr[x]]++; }
  void clear() { fill(all(cnt), 0); }
};
struct GraphMat {
  UF uf;
  vt<array<int, 2>> e;
  GraphMat(int n, vt<array<int, 2>> e) : uf(n), e(e) {}
  bool check(int x) { return !uf.sameSet(e[x][0], e[x][1]); }
  void add(int x) { uf.join(e[x][0], e[x][1]); }
  void clear() { uf = UF(size(uf.e)); }
};
template <class M1, class M2> struct MatroidIsect {
  int n;
  vt<char> iset;
  M1 m1; M2 m2;
  MatroidIsect(M1 m1, M2 m2, int n) : n(n), iset(n + 1), m1(m1), m2(m2
      ) {}
  vi solve() {
    F0R (i, n) if (m1.check(i) && m2.check(i))
      iset[i] = true, m1.add(i), m2.add(i);
    while (augment());
    vi ans;
```

```
    F0R (i, n) if (iset[i]) ans.pb(i);
    return ans;
  }
  bool augment() {
    vi frm(n, -1);
    queue<int> q({n}); // starts at dummy node
    auto fwdE = [&] (int a) {
      vi ans;
      m1.clear();
      F0R (v, n) if (iset[v] && v != a) m1.add(v);
      F0R (b, n) if (!iset[b] && frm[b] == -1 && m1.check(b))
        ans.pb(b), frm[b] = a;
      return ans;
    };
    auto backE = [&] (int b) {
      m2.clear();
      F0R (cas, 2) F0R (v, n)
        if ((v == b || iset[v]) && (frm[v] == -1) == cas) {
          if (!m2.check(v))
            return cas ? q.push(v), frm[v] = b, v : -1;
          m2.add(v);
        }
      return n;
    };
    while (!q.empty()) {
      int a = q.front(), c; q.pop();
      for (int b : fwdE(a))
        while ((c = backE(b)) >= 0) if (c == n) {
          while (b != n) iset[b] ^= 1, b = frm[b];
          return true;
        }
    }
    return false;
  }
};
```

Geometry (8)

8.1 Notes

For checking imprecise equalities use `abs(x) < eps`.
Clamp values to $[-1, 1]$ for `acos`, etc.

8.2 Precision

double can precisely store integers up to 9×10^{15}, and **long double** can store up to 1.8×10^{19}.
Operations $(+, \times, -, /, \sqrt{x})$ will be rounded to the nearest representable value. Hence, the **relative** error on the result of one of these operations is bounded by $\epsilon = 1.2 \times 10^{-16}$ for **double** and $\epsilon = 5.5 \times 10^{-20}$ for **long double**.
If computations are precise up to a relative error of ϵ, and the magnitude of our values never exceeds M, then the absolute error of an operation is at most $M\epsilon$.
It is typically **more useful to consider absolute error** instead of relative error (1).
A series of n + and − operations result in an absolute error of at most $nM\epsilon$.
With $\times$, the absolute error of a d-dimensional value computed in n operations is $M^d((1+\epsilon)^n - 1) \approx nM^d\epsilon$ (excluding multiplication by adimensional values).
Imprecisions on x in $\frac{1}{x}$ can cause an arbitrary error (2).
Imprecisions on x in $\sqrt{x}$ are typically bad, particularly near 0. (3)
Note that $\frac{1}{x}$ and $\sqrt{x}$ perform poorly on imprecise inputs: when working with exact inputs, the `IEEE 754` standard guarantees a relative error of ϵ/absolute error of $M\epsilon$.
Circle intersections take $\sqrt{r^2 - d^2}$, so near tangency the error is about $M\sqrt{\epsilon}$ (use an eps).

(1) subtracting two large imprecise values can blow up relative errors
(2) division by a value near 0 can result in an arbitrarily large error in both positive and negative
(3) assuming arguments to $\sqrt{x}$ are non-negative, imprecisions on x will have the most impact when x is near 0: for a given imprecision δ, the biggest imprecision on $\sqrt{x}$ it might cause is $\sqrt{\delta}$. This is quite bad: an argument with imprecision $nM^2\epsilon$ will become an imprecision of $\sqrt{n}M\sqrt{\epsilon}$. This can appear with circle intersections and computing roots to quadratics.

StableSum.h

Description: Accumulates non-negative floating point numbers with better precision than a running sum: a binary counter of partial sums, so only numbers of similar magnitude are ever added together.
Usage: `StableSum t; t += x; db r = t.val();`
Time: $\mathcal{O}(\log N)$ amortised per `+=` cf4661, 16 lines

```
struct StableSum {
  int cnt = 0;
  vt<db> v, pref{0};
  void operator+=(db a) {
    assert(a >= 0);
    int s = ++cnt;
    while (s % 2 == 0) {
      a += v.back();
      v.pop_back(), pref.pop_back();
      s /= 2;
    }
    v.pb(a);
    pref.pb(pref.back() + a);
  }
  db val() { return pref.back(); }
};
```

8.3 Common

8.3.1 Equation of line through two points

Given points p and q, equation of the line $ax + by + c = 0$ is given by:

$$a = p_y - q_y$$
$$b = q_x - p_x$$
$$c = p \times q$$

8.3.2 Center of mass of a polygon

For a simple polygon $p_0, \ldots, p_{n-1}$ (either orientation, $p_n = p_0$), with signed area $A = \frac{1}{2}\sum_i p_i \times p_{i+1}$:

$$C = \frac{1}{6A}\sum_{i=0}^{n-1}(p_i + p_{i+1})(p_i \times p_{i+1})$$

(`PolygonCenter.h`). This is the centroid of the filled region. For the boundary (a uniform wire), weight each edge midpoint by its length; for point masses at the vertices, take the average of the vertices. These three differ in general.

8.4 Geometric primitives

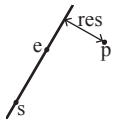
Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.) unit, normal, rotate need T = double. This struct does not implement a custom comma operator. 490423, 39 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  using P = Point<T>;
  T x = 0, y = 0;
  #define op1(o) P operator o (P p) const { return {x o p.x, y o p.y};
       }
  op1(+) op1(-)
  #define op2(o) P operator o (T z) const { return {x o z, y o z}; }
  op2(*) op2(/)
  T dot(P p) const { return x * p.x + y * p.y; }
  T cross(P p) const { return x * p.y - y * p.x; }
  #define op3(o) T o (P a, P b) const { return (a - *this). o (b - *
       this); }
  op3(dot) op3(cross)
  #define op4(o) bool operator o (P p) const { return tie(x, y) o tie(
       p.x, p.y); }
  op4(<) op4(==) op4(!=)
  int half() const { return y < 0 || (y == 0 && x < 0); }
  // radial
  // bool operator<(P p) const {
  //   return make_pair(half(), (T) 0) < make_pair(p.half(), cross(p))
       ;
  // }
  T dist2() const { return x * x + y * y; }
  db dist() const { return sqrtl((db) dist2()); }
  // angle to x-axis in interval [-pi, pi]
  db angle() const { return atan2l(y, x); }
  P unit() const { return *this / dist(); }
  P perp() const { return {-y, x}; } // rotate 90 degrees left
  P normal() const { return perp().unit(); }
  // ccw by a radians about origin
  P rotate(db a) const {
    return P{x * cos(a) - y * sin(a), x * sin(a) + y * cos(a)};
  }
  // angle at *this between a and b, in [-pi, pi]
  db angle(P a, P b) const {
    return atan2l((db) cross(a, b), (db) dot(a, b));
  }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << ", " << p.y << ") ";
  }
};
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

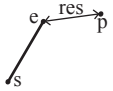


"Point.h" 9d3383, 4 lines

```
template<class P>
db line_dist(const P& a, const P& b, const P& p) {
  return (db) (b - a).cross(p - a) / (b - a).dist();
}
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.
Usage: Point<db> a{0,0}, b{2,2}, p{1,1};
bool on_segment = seg_dist(a,b,p) < 1e-10;

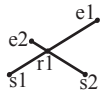


"Point.h" cc56c9, 6 lines

```
template<class P>
db seg_dist(const P &s, const P &e, const P &p) {
  if (s == e) return (p - s).dist();
  auto d = (e - s).dist2(), t = min(d, max(.0, (p - s).dot(e - s)));
  return ((p - s) * d - (e - s) * t).dist() / d;
}
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Intermediate products of three coordinates need |coord| <= 500 for int, 5e5 for ll.
Usage: vt<P> inter = seg_inter(s1,e1,s2,e2);
if (size(inter) == 1)
cout << "segments intersect at " << inter[0] << endl;

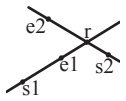


"Point.h", "OnSegment.h" bd6e14, 13 lines

```
template<class P> vt<P> seg_inter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
     oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (on_segment(c, d, a)) s.insert(a);
  if (on_segment(c, d, b)) s.insert(b);
  if (on_segment(a, b, c)) s.insert(c);
  if (on_segment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 (s1!=e1, s2!=e2) exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Intermediate products of three coordinates need |coord| <= 500 for int, 5e5 for ll.
Usage: auto res = line_inter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;



"Point.h" 5a2c8e, 8 lines

```
template<class P>
pair<int, P> line_inter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P{}};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: bool left = side_of(p1,p2,q)==1;

"Point.h" dcdd4f, 9 lines

```
template<class P>
int side_of(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int side_of(const P& s, const P& e, const P& p, db eps) {
  auto a = (e - s).cross(p - s);
  db l = (e - s).dist() * eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

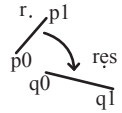
Description: Returns true iff p lies on the line segment from s to e, endpoints included. Use (seg_dist(s,e,p) < epsilon) instead when using Point<double>.

"Point.h" 0c1f75, 3 lines

```
template<class P> bool on_segment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes p0, p1 to q0, q1 to point r. Needs p0 != p1.



"Point.h" 54c481, 6 lines

```
using P = Point<db>;
P linear_transformation(const P &p0, const P &p1,
    const P &q0, const P &q1, const P &r) {
  P dp = p1 - p0, dq = q1 - q0, num{dp.cross(dq), dp.dot(dq)};
  return q0 + P{(r - p0).cross(num), (r - p0).dot(num)} / dp.dist2();
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" da84d5, 5 lines

```
template<class P>
P proj(P a, P b, P p, bool refl = false) {
  P v = b - a;
  return p - v.perp() * (1 + refl) * v.cross(p - a) / v.dist2();
}
```

8.5 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect, written to out. Returns false in case of no intersection. The circles must not be identical.

"Point.h" b70280, 11 lines

```
using P = Point<db>;
bool circle_inter(P a, P b, db r1, db r2, pair<P, P> *out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  db d2 = vec.dist2(), sum = r1 + r2, dif = r1 - r2,
       p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2), h2 = r1 * r1 - p * p *
           d2;
  if (sum * sum < d2 || dif * dif > d2) return false;
  P mid = a + vec * p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. P is intended to be Point<double>.

"Point.h" 893a72, 13 lines

```
template<class P>
vt<pair<P, P>> tangents(P c1, db r1, P c2, db r2) {
  P d = c2 - c1;
  db dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0) return {};
  vt<pair<P, P>> out;
  for (db sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.pb({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. The line passes through a != b; two points are ordered from a to b. P is intended to be Point<double>.

"Point.h" 8d20c7, 9 lines

```
template<class P>
vt<P> circle_line(P c, db r, P a, P b) {
  P ab = b - a, p = a + ab * (c - a).dot(ab) / ab.dist2();
  db s = a.cross(b, c), h2 = r * r - s * s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
```

```
  return {p - h, p + h};
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a polygon (signed area, positive if ccw).

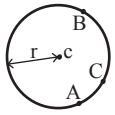
Time: $\mathcal{O}(n)$ "Point.h" 0dac94, 19 lines

```
using P = Point<db>;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
db circlePoly(P c, db r, vt<P> ps) {
  auto tri = [&] (P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p) / d.dist2(), b = (p.dist2() - r * r) / d.dist2()
        ;
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a - sqrt(det)), t = min(1., -a + sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t - 1);
    return arg(p, u) * r2 + u.cross(v) / 2 + arg(v, q) * r2;
  };
  auto sum = 0.0;
  F0R (i, size(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % size(ps)] - c);
  return sum;
}
```

circumcircle.h

Description:

The circumcircle of a triangle is the circle intersecting all three vertices. `cc_radius` returns the radius of the circle going through points A, B and C and `cc_center` returns the center of the same circle. A, B, C must not be collinear.



"Point.h" 6a4356, 9 lines

```
using P = Point<db>;
db cc_radius(const P& A, const P& B, const P& C) {
  return (B - A).dist() * (C - B).dist() * (A - C).dist() /
      abs((B - A).cross(C - A)) / 2;
}
P cc_center(const P& A, const P& B, const P& C) {
  P b = C - A, c = B - A;
  return A + (b * c.dist2() - c * b.dist2()).perp() / b.cross(c) / 2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points. ps must be non-empty. Returns the pair (center, radius).

Time: expected $\mathcal{O}(n)$ "circumcircle.h" 5ed9d9, 17 lines

```
pair<P, db> mec(vt<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  db r = 0, EPS = 1 + 1e-8;
  F0R (i, size(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    F0R (j, i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      F0R (k, j) if ((o - ps[k]).dist() > r * EPS) {
        o = cc_center(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

8.6 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. Exact for ll with $|coord| \leq 10^9$.

Usage: `vt<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = in_polygon(v, P{3, 3}, false);`

Time: $\mathcal{O}(n)$ "Point.h", "OnSegment.h" 42c990, 11 lines

```
template<class P>
bool in_polygon(vt<P> &p, P a, bool strict = true) {
  int cnt = 0, n = size(p);
  F0R (i, n) {
    P q = p[(i + 1) % n];
    if (on_segment(p[i], q, a)) return !strict;
    //or: if (seg_dist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y < p[i].y) - (a.y < q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" 3d0f63, 6 lines

```
template<class T>
T polygon_area(const vt<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  F0R (i, size(v) - 1) a += v[i].cross(v[i + 1]);
  return a;
}
```

PolygonCenter.h

Description: Area centroid of a polygon (needs nonzero area): $C = \frac{1}{6A}\sum(p_i + p_{i+1})(p_i \times p_{i+1})$, A the signed area.

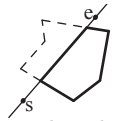
Time: $\mathcal{O}(n)$ "Point.h" 04e52b, 9 lines

```
using P = Point<db>;
P polygon_center(const vt<P>& v) {
  P res{}; db a = 0;
  for (int i = 0, j = size(v) - 1; i < size(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    a += v[j].cross(v[i]);
  }
  return res / a / 3;
}
```

PolygonCut.h

Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.



Usage: `vt<P> p = ...;`
`p = polygonCut(p, P{0,0}, P{1,0});` "Point.h" 0686d7, 13 lines

```
using P = Point<db>;
vt<P> polygonCut(const vt<P>& poly, P s, P e) {
  vt<P> res;
  F0R (i, size(poly)) {
    P cur = poly[i], prev = i ? poly[i - 1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.pb(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.pb(cur);
  }
  return res;
}
```

HalfPlane.h

Description: Intersection of half-planes. `Hp(a, b)` is the set of points strictly left of the line $a \to b$. Returns the polygon in CCW order, empty if the area is 0. Add 4 half-planes of a big box first: the result must be bounded.

Usage: `vt<Hp> h = {Hp(P{-B, -B}, P{B, -B}), ...};`
`vt<P> r = hpi(h);`

Time: $\mathcal{O}(N \log N)$ "Point.h" a8ac22, 29 lines

```
using P = Point<db>;
struct Hp {
  P p, d; db a; // d = direction, a = its angle
  Hp(P s, P e) : p(s), d(e - s), a(atan2(d.y, d.x)) {}
  bool out(P q) const { return d.cross(q - p) < -1e-9; }
  P inter(const Hp& o) const { // not parallel
    return p + d * (o.d.cross(o.p - p) / o.d.cross(d));
  }
};
vt<P> hpi(vt<Hp> h) {
  sort(all(h), [](Hp& a, Hp& b) { return a.a < b.a; });
  int n = size(h), l = 0, r = 0;
  vt<Hp> q(n, h[0]); vt<P> t(n);
  FOR (i, 1, n) {
    while (l < r && h[i].out(t[r - 1])) r--;
    while (l < r && h[i].out(t[l])) l++;
    q[++r] = h[i];
    if (fabs(q[r].d.cross(q[r - 1].d)) < 1e-12) { // parallel
      if (q[r].d.dot(q[r - 1].d) < 0) return {}; // opposite
      r--; if (h[i].out(q[r].p)) q[r] = h[i];
    }
    if (l < r) t[r - 1] = q[r - 1].inter(q[r]);
  }
  while (r - l > 1 && q[l].out(t[r - 1])) r--;
  while (r - l > 1 && q[r].out(t[l])) l++;
  if (r - l <= 1) return {};
  t[r] = q[r].inter(q[l]);
  return {t.begin() + l, t.begin() + r + 1};
}
```

PolygonTriangulation.h

Description: Triangulates a simple CCW polygon with distinct vertices (collinear vertices and horizontal edges are fine), $|coord| \leq 10^9$. Returns $n-2$ CCW index triples. A sweep by (x, y) adds diagonals at split and merge vertices, giving monotone faces whose convex ears are clipped in sweep order.

Usage: `vt<P> poly = {{0,0}, {2,0}, {1,1}, {2,2}, {0,2}};`
`vt<array<int, 3>> tri = triangulate(poly);`

Time: $\mathcal{O}(n \log n)$ "Point.h" 379f9c, 46 lines

```
using P = Point<ll>;
vt<array<int, 3>> triangulate(vt<P> p) {
  int n = size(p), m = n;
  p.pb(p[0]);
  vt<vi> E(2, vi(3 * n));
  vi id(3 * n), sm(3 * n, -1), hl(n), mg(n), o(n);
  iota(all(id), 0), iota(all(o), 0);
  auto ln = [&](int x, int y) { E[0][x] = y, E[1][y] = x; };
  F0R(i, n) ln(i, (i + 1) % n);
  auto d = [&](int a, int b) {
    int A = m, B = m + 1;
    for (int x : {a, b})
      x = id[x], id[m] = x, sm[m] = sm[x], sm[x] = m++;
    ln(E[1][a], A), ln(A, B), ln(B, E[0][b]), ln(b, a);
    return A;
  };
  auto cmp = [&](int x, int y) {
    if (p[x] < p[y]) return p[x].cross(p[x + 1], p[y]) > 0;
    return p[y].cross(p[y + 1], p[x]) < 0;
  };
  set<int, decltype(cmp)> T(cmp);
  sort(all(o), [&](int a, int b) { return p[a] < p[b]; });
```

```
  for (int v : o) {
    int u = (v + n - 1) % n, w = (v + 1) % n;
    bool L = p[u] < p[v], R = p[w] < p[v];
    if (L) T.erase(u), mg[id[hl[u]]] && d(v, hl[u]);
    auto it = T.insert(v).first;
    hl[v] = v;
    if (L == R ? p[v].cross(p[u], p[w]) > 0 : R) {
      int e = *--it;
      hl[e] = mg[id[hl[e]]] || !(L || R) ? d(v, hl[e]) : v;
      mg[v] = L && R;
    }
    if (R) T.erase(v);
  }
  vt<array<int, 3>> r;
  vi dn(m);
  for (int v : o) for (int u = v; u >= 0; u = sm[u]) {
    F0R(k, 2) for (int w, z; w = E[k][u], z = E[k][w],
      dn[w] && dn[z] && sgn(p[id[u]].cross(p[id[w]], p[id[z]]))
      == 1 - 2 * k; E[k][u] = z, E[!k][z] = u)
      r.pb({id[u], id[k ? z : w], id[k ? w : z]});
    dn[u] = 1;
  }
  return r;
}
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. Collinearity uses `eps`.
Time: $\mathcal{O}(N^2)$, where N is the total number of points

"Point.h", "sideOf.h" 198992, 34 lines

```
using P = Point<db>;
const db eps = 1e-9;
db rat(P a, P b) { return sgn(b.x) ? a.x / b.x : a.y / b.y; }
db poly_union(vt<vt<P>>& poly) {
  db ret = 0;
  F0R (i, size(poly)) F0R (v, size(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % size(poly[i])];
    vt<pair<db, int>> segs = {{0, 0}, {1, 0}};
    F0R (j, size(poly)) if (i != j) {
      F0R (u, size(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % size(poly[j])];
        int sc = side_of(A, B, C, eps), sd = side_of(A, B, D, eps);
        if (sc != sd) {
          db sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j < i && sgn((B - A).dot(D - C)) > 0)
            {
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      }
    }
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, (db) 0), (db) 1);
    db sum = 0;
    int cnt = segs[0].second;
    FOR (j, 1, size(segs)) {
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```

ConvexHull.h



Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.
Time: $\mathcal{O}(n \log n)$

"Point.h" 5fdddf, 12 lines

```
vt<P> convex_hull(vt<P> pts) {
  if (size(pts) <= 1) return pts;
  sort(all(pts)); // lex sort
  vt<P> h(size(pts) + 1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t - 2].cross(h[t - 1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

IncrementalHull.h

Description: Convex hull under point insertions in arbitrary order. Returns minimal hull vertices CCW from the lexicographically smallest point, without repeating the first point. Handles empty input, duplicates and collinearity; an entirely collinear hull has at most two endpoints. T must be signed; ll needs $|coord| \leq 10^9$. Requires reliable signs.
Usage: `IncrementalHull<ll> h; h.add({x, y}); auto ps = h.hull();`
Time: Amortized $\mathcal{O}(\log n)$ per insertion; $\mathcal{O}(h)$ to report h hull vertices.
Memory: $\mathcal{O}(h)$

"Point.h" 389aed, 23 lines

```
template<class T> struct IncrementalHull {
  using P = Point<T>;
  set<P> lo, hi;
  void insert(set<P>& chain, P p, bool upper) {
    auto [it, added] = chain.insert(p);
    if (!added) return;
    auto bad = [&](auto v) {
      if (v == chain.begin() || next(v) == chain.end()) return false;
      auto turn = prev(v)->cross(*v, *next(v));
      return upper ? turn >= 0 : turn <= 0;
    };
    if (bad(it)) { chain.erase(it); return; }
    while (it != chain.begin() && bad(prev(it))) chain.erase(prev(it))
        ;
    while (next(it) != chain.end() && bad(next(it))) chain.erase(next(
        it));
  }
  void add(P p) { insert(lo, p, false); insert(hi, p, true); }
  vt<P> hull() const {
    if (size(lo) <= 1) return {all(lo)};
    vt<P> ans(lo.begin(), prev(lo.end()));
    ans.insert(ans.end(), hi.rbegin(), prev(hi.rend()));
    return ans;
  }
};
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Usage: `auto [p, q] = hull_diameter(convex_hull(pts));`
Time: $\mathcal{O}(n)$

"Point.h" f842ab, 12 lines

```
using P = Point<ll>;
array<P, 2> hull_diameter(vt<P> s) {
  int n = size(s), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {s[0], s[0]}});
  F0R (i, j)
    for (; ; j = (j + 1) % n) {
      res = max(res, {(s[i] - s[j]).dist2(), {s[i], s[j]}});
      if ((s[(j + 1) % n] - s[j]).cross(s[i + 1] - s[i]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point p lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. strict (default true) excludes points on the boundary.
Usage: `in_hull(hull, p, false); // boundary counts`
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 1206d5, 13 lines

```
using P = Point<ll>;
bool in_hull(const vt<P>& l, P p, bool strict = true) {
  int a = 1, b = size(l) - 1, r = !strict;
  if (size(l) < 3) return r && on_segment(l[0], l.back(), p);
  if (side_of(l[0], l[a], l[b]) > 0) swap(a, b);
  if (side_of(l[0], l[a], p) >= r || side_of(l[0], l[b], p) <= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (side_of(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. Exact for ll with $|coord| \leq 10^9$. lineHull(a, b, poly) returns a pair of vertex indices describing the intersection of line ab with the polygon: • $(-1, -1)$ if no collision, • $(i, -1)$ if touching the corner i, • (i, i) if along side $(i, i+1)$, • (i, j) if crossing sides $(i, i+1)$ and $(j, j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h" 4c43fc, 41 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vt<P>& poly, P dir) {
  int n = size(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}
#undef cmp
#undef extr

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vt<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  F0R (i, 2) {
    int lo = endB, hi = endA, n = size(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
```

```
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + size(poly) + 1) % size(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```

HullTangents.h

Description: Tangents from a point p strictly outside a ccw convex polygon with no collinear points, $|coord| \le 10^9$. Returns $\{a, b\}$: the polygon lies to the right of the ray $p \to a$ and to the left of $p \to b$, so the ccw walk from a to b is the side of the hull facing p. A tangent is $(i, -1)$ if it touches only vertex i, or $(i, i+1)$ (indices in hull order) if it contains side $(i, i+1)$. For doubles use the eps side_of (sideOf.h) as in the comment; every vertex not on a tangent must then be further than eps from it. Visible chain: ccw from `a.s` (`a.f` if `a.s` $= -1$) to `b.f`.

Time: $\mathcal{O}(\log n)$ "Point.h" dde319, 20 lines

```
// doubles: d * side_of(p, poly[(i)%n], poly[(j)%n], eps)
#define ang(i,j) sgn(d * p.cross(poly[(i)%n], poly[(j)%n]))
#define tang(i) (ang(i + 1, i) >= 0 && ang(i, i - 1 + n) < 0)
template<class P>
pair<pi, pi> hull_tangents(vt<P>& poly, P p) {
  int n = size(poly);
  auto go = [&](int d) -> pi {
    int lo = 0, hi = n;
    if (!tang(0)) while (lo + 1 < hi) {
      int m = (lo + hi) / 2;
      if (tang(m)) { lo = m; break; }
      int a = ang(lo + 1, lo), b = ang(m + 1, m);
      (a < b || (a == b && a == ang(lo, m)) ? hi : lo) = m;
    }
    return {lo, ang(lo, lo + 1) || n < 2 ? -1 : (lo + 1) % n};
  };
  return {go(1), go(-1)};
}
#undef ang
#undef tang
```

MinkowskiSum.h

Description: Minkowski sum of two convex polygons given in CCW order with no collinear points (e.g. convex_hull output). Uses Point's default (lexicographic) operator< for min_element.

Usage: `in_hull(minkowski_sum(a, nb), P{0, 0}, 0)`
`// nb = -b; a and b meet iff true`

Time: $\mathcal{O}(N)$ "Point.h", "ConvexHull.h" 729800, 31 lines

```
using P = Point<ll>; using vP = vt<P>;

vP minkowski_sum(vP a, vP b) {
  if (size(a) > size(b)) swap(a, b);
  if (!size(a)) return {};
  if (size(a) == 1) {
    for (auto &t : b) t = t + a[0];
    return b;
  }
  rotate(begin(a), min_element(all(a)), end(a));
  rotate(begin(b), min_element(all(b)), end(b));
  a.pb(a[0]), a.pb(a[1]);
  b.pb(b[0]), b.pb(b[1]);
  vP result;
  int i = 0, j = 0;
  while (i < size(a) - 2 || j < size(b) - 2) {
    result.pb(a[i] + b[j]);
    ll crs = (a[i + 1] - a[i]).cross(b[j + 1] - b[j]);
    i += (crs >= 0);
    j += (crs <= 0);
  }
  return result;
}

ll diameter2(vP p) { // example application: squared diameter
  vP a = convex_hull(p);
  vP b = a; for (auto &t : b) t = t * -1;
  vP c = minkowski_sum(a, b);
  ll ret = 0; for (auto &t : c) ret = max(ret, t.dist2());
  return ret;
}
```

8.7 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$ "Point.h" b1dbd1, 17 lines

```
using P = Point<ll>;
pair<P, P> closest(vt<P> v) {
  assert(size(v) > 1);
  set<P> S;
  sort(all(v), [] (P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll) sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = |p.x - q.x| + |p.y - q.y|. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST. Coordinates must stay below ~5e8 in magnitude (int arithmetic: differences and d.x + d.y must fit in int).

Time: $\mathcal{O}(N \log N)$ "Point.h" 0d410c, 23 lines

```
using P = Point<int>;
vt<array<int, 3>> manhattanMST(vt<P> ps) {
  vi id(size(ps));
  iota(all(id), 0);
  vt<array<int, 3>> edges;
  F0R (k, 4) {
    sort(all(id), [&] (int i, int j) {
      return (ps[i] - ps[j]).x < (ps[j] - ps[i]).y; });
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
           it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.pb({d.y + d.x, i, j});
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
  }
  return edges;
}
```

kdTree.h

Description: KD-tree (2d), squared distances. Coordinates in $[-10^9, 10^9]$.

Usage: `vt<P> v; // P = array<int, 2>`
`Node *t = new Node(all(v), 0); // reorders v`
`ll b = LLONG_MAX; t->search(q, b); // b = nearest dist2`

"Point.h" 7571f1, 57 lines

```
using P = array<int, 2>;
struct Node {
  #define _sq(x) (x) * (x)
  P lo, hi;
  struct Node *lc, *rc;

  ll dist2(const P &a, const P &b) const {
    return 1ll * _sq(a[0] - b[0]) + 1ll * _sq(a[1] - b[1]);
  }

  ll dist2(P &p) {
    #define _loc(i) (p[i] < lo[i] ? lo[i] : (p[i] > hi[i] ? hi[i] : p[
        i]))
    return dist2(p, {_loc(0), _loc(1)});
  }

  template<class ptr>
  Node (ptr l, ptr r, int d) : lc(0), rc(0) {
    assert(l != r);
    lo = {inf, inf}, hi = {-inf, -inf};
    for (ptr p = l; p < r; p++) {
      F0R (i, 2) lo[i] = min(lo[i], (*p)[i]), hi[i] = max(hi[i], (*p)[
          i]);
    }
    if (r - l == 1) return;
    ptr m = l + (r - l) / 2;
    nth_element(l, m, r, [&] (auto a, auto b) { return a[d] < b[d]; })
        ;
    lc = new Node(l, m, d ^ 1);
    rc = new Node(m, r, d ^ 1);
  }

  // nearest neighbour: init best = LLONG_MAX; 0 if p in set
  void search(P p, ll &best) {
    if (lc) { // rc will also exist
      ll dl = lc->dist2(p), dr = rc->dist2(p);
      if (dl > dr) swap(lc, rc), dr = dl;
      lc->search(p, best);
      if (dr < best) rc->search(p, best);
    } else best = min(best, dist2(p, lo));
  }

  // push k LLONG_MAX; top() = kth nearest
  void search(P p, priority_queue<ll> &pq) {
    if (lc) {
      ll dl = lc->dist2(p), dr = rc->dist2(p);
      if (dl > dr) swap(lc, rc), dr = dl;
      lc->search(p, pq);
      if (dr < pq.top()) rc->search(p, pq);
    } else pq.push(dist2(p, lo)), pq.pop();
  }
};
// Degenerate input (many equal x or y): rotate all points and
// queries by p -> {3x - 4y, 4x + 3y}. M^T M = 25 I, so dist
// scales by 5 and dist2 by exactly 25 (divide b by 25). Needs
// |x|, |y| <= 1.4e8: rotated coords <= 7 max|x| must fit inf.
// ~8x fewer nodes visited on equal-x input, same on random.
// Other 2D: {a, -b; b, a}, a^2 + b^2 = s^2, scale s:
// (5, 12, 13), (20, 21, 29).
// 3D, scale 7: rows (3, -2, 6), (6, 3, -2), (-2, 6, 3)
```

DelaunayTriangulation.h

Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.
Usage: delaunay(ps, [&](int a, int b, int c) {}); // ccw
Time: $\mathcal{O}(n^2)$ "Point.h", "3dHull.h" 22fa69, 10 lines

```
template<class P, class F>
void delaunay(vt<P>& ps, F trifun) {
  if (size(ps) == 3) { int d = (ps[0].cross(ps[1], ps[2]) < 0);
    trifun(0, 1 + d, 2 - d); }
  vt<P3> p3;
  for (P p : ps) p3.emplace_back(p.x, p.y, p.dist2());
  if (size(ps) > 3) for (auto t:hull3d(p3)) if ((p3[t.b] - p3[t.a]).
      cross(p3[t.c] - p3[t.a]).dot(P3(0, 0, 1)) < 0)
    trifun(t.a, t.c, t.b);
}
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns corner points (3 per triangle) in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise. $|coord| \leq 10^9$.
Time: $\mathcal{O}(n \log n)$ "Point.h" d33ecb, 89 lines

```
using P = Point<ll>;
using Q = struct Quad*;
using lll = __int128_t; // (can be ll if coords are < 2e4)
P arb{LLONG_MAX, LLONG_MAX}; // not equal to any other point

struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2() - p2,
    B = b.dist2() - p2, C = c.dist2() - p2;
  return p.cross(a, b) * C + p.cross(b, c) * A
    + p.cross(c, a) * B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  FOR (i, 4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}

pair<Q, Q> rec(const vt<P>& s) {
  if (size(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (size(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = size(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({size(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (; ;) {
    DEL(LC, base->r(), o); DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}

vt<P> triangulate(vt<P> pts) {
  sort(all(pts)); assert(unique(all(pts)) == pts.end());
  if (size(pts) < 2) return {};
  Q e = rec(pts).first;
  vt<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.pb(c->p); \
  q.pb(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < size(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```

AllPointPairs.h

Description: Rotating sweep: visits the points sorted by projection onto every direction, one angle at a time. Needs Point<ll> with the radial operator< in place of the lexicographic one, and distinct points. Two variants: first assumes no three points colinear. Example use: min/max of twice the triangle area over all triples.
Time: $\mathcal{O}(n^2 \log n)$
Memory: $\mathcal{O}(n^2)$ 76ce87, 61 lines

```
int n; cin >> n;
vt<P> pts(n); // use radial sort
for (auto &[x, y] : pts) cin >> x >> y;

vi ord(n), loc(n);
FOR (i, n) ord[i] = i;
sort(all(ord), [&] (int i, int j) { // sorted for angle -eps
  return pts[i].x != pts[j].x ? pts[i].x < pts[j].x
                              : pts[i].y > pts[j].y; });
FOR (i, n) loc[ord[i]] = i;

// event (x, y) fires when the sweep reaches perp(y - x):
// before it x precedes y in ord, after it y precedes x
vt<tuple<P, int, int>> evs;
FOR (x, n) FOR (y, n) if (x != y)
  evs.pb({(pts[y] - pts[x]).perp(), x, y});
sort(all(evs));
ll mn = INF, mx = -INF;

for (auto [dir, x, y] : evs) { // no three collinear
  int &i = loc[x], &j = loc[y];
  if (i < j) swap(ord[i], ord[j]), swap(i, j);
  // ord[j - 1], ord[i + 1]: nearest points on either side of
  // line xy; ord[0], ord[n - 1]: the farthest
  auto area = [&] (int k) {
    return abs(pts[x].cross(pts[y], pts[ord[k]])); };
  if (i + 1 < n) mn = min(mn, area(i + 1));
  if (j > 0) mn = min(mn, area(j - 1));
  mx = max({mx, area(0), area(n - 1)});
}

#if 0 // collinear-safe variant: use INSTEAD of the loop above
for (int l = 0, r = 0; l < size(evs); l = r) {
  P d0 = get<0>(evs[l]);
  vi pos; // positions touched by this direction
  while (r < size(evs)) {
    auto [d1, x, y] = evs[r];
    if (d0.cross(d1) || d0.dot(d1) < 0) break;
    pos.pb(loc[x]), pos.pb(loc[y]), r++;
  }
  sort(all(pos)), pos.erase(unique(all(pos)), end(pos));
  for (int s = 0, t; s < size(pos); s = t) { // one line each
    auto pr = [&] (int k) {
      return pts[ord[pos[k]]].dot(d0); };
    for (t = s; t + 1 < size(pos) && pos[t + 1] == pos[t] + 1
      && pr(t + 1) == pr(t); t++);
    int lo = pos[s], hi = pos[t++] + 1;
    reverse(ord.begin() + lo, ord.begin() + hi);
    FOR (i, lo, hi) loc[ord[i]] = i;
    // ord[lo - 1], ord[hi]: nearest points on either side
    // of the line; ord[0], ord[n - 1]: the farthest
    if (hi - lo > 2) mn = 0; // three collinear points
    P a = pts[ord[lo]], b = pts[ord[hi - 1]];
    auto area = [&] (int k) {
      return abs(a.cross(b, pts[ord[k]])); };
    if (hi < n) mn = min(mn, area(hi));
    if (lo > 0) mn = min(mn, area(lo - 1));
    mx = max({mx, area(0), area(n - 1)});
  }
}
#endif
```

8.8 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.
Usage: vt<array<int, 3>> t; // indices into A
for (F x : hull3d(A)) t.pb({x.a, x.b, x.c});
db vol = signedPolyVolume(A, t); 6ce6b2, 6 lines

```
template<class P>
db signedPolyVolume(const P &pts, const vt<array<int, 3>> &trilist) {
  db v = 0;
  for (auto [a, b, c] : trilist) v += pts[a].cross(pts[b]).dot(pts[c])
      ;
  return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long. unit, normal, rotate need T = double.
Usage: `Point3D<db> p(x, y, z);` ea8121, 32 lines

```
template<class T> struct Point3D {
  using P = Point3D;
  using R = const P&;
  T x, y, z;
  explicit Point3D(T x = 0, T y = 0, T z = 0) : x(x), y(y), z(z) {}
  bool operator<(R p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); }
  bool operator==(R p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); }
  P operator+(R p) const { return P(x + p.x, y + p.y, z + p.z); }
  P operator-(R p) const { return P(x - p.x, y - p.y, z - p.z); }
  P operator*(T d) const { return P(x * d, y * d, z * d); }
  P operator/(T d) const { return P(x / d, y / d, z / d); }
  T dot(R p) const { return x * p.x + y * p.y + z * p.z; }
  P cross(R p) const {
    return P(y * p.z - z * p.y, z * p.x - x * p.z, x * p.y - y * p.x);
  }
  T dist2() const { return x * x + y * y + z * z; }
  db dist() const { return sqrt((db) dist2()); }
  // Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
  db phi() const { return atan2(y, x); }
  // Zenith angle (latitude) to the z-axis in interval [0, pi]
  db theta() const { return atan2(sqrt(x * x + y * y), z); }
  P unit() const { return *this / (T)dist(); } //makes dist()=1
  // returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  // returns point rotated 'angle' radians ccw around axis
  P rotate(db angle, P axis) const {
    db s = sin(angle), c = cos(angle); P u = axis.unit();
    return u * dot(u) * (1 - c) + (*this) * c - cross(u) * s;
  }
};
```

3dHull.h

Description: Incremental 3D hull. The first four points must be non-coplanar. Returns outward triangles with original indices; later coplanarity and duplicates are allowed. May retain redundant boundary vertices. `hull3d_vertices` returns only extreme vertex indices; faces are unchanged. Face F: indices a, b, c (ccw from outside), q = outward normal. P3 is Point3D<db>; use Point3D<ll> for exact integer input. Requires reliable orientation signs; roundoff can affect degeneracies.
Time: Hull: $\mathcal{O}(n^2)$ time and space. Vertices: $\mathcal{O}(n + m \log m)$, m = number of triangles. "Point3D.h" 8299e6, 53 lines

```
using P3 = Point3D<db>;
struct F { P3 q; int a, b, c; };

vt<F> hull3d(const vt<P3>& A) {
  assert(size(A) >= 4 && (A[1] - A[0]).cross(A[2] - A[0]).dot(A[3] - A
      [0]) != 0);
  vt<vt<bool>> dead(size(A), vt<bool>(size(A), true));
  vt<F> FS;
  auto add = [&](int a, int b, int c) {
    FS.pb({(A[b] - A[a]).cross(A[c] - A[a]), a, b, c});
    dead[a][b] = dead[b][c] = dead[c][a] = false;
  };
  add(0, 1, 2); add(0, 2, 1); // Two-sided disk; point 3 makes a
      tetrahedron.
  FOR (i, 3, size(A)) {
    vt<F> keep;
    for (F f : FS) {
      if (f.q.dot(A[i] - A[f.a]) > 0)
        dead[f.a][f.b] = dead[f.b][f.c] = dead[f.c][f.a] = true;
      else keep.pb(f);
    }
    FS.clear();
    for (F f : keep) {
      int v[] = {f.a, f.b, f.c};
      F0R (j, 3) {
        int a = v[j], b = v[(j + 1) % 3];
        if (dead[b][a]) add(b, a, i);
      }
    }
    FS.insert(FS.end(), all(keep));
  }
  return FS;
} // 0c1a0e

// For a full-dimensional hull from hull3d. Does not modify its
    triangulation.
vt<int> hull3d_vertices(const vt<P3>& A, const vt<F>& faces) {
  map<pair<int, int>, int> edges;
  vt<int> neighbor(size(A), -1), corner(size(A)), ans;
  for (F f : faces) {
    int v[] = {f.a, f.b, f.c};
    F0R (j, 3) {
      int a = v[j], b = v[(j + 1) % 3], c = v[(j + 2) % 3];
      auto [it, added] = edges.emplace(minmax(a, b), c);
      if (added || f.q.dot(A[it->second] - A[a]) == 0) continue;
      F0R (k, 2) {
        if (neighbor[a] < 0) neighbor[a] = b;
        else if (!((A[neighbor[a]] - A[a]).cross(A[b] - A[a]) == P3())
            )
          corner[a] = 1;
        swap(a, b);
      }
    }
  }
  F0R (i, size(A)) if (corner[i]) ans.pb(i);
  return ans;
}
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (colatitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points. 0c341f, 8 lines

```
db sphericalDistance(db f1, db t1,
    db f2, db t2, db radius) {
  db dx = sin(t2) * cos(f2) - sin(t1) * cos(f1);
  db dy = sin(t2) * sin(f2) - sin(t1) * sin(f1);
  db dz = cos(t2) - cos(t1);
  db d = sqrt(dx * dx + dy * dy + dz * dz);
  return radius * 2 * asin(min(d / 2, 1.0));
}
```

Strings (9)

KMP.h

Description: pfun[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). match(s, pat) returns the start of each occurrence of pat in s; pat nonempty.
Time: $\mathcal{O}(n)$ 6a12d1, 16 lines

```
vi pfun(const string& s) {
  vi p(size(s));
  FOR (i, 1, size(s)) {
    int g = p[i - 1];
    while (g && s[i] != s[g]) g = p[g - 1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

vi match(const string &s, const string &pat) {
  vi p = pfun(pat + '\0' + s), res;
  FOR (i, size(p) - size(s), size(p))
    if (p[i] == size(pat)) res.pb(i - 2 * size(pat));
  return res;
}
```

BitsetLCS.h

Description: Length of the longest common subsequence of two strings (bytes). One bit string per row, one big addition per character of a: $|a|, |b| = 10^5$ in 0.1s.
Time: $\mathcal{O}(nm/64)$ 0556bc, 18 lines

```
using ull = unsigned long long;
int lcs(const string& a, const string& b) {
  int m = size(b), W = m / 64 + 1;
  vt<vt<ull>> M(256, vt<ull>(W));
  F0R (j, m) M[(uint8_t) b[j]][j / 64] |= 1ull << j % 64;
  vt<ull> V(W, ~0ull);
  for (char c : a) {
    ull carry = 0; auto& mk = M[(uint8_t) c];
    F0R (w, W) {
      ull v = V[w], t = v + (v & mk[w]), x = t + carry;
      carry = (t < v) | (x < t);
      V[w] = x | (v & ~mk[w]);
    }
  }
  int res = 0;
  F0R (j, m) res += ~V[j / 64] >> j % 64 & 1;
  return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Usage: `vi z = Z(s); occurrences of p in t: Z(p + '\0' + t), at j iff z[size(p) + 1 + j] >= size(p)`
Time: $\mathcal{O}(n)$ 98e47b, 12 lines

```
vi Z(const string& S) {
  vi z(size(S));
  int l = -1, r = -1;
  FOR (i, 1, size(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < size(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

BigMod.h

Description: Type H: integers mod $2^{64} - 1$ with $+, -, *, ==, <$. Hash base C. 6604be, 19 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "using H = ull;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
using ull = unsigned long long;
struct H {
```

```
  ull x; H(ull x = 0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t) x * o.x;
    return H((ull) m) + (ull) (m >> 64); }
  ull get() const { return x + !~x; }
  #define op(o) bool operator o (H oth) const { return get() o oth.get
       (); }
  op(==) op(<)
};
#undef op

static const H C = (ll) 1e11 + 3; // (order ~ 3e9; random also ok)
```

Hashing.h

Description: Substring hashes of a string or vector of values ≥ 0. Skip the stuff that starts with r if you don't care about reverse hashes etc.
Usage: HashInterval h(s); // s is an lvalue
h.hash_interval(l, r); // hash of s[l, r)
h.rhash_interval(l, r); // hash of reversed s[l, r)
// s[l, r) is a palindrome iff the two are equal
get_hashes(s, len); // hashes of all windows of length len
hash string(s); "BigMod.h" a4f961, 39 lines

```
struct HashInterval {
  vt<H> ha, pw, rha;
  template<class T>
  HashInterval(T& str) : ha(size(str) + 1), pw(ha), rha(ha) {
    pw[0] = 1;
    F0R (i, size(str)) {
      ha[i + 1] = ha[i] * C + str[i] + 1;
      pw[i + 1] = pw[i] * C;
    }
    ROF (i, 0, size(str)) rha[i] = rha[i + 1] * C + str[i] + 1;
  } // 54b6ee
  H hash_interval(int l, int r) { // hash [l, r)
    return ha[r] - ha[l] * pw[r - l];
  }
  H rhash_interval(int l, int r) { // hash of reversed [l, r)
    return rha[l] - rha[r] * pw[r - l];
  }
};

// get all hashes of length <len>
template<class T>
vt<H> get_hashes(T& str, int length) {
  if (size(str) < length) return {};
  H h = 0, pw = 1;
  F0R (i, length) h = h * C + str[i] + 1, pw = pw * C;
  vt<H> ret = {h};
  FOR (i, length, size(str)) {
    ret.pb(h = h * C + str[i] + 1
      - pw * (str[i - length] + 1));
  }
  return ret;
}

template<class T>
H hash_string(T& s) {
  H h = 0;
  for (auto c : s) h = h * C + c + 1;
  return h;
}
```

Manacher.h

Description: p[0][i] = k: s[i-k, i+k) is the longest even palindrome (length 2k). p[1][i] = k: s[i-k, i+k] is the longest odd palindrome (length 2k+1).
Time: $\mathcal{O}(N)$ 1c24a1, 13 lines

```
array<vi, 2> manacher(const string &s) {
  int n = size(s);
  array<vi, 2> p = {vi(n + 1), vi(n)};
  F0R (z, 2) for (int i = 0, l = 0, r = 0; i < n; i++) {
    int t = r - i + !z;
    if (i < r) p[z][i] = min(t, p[z][l + t]);
    int L = i - p[z][i], R = i + p[z][i] - !z;
    while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
      p[z][i]++, L--, R++;
    if (R > r) l = L, r = R;
  }
  return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: rotate(v.begin(), v.begin() + minRotation(v), v.end());
Time: $\mathcal{O}(N)$ 9bd725, 8 lines

```
int minRotation(string s) {
  int a = 0, N = size(s); s += s;
  F0R (b, N) F0R (k, N) {
    if (a + k == b || s[a + k] < s[b + k]) { b += max(0, k - 1); break
        ; }
    if (s[a + k] > s[b + k]) { a = b; break; }
  }
  return a;
}
```

Lyndon.h

Description: Lyndon factorization: the unique split $s = w_1 w_2 \cdots w_k$ with $w_1 \geq \cdots \geq w_k$, each w_i smaller than all its proper suffixes. Returns the start index of every factor.
Time: $\mathcal{O}(N)$ 20cbcd, 10 lines

```
vi duval(const string& s) {
  int n = size(s); vi st;
  for (int i = 0; i < n;) {
    int j = i + 1, k = i;
    while (j < n && s[k] <= s[j])
      k = s[k] < s[j] ? i : k + 1, j++;
    while (i <= k) st.pb(i), i += j - k;
  }
  return st;
}
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n + 1$, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. For $i < j$, lcp(sa[i], sa[j]) is the min of lcp[i+1..j]. The input string must not contain any nul chars. For integer input pass lim > max value (costs O(lim)); values in [1, lim) (bytes >= 128 under signed char also break the counting sort). Distinct substrings: $n(n+1)/2 - \sum$ lcp; longest repeated substring: max lcp. Longest common substring of a, b: build on $a + $ '\1' $+ b$ and take the max lcp of adjacent suffixes starting in different strings (not at the separator).
Time: $\mathcal{O}(N \log N)$ dcc93c, 22 lines

```
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string s, int lim = 256) { // vi s for ints
    s.pb(0); int n = size(s), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
      p = j, iota(all(y), n - j);
      F0R (i, n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      F0R (i, n) ws[x[i]]++;
      FOR (i, 1, lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      FOR (i, 1, n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

SuffixTree.h

Description: Ukkonen's algorithm for the compact trie of all suffixes. Edges are intervals $[l, r)$ of the input, so the tree has $O(N)$ nodes. Each internal node's suffix link removes the first character from its path. Node v has parent p[v], suffix link suf[v], children t[v][c] (-1 if none) and incoming edge a[l[v]..r[v]). The root is 0 (l = -1, r = 0); node 1 is auxiliary. Append a unique dummy symbol to make every suffix an explicit leaf; without one, some suffixes end inside edges (substring matching still works). Every character, the dummy included, lies in ['a', 'a'+alpha); pass alpha to the constructor (default 27, dummy '{').
Time: $\mathcal{O}(\Sigma N)$
Memory: $\approx (8\Sigma + 120)N$ bytes b67f06, 50 lines

```
struct SuffixTree {
  int toi(char c) { return c - 'a'; }
  string a; // v = current node, q = current position
  int A, v = 0, q = 0, m = 2;
  vt<vi> t;
  vi l, r, p, suf;

  void ukkadd(int i, int c) { suff:
    if (r[v] <= q) {
      if (t[v][c] == -1) { t[v][c] = m; l[m] = i;
        p[m++] = v; v = suf[v]; q = r[v]; goto suff; }
      v = t[v][c]; q = l[v];
    } // 30b98e
    if (q == -1 || c == toi(a[q])) q++; else {
      l[m + 1] = i; p[m + 1] = m; l[m] = l[v]; r[m] = q;
      p[m] = p[v]; t[m][c] = m + 1; t[m][toi(a[q])] = v;
      l[v] = q; p[v] = m; t[p[m]][toi(a[l[m]])] = m;
      v = suf[p[m]]; q = l[m];
      while (q < r[m]) { v = t[v][toi(a[q])]; q += r[v] - l[v]; }
      suf[m] = q == r[m] ? v : m + 2;
      q = r[v] - (q - r[m]); m += 2; goto suff;
    }
  } // af1da8

  SuffixTree(string a, int alpha = 27) : a(a), A(alpha),
    t(max(2, 2 * size(a) + 1), vi(A, -1)), l(size(t)), r(size(t), size
        (a)),
    p(size(t)), suf(size(t)) {
    F0R (c, A) t[1][c] = 0;
    suf[0] = 1; l[0] = l[1] = -1;
    r[0] = r[1] = p[0] = p[1] = 0;
    F0R (i, size(a)) ukkadd(i, toi(a[i]));
  } // b6caed

  // Example: longest common substring as {length, start in s};
       assumes a-z.
  pi best;
  int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
```

```
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + r[node] - l[node] : 0;
    F0R (c, A) if (t[node][c] != -1)
      mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3) best = max(best, pi{len, r[node] - len});
    return mask;
  }
  static pi LCS(string s, string t) {
    SuffixTree st(s + char('z'+1) + t + char('z'+2), 28);
    st.lcs(0, size(s), size(s) + 1 + size(t), 0);
    return st.best;
  }
};
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(pat); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(pat, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. **Time:** construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$. 3adc8b, 66 lines

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // 'a' for lowercase
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  }; // 9e9dd8
  vt<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = size(N); N.emplace_back(-1); }
      else n = m;
    } // 7b7b40
    if (N[n].end == -1) N[n].start = j;
    backp.pb(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vt<string>& pat) : N(1, -1) {
    F0R (i, size(pat)) insert(pat[i], i);
    N[0].back = size(N);
    N.emplace_back(0); // fc0d63

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      F0R (i, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    } // 6492fc
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.pb(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  } // f5fdfc
  vt<vi> findAll(vt<string>& pat, string word) {
    vi r = find(word);
    vt<vi> res(size(word));
    F0R (i, size(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - size(pat[ind]) + 1].pb(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

SuffixAutomaton.h

Description: Used infrequently. Constructs minimal deterministic finite automaton (DFA) that recognizes all suffixes of a string. `len` corresponds to the maximum length of a string in the equivalence class, `pos` corresponds to the first ending position of such a string, `lnk` corresponds to the longest suffix that is in a different class. Suffix links correspond to suffix tree of the reversed string!
Usage: `S.init(s); S.genIlnk(); S.allOccur(t); // start idx`
`Online use: p = 0; p = S.add(p, c). Root is state 0.`
Time: $\mathcal{O}(N \log \sum)$ 0d916e, 71 lines

```
struct SuffixAutomaton {
  int N = 1; vi lnk{-1}, len{0}, pos{-1}; // suffix link,
  // max length of state, last pos of first occurrence of state
  vt<map<char, int>> nex{1}; vt<bool> isClone{0};
  // transitions, cloned -> not terminal state
  vt<vi> iLnk; // inverse links
  int add(int p, char c) { // ~p nonzero if p != -1
    auto getNex = [&] () {
      if (p == -1) return 0;
      int q = nex[p][c]; if (len[p] + 1 == len[q]) return q;
      int clone = N++; lnk.pb(lnk[q]); lnk[q] = clone;
      len.pb(len[p] + 1), nex.pb(nex[q]),
      pos.pb(pos[q]), isClone.pb(1);
      for (; ~p && nex[p][c] == q; p = lnk[p]) nex[p][c] = clone;
      return clone;
    }; // ce7647
    // if (nex[p].count(c)) return getNex();
    // ^ need if adding > 1 string
    int cur = N++; // make new state
    lnk.emplace_back(), len.pb(len[p] + 1), nex.emplace_back(),
    pos.pb(pos[p] + 1), isClone.pb(0);
    for (; ~p && !nex[p].count(c); p = lnk[p]) nex[p][c] = cur;
    int x = getNex(); lnk[cur] = x; return cur;
  } // 54eddc
  void init(string s) { int p = 0;
    for (auto& x : s) p = add(p, x); }
  // inverse links
  void genIlnk() {iLnk.resize(N); FOR (v, 1, N)iLnk[lnk[v]].pb(v); }
  // APPLICATIONS
  void getAllOccur(vi& oc, int v) {
    if (!isClone[v]) oc.pb(pos[v]); // terminal position
    for (auto& u : iLnk[v]) getAllOccur(oc, u); }
  vi allOccur(string s) { // all occurrences of s
    int cur = 0;
    for (auto& x : s) {
      if (!nex[cur].count(x)) return {};
      cur = nex[cur][x]; }
    // convert end pos -> start pos
    vi oc; getAllOccur(oc, cur);
    for (auto& t : oc) t += 1 - size(s);
    sort(all(oc)); return oc;
  } // 3cf3bf
  vl distinct;
  ll getDistinct(int x) {
    // # distinct strings starting at state x
    if (distinct[x]) return distinct[x];
    distinct[x] = 1;
    for (auto& y : nex[x]) distinct[x] += getDistinct(y.s);
    return distinct[x]; }
  ll numDistinct() { // # distinct substrings including empty
    distinct.resize(N); return getDistinct(0); }
  ll numDistinct2() { // assert(numDistinct()==numDistinct2());
    ll ans = 1; FOR (i, 1, N) ans += len[i] - len[lnk[i]];
    return ans; }
}; // 82d2b0

SuffixAutomaton S;
vi sa; string s;
void dfs(int x) {
  if (!S.isClone[x]) sa.pb(size(s) - 1 - S.pos[x]);
  vt<pair<char, int>> chr;
  for (auto& t : S.iLnk[x]) chr.pb({s[S.pos[t] - S.len[x]], t});
  sort(all(chr)); for (auto& t : chr) dfs(t.s);
}

int main() {
  cin >> s; reverse(all(s));
  S.init(s); S.genIlnk();
  dfs(0); // generating suffix array for s
  for (int x : sa) cout << x << ' '; cout << '\n';
}
```

PalTree.h

Description: Used infrequently. Palindromic tree computes number of occurrences `d[v].oc` of each palindrome within string, valid after calling `numOc()` once. Only a-z. `ans[i][0]` stores min even x such that the prefix $s[1..i]$ can be split into exactly x palindromes, `ans[i][1]` does the same for odd x.
Time: $\mathcal{O}(N \sum)$ for addChar, $\mathcal{O}(N \log N)$ for updAns 8c6926, 44 lines

```
struct PalTree {
  static const int ASZ = 26;
  struct node {
    array<int, ASZ> to = array<int, ASZ>();
    int len, link, oc = 0; // # occurrences of pal
    int slink = 0, diff = 0;
    array<int, 2> seriesAns;
    node(int _len, int _link) : len(_len), link(_link) {}
  };
  string s = "@"; vt<array<int, 2>> ans = {{0, MOD}};
  vt<node> d = {{0, 1}, {-1, 0}}; // dummy pals of len 0,-1
  int last = 1;
  int getLink(int v) {
    while (s[size(s) - d[v].len - 2] != s.back()) v = d[v].link;
    return v;
  } // 532c07
  void updAns() { // serial path has O(log n) vertices
    ans.pb({MOD, MOD});
    for (int v = last; d[v].len > 0; v = d[v].slink) {
      d[v].seriesAns =
        ans[size(s) - 1 - d[d[v].slink].len - d[v].diff];
      if (d[v].diff == d[d[v].link].diff)
        F0R (i, 2) d[v].seriesAns[i] = min(d[v].seriesAns[i],
```

```
          d[d[v].link].seriesAns[i]);
      // start of previous oc of link[v]=start of last oc of v
      FOR (i, 2) ans.back()[i] = min(ans.back()[i],
        d[v].seriesAns[i ^ 1] + 1);
    }
  } // 671177
  void addChar(char C) {
    s += C; int c = C-'a'; last = getLink(last);
    if (!d[last].to[c]) {
      d.emplace_back(d[last].len + 2,
        d[getLink(d[last].link)].to[c]);
      d[last].to[c] = size(d) - 1;
      auto& z = d.back(); z.diff = z.len - d[z.link].len;
      z.slink = z.diff == d[z.link].diff
        ? d[z.link].slink : z.link;
    } // max suf with different dif
    last = d[last].to[c]; ++d[last].oc;
    updAns();
  }
  void numOc() { ROF (i, 2, size(d)) d[d[i].link].oc += d[i].oc; }
};
```

WildcardPatternMatching.h

Description: Wildcard pattern matching. wc is wildcard character. res[i] = '1' iff pattern matches text starting at i. Idea: give every character a random value r; over the non-wildcard pairs a match means $\sum_j r(p_j)(r(p_j) - r(t_{i+j})) = 0$, which two convolutions evaluate for all i. A mismatch survives with probability $\leq 2/mod$ per alignment, so $\sim 2N/mod$ overall; to square that, run again with fresh random values (each call) or another wpm_mod and AND.
Usage: string r = wildcard_pattern_matching(t, p, '?');
t, p same type, p nonempty; r has |t|-|p|+1 chars
Time: $\mathcal{O}(N \log N)$ "../numerical/FastFourierTransformMod.h" 59f996, 22 lines

```
const int wpm_mod = (119 << 23) + 1;
template<class T_arr, class T>
string wildcard_pattern_matching(const T_arr& text, const T_arr&
    pattern, T wc) {
  static mt19937_64 rng;
  int N = size(text), M = size(pattern);
  if (N == 0 || M > N) return "";
  assert(M);
  map<T, ll> char_to_rnd;
  auto f = [&] (T x) { ll& r = char_to_rnd[x];
    if (!r) r = rng() % (wpm_mod - 1) + 1; return r; };
  vl mt(N), mp(M);
  FOR (i, N) mt[i] = text[i] == wc ? 0 : f(text[i]);
  FOR (i, M) mp[M - 1 - i] = pattern[i] == wc ? 0 : f(pattern[i]);
  vl sc1 = convMod<wpm_mod>(mp, mt);
  FOR (i, N) mt[i] = text[i] == wc;
  ll must = 0;
  for (ll& x : mp) x = x * x % wpm_mod, must = (must + x) % wpm_mod;
  vl sc2 = convMod<wpm_mod>(mp, mt);
  string res(N - M + 1, '0');
  FOR (i, M - 1, N) res[i + 1 - M] += (sc1[i] + sc2[i]) % wpm_mod ==
      must;
  return res;
}
```

Heuristics (10)

10.1 Tips

10.1.1 Simulated Annealing

Default temperature schedule for SA:
$t = t_0 \cdot (\frac{t_n}{t_0})^{\text{time_passed}}, \text{time_passed} \in [0, 1]$
Acceptance function: RNG() $< \exp((\text{cur} - \text{new})/t)$ (minimising; swap cur and new to maximise)
A good schedule should have a slowly decreasing acceptance rate.
Priorities:

- Prototype different transitions.
- Optimize eval function (= make it dynamic).
- Find a proper temp schedule. Consider starting with lower temperatures and increasing until it starts getting worse.

Things to do if its easy to get stuck in a local optimum:

- Use a very high temperature/acceptance rate
- Complex transitions
- Scale transitions with temperature (maybe?)
- (rarely) kicks
- (rarely) restarting temp schedule
- (rarely) multiple runs

Things to do when there are lots of invalid states:

- Add an additional scoring component based on number of collisions
- Do many short SA runs with high temp on partial subsolutions

If your eval is very fast, make sure RNG, exp and elapsed() are not bottlenecks. When SA is bad:

- Very easy to get stuck in local optima
- Simple transitions are mixed with complex ones – have a separate acceptance function for each transition
- Eval is erratic during transitions

Use a good local tester: make sure it has multithreading and relative scoring. If you are considering whether its worth optimising speed, just run with higher time limit and see if the improvements are worth it. Removing bugs is #1 priority; results are meaningless when bugs are involved. If results do not align with intuition, investigate. Merge N-dimensional coordinates into 1D. Memory allocation is slow.

10.1.2 Beam search

Dedupe states by hash; the top-W often share a parent. Use nth_element, not a full sort. Evaluate incrementally. Choose W from the time of one generation.

RNG.h

Description: Mersenne twister (same output as mt19937).
Usage: RNG r(seed);
r.next(n) // [0, n)
r.next(a, b) // [a, b)
r.next_double() // (0, 1)
r.next() // any int
splitmix64() // global, 64-bit 740f0e, 34 lines

```
struct RNG {
  unsigned int MT[624];
  int index;
  RNG(int seed = 1) {init(seed); }
  void init(int seed = 1) {MT[0] = seed; FOR (i, 1, 624) MT[i] = (
      1812433253UL * (MT[i - 1] ^ (MT[i - 1] >> 30)) + i); index =
      0; }
  void generate() {
    const unsigned int MULT[] = {0, 2567483615UL};
    FOR (i, 227) {unsigned int y = (MT[i] & 0x80000000UL) + (MT[i + 1]
        & 0x7FFFFFFFUL); MT[i] = MT[i + 397] ^ (y >> 1); MT[i] ^=
        MULT[y & 1]; }
    FOR (i, 227, 623) {unsigned int y = (MT[i] & 0x80000000UL) + (MT[i
        + 1] & 0x7FFFFFFFUL); MT[i] = MT[i - 227] ^ (y >> 1); MT[i]
        ^= MULT[y & 1]; }
    unsigned int y = (MT[623] & 0x80000000UL) + (MT[0] & 0x7FFFFFFFUL)
        ; MT[623] = MT[623 - 227] ^ (y >> 1); MT[623] ^= MULT[y &
        1];
  }
  unsigned int rand() { if (index == 0) generate(); unsigned int y =
      MT[index]; y ^= y >> 11; y ^= y << 7 & 2636928640UL; y ^= y <<
      15 & 4022730752UL; y ^= y >> 18; index = index == 623 ? 0 :
      index + 1; return y; }
  inline int next() {return rand(); }
  inline int next(int x) {return rand() % x; }
  inline int next(int a, int b) {return a + (rand() % (b - a)); }
  inline db next_double() {return (rand() + 0.5) * (1.0 / 4294967296.0
      ); }
  inline db next_double(db a, db b) {return a + next_double() * (b - a
      ); }
};

static uint64_t splitmix64(uint64_t x) {
  // http://xorshift.di.unimi.it/splitmix64.c
  x += 0x9e3779b97f4a7c15;
  x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
  x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
  return x ^ (x >> 31);
}

static uint64_t x = 1234; // any value
static uint64_t splitmix64() {
  uint64_t z = (x += 0x9e3779b97f4a7c15);
  z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9;
  z = (z ^ (z >> 27)) * 0x94d049bb133111eb;
  return z ^ (z >> 31);
}
```

SimAnneal.h

Description: Complete SA program (own main) with an example problem. gen(l, r) is inclusive. <bits/extc++.h>, <sys/time.h> 4a6f97, 117 lines

```
#pragma GCC optimize("Ofast,omit-frame-pointer,inline,unroll-all-loops
    ")


#pragma GCC target("avx2")

using namespace std;

// include standard macros
```

```
__gnu_cxx::sfmt19937 mt(random_device{}());
int gen(int l, int r) { return uniform_int_distribution<int>(l, r)(mt)
    ; }
db next_double() { return uniform_real_distribution<db>(0, 1)(mt); }

db get_time() {timeval tv; gettimeofday(&tv, NULL); return tv.tv_sec +
    tv.tv_usec * 1e-6; }
db start_time = get_time();
db elapsed() {return get_time() - start_time; }

const db TIME_LIMIT = 0.95;
db t0 = 1e9, tn = 0.1, time_passed = 1e-9;
const db maximise_score = -1; // -1 if minimise

// data
int n;
vi a;
// end data

using T = int;
struct State {
  T value;
  vi a;

  // static transition: consider optimising to dynamic
  void to_neighbour() {
    int i, j;
    switch (gen(0, 4)) {
      case 0:
        i = gen(0, n - 1);
        j = gen(0, n - 2);
        if (j >= i) j++;
        swap(a[i], a[j]);
        break;
      case 1:
        a[gen(0, n - 1)] *= -1;
        break;
      case 2:
      case 3:
        i = gen(0, n);
        j = gen(0, n);
        if (i > j) swap(i, j);
        reverse(a.begin() + i, a.begin() + j);
        break;
      case 4:
        i = gen(0, n);
        j = gen(0, n);
        if (i > j) swap(i, j);
        FOR (k, i, j) a[k] *= -1;
        break;
    }
  }

  T calc_value() {
    T ans = 0, sum = 0;
    for (int x : a) sum += x, ans += abs(sum);
    return value = ans;
  }

  void print() {
    FOR (i, n) {
      if (a[i] > 0) cout << a[i] << " a\n";
      else cout << -a[i] << " f\n";
    }
    cout << value << '\n';
  }
};

State cur, best;

signed main() {
  cin.tie(0)->sync_with_stdio(0);

  cin >> n;
  a.resize(n);
  for (int &x : a) cin >> x;
  sort(all(a));
  FOR (i, n) if (a[i] % 4 == 1 || a[i] % 4 == 2) a[i] *= -1;
  cur.a = a;
  cur.calc_value();
  best = cur;
  t0 = cur.value * 2;

  if (n == 1) {
    cur.print();
    return 0;
  }

  int its = 0;
  db t;
  while (true) {
    if ((its & 511) == 0) { // lower if eval slow
      time_passed = elapsed() / TIME_LIMIT;
      if (time_passed > 1.0) break;
      t = t0 * pow(tn / t0, time_passed);
    }
    its++;

    State neighbour = cur;
    neighbour.to_neighbour();
    if ((neighbour.calc_value() - cur.value) * maximise_score >= 0
      || next_double() < exp(((neighbour.value - cur.value) *
          maximise_score) / t)) {
      cur = neighbour;
      if ((cur.value - best.value) * maximise_score > 0) {
        best = cur;
      }
    }
  }
  best.print();
}
```

BeamSearch.h

Description: Beam search example: split $a_i > 0$ into k buckets maximising the product of bucket sums. Edit `State`, the transitions and `calc_score`; set `TIME_LIMIT`. <bits/extc++.h>, <sys/time.h> 268bcf, 188 lines

```
#pragma GCC optimize("Ofast,omit-frame-pointer,inline,unroll-all-loops
    ")


#pragma GCC target("avx2")

using namespace std;
using ll = long long;

#define vt vector
#define f first
#define s second
#define pb push_back
#define all(x) begin(x), end(x)
#define size(x) ((int) (x).size())
#define FOR(i, a, b) for (int i = (a); i < (b); i++)
#define ROF(i, a, b) for (int i = (b) - 1; i >= (a); i--)
#define F0R(i, b) FOR (i, 0, b)
const int inf = 1e9;
const ll INF = 1e18;

using vi = vt<int>;
using db = long double;

__gnu_cxx::sfmt19937 mt(random_device{}());

int gen(int l, int r) {
  return uniform_int_distribution<int>(l, r)(mt);
}

double get_time() {timeval tv; gettimeofday(&tv, NULL); return tv.
    tv_sec + tv.tv_usec * 1e-6; }
double start_time = get_time();
double elapsed() {return get_time() - start_time; }

const db TIME_LIMIT = 5;

// data

int n, k;
vector<int> a;
db ik;

// end data

using T = db;

struct State {
  T value{};
  vt<vi> buckets;
  vt<db> sums;

  void print() {
    F0R (i, k) {
      for (int x : buckets[i]) cout << x << ' ';
      cout << '\n';
    }
    cerr << "value: " << fixed << setprecision(24) << value << '\n';
  }
};

class Transition {
public:
  T new_val{};
  virtual ~Transition() = default;
  virtual State accept() = 0;
  bool operator<(const Transition& oth) const { return new_val < oth.
      new_val; }
};

class Swap : public Transition {
  State& cur;
  int i{}, j{}, el{};
public:
  explicit Swap(State& cur_) : cur(cur_) {
    do { i = gen(0, k - 1); } while (size(cur.buckets[i]) <= 1);

    j = gen(0, k - 2);
    if (j >= i) ++j;

    el = gen(0, size(cur.buckets[i]) - 1);
    int x = cur.buckets[i][el];

    new_val = cur.value
      / powl(cur.sums[i], ik) / powl(cur.sums[j], ik)
      * powl(cur.sums[i] - x, ik) * powl(cur.sums[j] + x, ik);
  }

  State accept() override {
```

```
    State nxt = cur;
    int &x = nxt.buckets[i][el];
    nxt.buckets[j].pb(x);
    nxt.sums[i] -= x;
    nxt.sums[j] += x;
    swap(x, nxt.buckets[i].back());
    nxt.buckets[i].pop_back();
    nxt.value = new_val;
    return nxt;
  }
};

class Move : public Transition {
  State& cur;
  int i{}, j{}, xi{}, yi{};
public:
  explicit Move(State& cur_) : cur(cur_) { // exchanges two elements
    do { i = gen(0, k - 1); } while (cur.buckets[i].empty());
    do { j = gen(0, k - 2); if (j >= i) ++j; } while (cur.buckets[j].
        empty());

    xi = gen(0, size(cur.buckets[i]) - 1);
    yi = gen(0, size(cur.buckets[j]) - 1);

    int x = cur.buckets[i][xi];
    int y = cur.buckets[j][yi];

    new_val = cur.value
        / powl(cur.sums[i], ik) / powl(cur.sums[j], ik)
        * powl(cur.sums[i] - x + y, ik) * powl(cur.sums[j] - y + x, ik
            );
  }

  State accept() override {
    State nxt = cur;
    int &x = nxt.buckets[i][xi];
    int &y = nxt.buckets[j][yi];
    nxt.sums[j] += x - y;
    nxt.sums[i] += y - x;
    swap(x, y);
    nxt.value = new_val;
    return nxt;
  }
};

db calc_score(State& state) {
  db val = 1;
  F0R (i, k) val *= powl(state.sums[i], ik);
  return val;
}

signed main() {
  cin.tie(0)->sync_with_stdio(0);

  cin >> n >> k;
  a.resize(n);
  for (int &x : a) cin >> x;

  State cur;
  cur.buckets.resize(k);
  cur.sums.assign(k, 0);
  F0R (i, n) {
    int j = int(min_element(all(cur.sums)) - cur.sums.begin());
    cur.sums[j] += a[i];
    cur.buckets[j].push_back(a[i]);
  }
  ik = 1.0l / k;
  cur.value = calc_score(cur);

  const int beam_width = 100;
  const int neighbours = 100;

  vector<State> states{cur};
  State best = cur;

  int its = 0;
  while (elapsed() < TIME_LIMIT) {
    its++;
    vector<unique_ptr<Transition>> transitions;
    transitions.reserve(size(states) * neighbours * 2);

    for (auto &st : states) {
      if (st.value > best.value) best = st;
      F0R (_, neighbours) {
        // n > k: some bucket has 2+ els, else Swap's ctor spins
        if (k > 1 && n > k) transitions.emplace_back(make_unique<Swap>
            (st));
        if (k > 1 && n > 1) transitions.emplace_back(make_unique<Move>
            (st));
      }
    }

    if (transitions.empty()) break;

    sort(all(transitions), [] (const auto& a, const auto& b) { return
        a->new_val > b->new_val; });

    vector<State> new_states;
    int take = min<int>(beam_width, size(transitions));
    new_states.reserve(take);
    F0R (i, take) new_states.push_back(transitions[i]->accept());

    states = std::move(new_states);
  }

  for (auto &st : states) if (st.value > best.value) best = st;
  best.print();
}
```

Various (11)

11.1 Game Theory

Impartial games: Both players have the same set of moves from any position. Normal play: Last move wins

11.1.1 Grundy Numbers

For some position P:

$$G(P) = \text{mex}\{\, G(P') \mid P' \text{ is reachable from } P \,\}$$

Idea: $G(P) = 0$ iff P is losing.
Sprague-Grundy theorem: if you combine independent games $P_1, P_2, \ldots, P_k$:

$$G(\text{sum}) = G(P_1) \oplus G(P_2) \oplus \cdots \oplus G(P_k)$$

Green Hackenbush: a graph with some vertices on the ground; a move deletes an edge, then everything no longer connected to the ground disappears; last move wins.
A vertex's subtrees can be replaced by a single line of length $\bigoplus_{c \in \text{children}} (1 + \text{line}(c))$ (the connecting edge counts). A cycle can be contracted into a single node with its other edges sticking out; each cycle edge becomes a loop, worth a stalk of length 1. All ground vertices count as one.

11.1.2 Nim variants

Misere nim: normal nim, unless all piles are ≤ 1: then the first player wins iff the number of 1-piles is even. **Staircase nim** (coins move from step i to $i-1$, step 0 = ground): only odd steps count, nim on their XOR. **Moore's Nim$_k$** (take from up to k piles): losing iff for every bit the number of piles with that bit set is divisible by $k+1$. **Wythoff** (take from one pile, or equally from both): P-positions $(\lfloor m\varphi \rfloor, \lfloor m\varphi \rfloor + m)$, $\varphi = \frac{1+\sqrt{5}}{2}$: $(0,0), (1,2), (3,5), (4,7), (6,10), \ldots$ **Lasker's nim** (take, or split a pile in two): $G(4m+1) = 4m+1$, $G(4m+2) = 4m+2$, $G(4m+3) = 4m+4$, $G(4m+4) = 4m+3$.

11.2 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Merges the added interval with any overlapping or touching intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: $\mathcal{O}(\log N)$ 5cd25a, 23 lines

```
set<pi>::iterator addInterval(set<pi>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L, R});
}

void removeInterval(set<pi>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).
Usage: `vt<pi> I; vi r = cover({0, 10}, I); // indices into I`
Time: $\mathcal{O}(N \log N)$ 113d69, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vt<pair<T, T>> I) {
  vi S(size(I)), R;
  iota(all(S), 0);
  sort(all(S), [&] (int a, int b) { return I[a] < I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
```

```
    pair<T, int> mx = make_pair(cur, -1);
    while (at < size(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.pb(mx.second);
  }
  return R;
}
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: `constantIntervals(0, size(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`

Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$ 753a4c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid + 1, to, f, g, i, p, q);
  }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to <= from) return;
  int i = from; auto p = f(i), q = f(to - 1);
  rec(from, to - 1, f, g, i, p, q);
  g(i, to, q);
}
```

11.3 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f, change the $<$ at (A) and (B) to $>$.

Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`

Time: $\mathcal{O}(\log(b-a))$ 888bb8, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid + 1)) a = mid; // (A)
    else b = mid + 1;
  }
  FOR (i, a + 1, b + 1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

CyclicMax.h

Description: Argmax of a cyclically unimodal sequence $f(0..n-1)$: read from its minimum it strictly increases up to the maximum and then strictly decreases back, wrapping around. The max and the min may each be shared by two adjacent indices, never more, and no other neighbours may be equal. Returns $(i, i+1 \bmod n)$ if $f(i) = f(i+1)$ is the max, else $(i, -1)$. `cyc_peak` takes a 3-way `comp(i, j)` = sign of $f(i) - f(j)$, exactly $-1/0/1$, instead of values (angles via cross products, or negate for the argmin). At most $5 \log_2 n + 3$ comparisons. The eps version needs consecutive values to differ by more than eps except at the two ties.

Usage: `cyc_max(n, [&](int i) { return poly[i].dot(dir); })`

Time: $\mathcal{O}(\log n)$ 459005, 21 lines

```
template<class C> pi cyc_peak(int n, C comp) {
  auto c = [&](int i, int j) { return comp(i % n, j % n); };
  auto peak = [&](int i) {
    return c(i, i + 1) >= 0 && c(i - 1 + n, i) < 0; };
  int lo = 0, hi = n;
  if (!peak(0)) while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (peak(m)) { lo = m; break; }
    int ls = c(lo, lo + 1), ms = c(m, m + 1);
    (ls < ms || (ls == ms && ls == c(m, lo)) ? hi : lo) = m;
  }
  return {lo, c(lo, lo + 1) || n < 2 ? -1 : (lo + 1) % n};
}
template<class F> pi cyc_max(int n, F fn) { // anything with <
  return cyc_peak(n, [&](int i, int j) {
    auto a = fn(i), b = fn(j); return (b < a) - (a < b); });
} // 0a4555
template<class F> pi cyc_max(int n, F fn, db eps) { // doubles
  return cyc_peak(n, [&](int i, int j) {
    db d = fn(i) - fn(j); return (d > eps) - (d < -eps); });
}
```

LIS.h

Description: `lis_dp` gives the strictly increasing LIS length ending at each index, `lis_construct` the indices of one LIS.

Time: $\mathcal{O}(N \log N)$ 4df05b, 27 lines

```
vi lis_dp(const vi &a) {
  int n = size(a);
  vi ans(n), dp(n, INT_MAX);
  F0R (i, n) {
    int j = lower_bound(all(dp), a[i]) - dp.begin(); // strict
    dp[j] = a[i], ans[i] = j + 1;
  }
  return ans;
}

vi lis_construct(const vi &a) {
  if (a.empty()) return {};
  int n = size(a);
  vi prev(n);
  vt<pi> res;
  F0R (i, n) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(all(res), pi{a[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.end() - 1;
    *it = {a[i], i};
    prev[i] = it == res.begin() ? 0 : (it - 1)->second;
  }
  int m = size(res), cur = res.back().s;
  vi ans(m);
  while (m--) ans[m] = cur, cur = prev[cur];
  return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N \max(w_i))$

Memory: $\mathcal{O}(\max(w_i))$ 3150b0, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < size(w) && a + w[b] <= t) a += w[b++];
  if (b == size(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2 * m, -1);
  v[a + m - t] = b;
  FOR (i, b, size(w)) {
    u = v;
    F0R (x, m) v[x + w[i]] = max(v[x + w[i]], u[x]);
    for (x = 2 * m; --x > m;) FOR (j, max(0, u[x]), v[x])
      v[x - w[j]] = max(v[x - w[j]], j);
  }
  for (a = t; v[a + m - t] < 0; a--) ;
  return a;
}
```

11.4 Dynamic programming

11.4.1 DP optimisations

- $dp_i = \min_{j<i} dp_j + w(j,i)$ with w Monge $(w(a,c) + w(b,d) \leq w(a,d) + w(b,c)$ for $a \leq b \leq c \leq d$, e.g. $g(S_i - S_j)$, g convex, S sorted): argmin is monotone. CDQ D&C $O(n \log^2 n)$, or a monotone deque with binary search $O(n \log n)$.
- $dp_i = \min_j dp_j + b_j a_i$: CHT / Li Chao.
- Layers $dp_{t,i} = \min_j dp_{t-1,j} + w(j,i)$: D&C DP $O(kn \log n)$.

Aliens trick: $f(k)$ = optimum with exactly k items, f convex. Solve the relaxed DP $g(\lambda) = \min_k (f(k) + \lambda k)$, returning the item count $c(\lambda)$ with ties broken to the smallest. Binary search the smallest integer λ with $c(\lambda) \leq k$ (range $\pm \max |f(i+1) - f(i)|$); then $f(k) = g(\lambda) - \lambda k$.

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \leq f(a,d)$ and $f(a,c) + f(b,d) \leq f(a,d) + f(b,c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search. Take $p[i][i+1] = i+1$ and search $k \in [p[i][j-1], \min(j-1, p[i+1][j])]$.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$. lo/hi are half-open; solve(L, R) fills $[L, R)$. store gets the minimal optimal k (needs monotone argmin, e.g. quadrangle inequality).

Usage: `DP d; d.solve(0, n); // res[i] = {value, argmin}`

Time: $\mathcal{O}((N + (hi - lo)) \log N)$ 9a753e, 18 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
```

```
  void store(int ind, int k, ll v) { res[ind] = {v, k}; } // vt<pair<
       ll, int>> res

  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    FOR (k, max(LO, lo(mid)), min(HI, hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second + 1);
    rec(mid + 1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

11.5 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and underflows (16).

LeakSanitizer.h

Description: Disables the leak check under `-fsanitize=address`.

bc29d7, 4 lines

```
#ifdef __cplusplus
extern "C"
#endif
const char* __asan_default_options() { return "detect_leaks=0"; }
```

11.6 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

11.6.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x & -x, r = x + c; (((r ^ x) >> 2) / c) | r` is the next number after `x` with the same number of bits set.
- `F0R (b, K) F0R (i, (1 << K))`
  `if (i & 1 << b) D[i] += D[i ^ (1 << b)];`
  computes all sums of subsets.

11.6.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("O0", "trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$.

e79cd0, 8 lines

```
using ull = unsigned long long;
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {}
  ull reduce(ull a) { // a % b + (0 or b)
    return a - (ull) ((__uint128_t(m) * a) >> 64) * b;
  }
};
```

FastInput.h

Description: Read an `int` (32-bit, no EOF check) from stdin. Usage requires your program to pipe in input from file; don't mix with `cin`.
Usage: `./a.out < input.txt`
Time: About 5x as fast as cin/scanf.

7b3c70, 17 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
}

int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a * 10 + c - 480;
  return a - 48;
}
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation. Total bytes ever allocated must stay below 450MB (delete is a no-op). Standard alternative: `std::pmr::monotonic_buffer_resource` with `std::pmr::vector` etc. (C++17), but those are different container types, so overriding operator new is the drop-in option.

adee36, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
  static size_t i = sizeof buf;
  assert(s < i);
  return (void*) & buf[i = (i - s) & -16];
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.
Usage: `struct Node { ptr<Node> l, r; }; ptr<Node> p = new Node;`
Compare with `a.ind == b.ind`; null is `ind 0` (`if (p)`).

"BumpAllocator.h" 2dd6c9, 10 lines

```
template<class T> struct ptr {
  unsigned ind;
  ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
  }
  T& operator*() const { return *(T*)(buf + ind); }
  T* operator->() const { return &**this; }
  T& operator[] (int a) const { return (&**this)[a]; }
  explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.
Usage: `vt<vt<int, small<int>>> ed(N);`

a64201, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
  using value_type = T;
  small() {}
  template<class U> small(const U&) {}
  T* allocate(size_t n) {
    buf_ind -= n * sizeof(T);
    buf_ind &= 0 - alignof(T);
    return (T*)(buf + buf_ind);
  }
  void deallocate(T*, size_t) {}
};
```

Unrolling.h

Description: Paste inside a function with `from`, `to` in scope; body in F must be straight-line (no break/continue, do not touch i). `#undef F` after.

520e76, 5 lines

```
#define F {...; ++i;}
int i = from;
while (i & 3 && i < to) F // for alignment, if needed
while (i + 4 <= to) { F F F F }
while (i < to) F
```

11.7 Bigger types

- `__int128` / `unsigned __int128`: 128-bit ints, up to $\pm 1.7 \times 10^{38}$. No `cin/cout`: cast to `ll` or print digit by digit. Multiply is a few instructions, divide is slow (~100 cycles). Use for exact 64×64-bit products, mulmod, sums up to 10^{36}.
- `long double`: 80-bit x87 on Linux (64-bit mantissa, ~19 digits), ~3× slower than `double`. Use `sqrtl`, `powl`, `fabsl`, `%Lf`.
- `__float128`: 113-bit mantissa (~34 digits), software emulated (libgcc calls, ~25× slower than `double`). Arithmetic and casts just work; `sqrtq`, `expq` and printing need `<quadmath.h>` + `-lquadmath`, so print via `(long double)` or peel digits manually.

BigInt.h

Description: Unsigned bigint of exactly N 64-bit limbs, stored most significant first. Arithmetic is mod 2^{64N}. `Big(x)` puts x in the lowest limb, zero-extended. `bit(i)` is bit i from the LSB. `==` and `<` compare values.
Time: $\mathcal{O}(N)$ per operation.

24a323, 22 lines

```
using ull = unsigned long long;
using u128 = __uint128_t;
template <int N> struct Big : array<ull, N> {
  Big(ull x = 0) : array<ull, N>{} { (*this)[N - 1] = x; }
  Big& operator+=(const Big& b) {
    ull c = 0; u128 t;
    ROF (i, 0, N) t = (u128) (*this)[i] + b[i] + c,
      (*this)[i] = (ull) t, c = (ull) (t >> 64);
    return *this;
```

```
  }
  Big operator-() const {
    auto r = *this;
    for (ull &x : r) x = ~x;
    return r += 1;
  }
  Big& operator-=(const Big &b) { return (*this) += -b; }
  friend Big operator+(Big a, const Big& b) { return a += b; }
  friend Big operator-(Big a, const Big& b) { return a -= b; }
  bool bit(int i) const { return (*this)[N-1 - i/64] >> (i%64) & 1; }
  void flip(int i) { (*this)[N-1 - i/64] ^= 1ULL << (i%64); }
  void set(int i, bool v = 1) { if (bit(i) != v) flip(i); }
};
```

11.8 Other

GrayCode.h

Description: g(i) is the i-th Gray code, rev_g its inverse. g(i) and g(i+1) differ in bit ctz($i+1$) only. e87165, 8 lines

```
int g(int n) { return n ^ (n >> 1); }

int rev_g (int g) {
  int n = 0;
  for (; g; g >>= 1)
    n ^= g;
  return n;
}
```

STL.h

Description: how to do certain things in stl: custom comparators for containers, ordering your own structs, moving containers. 0cd457, 38 lines

```
// cmp is a strict order; ties are the same key
auto cmp = [] (T a, T b) { return a < b; };
set<T, decltype(cmp)> s(cmp);
map<T, int, decltype(cmp)> m(cmp);
// pq with a<b is a max-heap; use a>b for a min-heap
priority_queue<T, vt<T>, decltype(cmp)> pq(cmp);

struct S { // comparators for structs: tie = lexicographic
  int a, b;
  bool operator<(const S &o) const {
    return tie(a, b) < tie(o.a, o.b); }
  bool operator==(const S &o) const {
    return tie(a, b) == tie(o.a, o.b); }
};
struct S2 { // C++20: all of < <= > >= == != at once
  int a, b;
  auto operator<=>(const S2 &) const = default;
};
vt<S> w; // ad hoc order without operators:
sort(all(w), [] (S x, S y) { return x.a < y.a; });

// move(x): O(1) steal of x's buffer, x left empty (reusable).
// Plain push_back(x) / v = x copy in O(size).
vt<vi> st; vi cur;
st.push_back(move(cur)); // cur is now empty
vi top = move(st.back()); st.pop_back(); // pop: move out first
// pq.top() is const&: const_cast to move (fine in libstdc++)
vi t = move(const_cast<vi&>(pq.top())); pq.pop();
swap(a, b); // O(1) for any container except array (and pbds)
if (size(a) < size(b)) swap(a, b); // small-to-large merge
// emplace(args...) builds the element in place from ctor args
vt<pi> e; e.emplace_back(1, 2); // same as push_back({1, 2})
pq.emplace(d, u); q.emplace(u); s.emplace(x); // all have it
// emplace_back(x) == push_back(x): copies unless move(x).
// Don't move: return v (auto-moved), const objects (copy).
// CPU seconds since program start (e.g. stop at 0.9*limit)
double secs = (double)clock() / CLOCKS_PER_SEC;
```

Python.py

Description: In case you need to python crutch 62 lines

```
# might not need encoding='utf-8'
# write text
with open('in', 'w', encoding='utf-8') as f:
  f.write("Hello\n")
  f.write("Line 2\n")

# append text
with open('notes.txt', 'a', encoding='utf-8') as f:
  f.write("Appended line\n")

# read text
with open('notes.txt', 'r', encoding='utf-8') as f:
  all_text = f.read()           # whole file as str
  # or iterate lines:
  f.seek(0)
  for line in f:
    print(line.strip())

from decimal import Decimal, getcontext, localcontext, ROUND_HALF_UP,
     ROUND_HALF_EVEN
from fractions import Fraction

Decimal('0.1')
Decimal('123.456')
Decimal(10)

f = Fraction(1, 3)
Decimal(f.numerator) / Decimal(f.denominator)
d = Decimal('1.23')
Decimal(d)               # returns a copy

ctx = getcontext()
print(ctx.prec)          # integer precision (significant digits)
print(ctx.rounding)      # default rounding mode

# set global precision (affects operations that follow)
getcontext().prec = 50
Decimal('2').sqrt()

import random, math
random.seed(1)                  # reproducible
random.random()                 # float in [0, 1)
random.uniform(a, b)            # float in [a, b]
random.randint(a, b)            # int in [a, b], b included
random.randrange(a, b)          # int in [a, b)
random.choice(xs)               # one element
random.sample(xs, k)            # k without replacement
random.choices(xs, k=k)         # k with replacement
random.shuffle(xs)              # in place
random.getrandbits(64)

math.factorial(n)
math.comb(n, k)                 # n choose k, exact (3.8+)
math.perm(n, k)                 # n! / (n - k)!
math.gcd(a, b), math.lcm(a, b) # lcm is 3.9+
math.isqrt(n)                   # floor(sqrt(n)), exact
pow(a, b, m)                    # pow(a, -1, m): inverse (3.8+)
divmod(a, b)                    # (a // b, a % b), floored
math.log2(x), math.log10(x), math.log(x, base)
math.floor(x), math.ceil(x), round(x, 2) # half to even
math.hypot(x, y), math.atan2(y, x)
math.inf, math.pi, math.e
# >4300-digit print: sys.set_int_max_str_digits(0)
```

11.9 Test Session

TestSession.raw

37 lines

```
vimrc/hash shortcut, F5/F6/F7, clipboard (y in vim, paste outside)
sanitizers/gdb
template.cpp
Python
Printing
Sending clarifications
Keybinds don't turn off computer
Whitespace/case/stderr insensitivity in output
Do WA on samples count towards penalty?
Return values from main
Long double/long size (LDBL_MANT_DIG == 64, sizeof(long))
Check pragmas
Check available documentation
Submit script (?)
Source code limit
Compiler version, -std, flags; extc++.h, __int128 work
Is it possible to submit and read from non-source files?
SIMD support on machine and judge
Benchmark local vs judge runtime
    ORAC: ~0.8s for N = 2e6 min segtree with Q = 2e6 loops of query +
         update
Does clock() work? If not, try Timer.h
All judge errors:
    Accepted
    WA
    Presentation Error
    RTE (what results in RTE? Does MLE exist?)
    TLE
    Memory limit, both stack and heap
    Output limit
    Illegal function
    Compile error
    Compile time limit exceeded
    Compile memory limit exceeded
    Too late
Listing all available binaries
        echo $PATH | tr ':' ' ' | xargs ls | grep -v / |
        sort | uniq | tr '\n' ' ' > path.txt
```